

Big Integer Library

0.0.1

Generated by Doxygen 1.8.17

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 bigint Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 a	6
3.1.2.2 sign	6
3.1.2.3 wordlen	6
4 File Documentation	7
4.1 include/arithmic.h File Reference	7
4.1.1 Detailed Description	9
4.1.2 Function Documentation	9
4.1.2.1 add()	9
4.1.2.2 add_core()	11
4.1.2.3 add_single_word()	12
4.1.2.4 barret_reduction()	13
4.1.2.5 bi_long_div()	14
4.1.2.6 bigint_or()	16
4.1.2.7 exp_mod_montgomery()	17
4.1.2.8 general_div_core()	19
4.1.2.9 L2R()	20
4.1.2.10 mul_core_improved()	22
4.1.2.11 mul_core_karatsuba()	23
4.1.2.12 mul_core_tx()	25
4.1.2.13 mul_single_word()	27
4.1.2.14 quotient()	27
4.1.2.15 R2L()	28
4.1.2.16 squ_karatsuba()	31
4.1.2.17 squ_single_word()	33
4.1.2.18 squ_tx()	34
4.1.2.19 squaring()	35
4.1.2.20 sub()	37
4.1.2.21 sub_core()	38
4.1.2.22 sub_single_word()	39
4.2 include/bigint.h File Reference	40
4.2.1 Detailed Description	42
4.2.2 Macro Definition Documentation	42

4.2.2.1 MAXIMUM	42
4.2.2.2 MINIMUM	43
4.2.2.3 ONE	43
4.2.2.4 WORD_BITLEN	44
4.2.2.5 ZERO	44
4.2.3 Typedef Documentation	44
4.2.3.1 word	44
4.3 include/config.h File Reference	45
4.3.1 Detailed Description	46
4.3.2 Macro Definition Documentation	46
4.3.2.1 FLAG	46
4.3.2.2 IN	47
4.3.2.3 INOUT	47
4.3.2.4 NEGATIVE	47
4.3.2.5 NON_NEGATIVE	47
4.3.2.6 OUT	48
4.3.3 Typedef Documentation	48
4.3.3.1 i32	48
4.3.3.2 i64	48
4.3.3.3 i8	48
4.3.3.4 u32	48
4.3.3.5 u64	49
4.3.3.6 u8	49
4.4 include/test.h File Reference	49
4.4.1 Detailed Description	51
4.4.2 Macro Definition Documentation	51
4.4.2.1 MAX_BIT_LEN	51
4.4.2.2 TEST_CASE	51
4.4.2.3 TIME	52
4.4.3 Function Documentation	52
4.4.3.1 measure_clock_cycles()	52
4.4.3.2 measure_cycles()	53
4.4.3.3 print_bi_hex_py()	54
4.4.3.4 test_barret()	55
4.4.3.5 test_basic_operation()	57
4.4.3.6 test_div()	58
4.4.3.7 test_exp_mod()	59
4.4.3.8 test_squ()	60
4.5 include/utils.h File Reference	61
4.5.1 Detailed Description	63
4.5.2 Macro Definition Documentation	63
4.5.2.1 INVAILD_DATA	64

4.5.2.2 MEM_ALLOCATION_FAIL	64
4.5.2.3 SET_ARRAY_FAIL	65
4.5.2.4 SET_STRING_FAIL	65
4.5.3 Function Documentation	65
4.5.3.1 array_init()	65
4.5.3.2 array_rand()	67
4.5.3.3 bi_assign()	68
4.5.3.4 bi_delete()	69
4.5.3.5 bi_gen_rand()	71
4.5.3.6 bi_new()	73
4.5.3.7 bi_refine()	74
4.5.3.8 bi_refine_word()	75
4.5.3.9 bi_reset()	76
4.5.3.10 bi_set_by_array()	77
4.5.3.11 bi_set_by_string()	78
4.5.3.12 bi_set_one()	79
4.5.3.13 bi_set_zero()	79
4.5.3.14 bi_show_hex()	80
4.5.3.15 compare()	81
4.5.3.16 compareABS()	82
4.5.3.17 get_bit_length()	83
4.5.3.18 get_jth_bit()	84
4.5.3.19 get_word()	84
4.5.3.20 is_zero()	85
4.5.3.21 left_shift_bit()	86
4.5.3.22 left_shift_word()	87
4.5.3.23 makeEven()	88
4.5.3.24 match_wordlen()	89
4.5.3.25 reduction()	91
4.5.3.26 right_shift_bit()	92
4.5.3.27 right_shift_word()	92
4.5.3.28 swap_bigint()	93
4.6 src/arithmetic.c File Reference	94
4.6.1 Function Documentation	95
4.6.1.1 add()	95
4.6.1.2 add_core()	97
4.6.1.3 add_single_word()	98
4.6.1.4 barret_reduction()	99
4.6.1.5 bi_long_div()	100
4.6.1.6 bigint_or()	102
4.6.1.7 exp_mod_montgomery()	103
4.6.1.8 general_div_core()	105

4.6.1.9 L2R()	106
4.6.1.10 mul_core_improved()	108
4.6.1.11 mul_core_karatsuba()	109
4.6.1.12 mul_core_tx()	111
4.6.1.13 mul_single_word()	113
4.6.1.14 quotient()	113
4.6.1.15 R2L()	114
4.6.1.16 squ_karatsuba()	117
4.6.1.17 squ_single_word()	119
4.6.1.18 squ_tx()	120
4.6.1.19 squaring()	121
4.6.1.20 sub()	123
4.6.1.21 sub_core()	124
4.6.1.22 sub_single_word()	126
4.7 src/bigint.c File Reference	127
4.8 src/main.c File Reference	127
4.8.1 Function Documentation	128
4.8.1.1 main()	128
4.9 src/utils.c File Reference	129
4.9.1 Function Documentation	131
4.9.1.1 array_init()	131
4.9.1.2 array_rand()	133
4.9.1.3 bi_assign()	133
4.9.1.4 bi_delete()	134
4.9.1.5 bi_gen_rand()	136
4.9.1.6 bi_new()	138
4.9.1.7 bi_refine()	139
4.9.1.8 bi_refine_word()	140
4.9.1.9 bi_reset()	141
4.9.1.10 bi_set_by_array()	142
4.9.1.11 bi_set_by_string()	143
4.9.1.12 bi_set_one()	144
4.9.1.13 bi_set_zero()	144
4.9.1.14 bi_show_hex()	145
4.9.1.15 compare()	146
4.9.1.16 compareABS()	147
4.9.1.17 get_bit_length()	148
4.9.1.18 get_jth_bit()	149
4.9.1.19 get_word()	149
4.9.1.20 is_zero()	150
4.9.1.21 left_shift_bit()	151
4.9.1.22 left_shift_word()	152

4.9.1.23 makeEven()	153
4.9.1.24 match_wordlen()	154
4.9.1.25 reduction()	156
4.9.1.26 right_shift_bit()	157
4.9.1.27 right_shift_word()	157
4.9.1.28 swap_bigint()	158
4.10 test/test.c File Reference	159
4.10.1 Function Documentation	160
4.10.1.1 measure_clock_cycles()	160
4.10.1.2 measure_cycles()	161
4.10.1.3 print_bi_hex_py()	161
4.10.1.4 test_barret()	162
4.10.1.5 test_basic_operation()	164
4.10.1.6 test_div()	165
4.10.1.7 test_exp_mod()	166
4.10.1.8 test_squ()	167

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

bigint	Represents a big integer for arbitrary-precision arithmetic	5
------------------------	---	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

include/	arithmetic.h	
	This file contains the function declarations for various arithmetic operations on big integers, such as addition, subtraction, multiplication, squaring, division, and modular exponentiation	7
include/	bigint.h	
	This file defines the data structures and macros for representing and manipulating big integers	40
include/	config.h	
	This file contains configuration settings, type definitions, and necessary includes for the project	45
include/	test.h	
	This file declares the functions and definitions used for testing and measuring the performance of various arithmetic operations in the BigInt library	49
include/	utils.h	
	This file provides utility functions for managing and manipulating big integers (bigint type), including memory allocation, bitwise operations, shifting, and random number generation	61
src/	arithmetic.c	94
src/	bigint.c	127
src/	main.c	127
src/	utils.c	129
test/	test.c	159

Chapter 3

Class Documentation

3.1 `bigint` Struct Reference

Represents a big integer for arbitrary-precision arithmetic.

```
#include <bigint.h>
```

Public Attributes

- `int sign`
- `int wordlen`
- `word * a`

3.1.1 Detailed Description

Represents a big integer for arbitrary-precision arithmetic.

The `bigint` structure contains:

- `sign`: The sign of the integer (`NEGATIVE` or `NON_NEGATIVE`).
- `wordlen`: The number of words used to represent the integer (must be non-negative).
- `a`: A pointer to an array of `word` elements that store the integer's value.

Definition at line 67 of file `bigint.h`.

3.1.2 Member Data Documentation

3.1.2.1 a

```
word* bigint::a
```

Pointer to the array storing the big integer's data.

Definition at line 70 of file bigint.h.

3.1.2.2 sign

```
int bigint::sign
```

The sign of the big integer (NEGATIVE or NON_NEGATIVE).

Definition at line 68 of file bigint.h.

3.1.2.3 wordlen

```
int bigint::wordlen
```

The length of the a array in words.

Definition at line 69 of file bigint.h.

The documentation for this struct was generated from the following file:

- include/[bigint.h](#)

Chapter 4

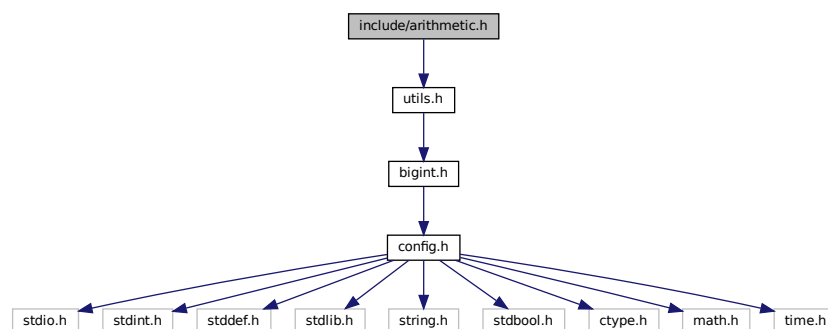
File Documentation

4.1 include/arithmetic.h File Reference

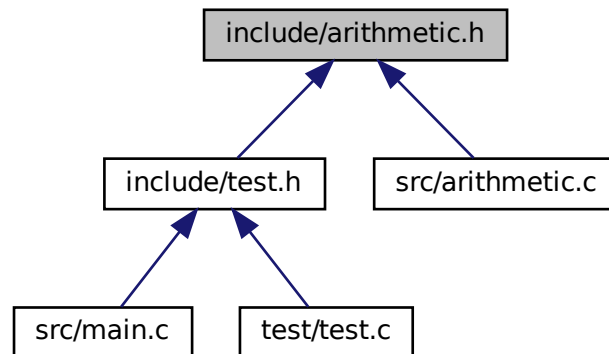
This file contains the function declarations for various arithmetic operations on big integers, such as addition, subtraction, multiplication, squaring, division, and modular exponentiation.

```
#include "utils.h"
```

Include dependency graph for arithmetic.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `bigint_or` (IN bigint **x, IN bigint **y, OUT bigint **z)
Performs a bitwise OR operation on two binary integers.
- void `add_single_word` (IN word A, IN word B, IN word carry_in, OUT word *carry_out, OUT word *result)
Adds two words with carry handling.
- void `add_core` (IN bigint **x, IN bigint **y, OUT bigint **z)
Adds two big integers and stores the result in a third bigint.
- void `add` (IN bigint **x, IN bigint **y, OUT bigint **z)
Adds two big integers x and y, and stores the result in z.
- void `sub_single_word` (IN word A, IN word B, IN word borrow_in, OUT word *borrow_out, OUT word *result)
Performs subtraction of B from A and manages borrowing.
- void `sub_core` (IN bigint **x, IN bigint **y, OUT bigint **z)
Performs subtraction of two big integers.
- void `sub` (IN bigint **x, IN bigint **y, OUT bigint **z)
Subtracts big integer y from big integer x and stores the result in z.
- void `mul_single_word` (IN word A, IN word B, OUT bigint **result)
Multiplies two single words and stores the result in a bigint structure.
- void `mul_core_tx` (IN bigint **x, IN bigint **y, OUT bigint **z)
Multiplies two big integers and stores the result in the output.
- void `mul_core_improved` (IN bigint **x, IN bigint **y, OUT bigint **z)
Multiplies two binary integers using an improved Textbook Multiplication algorithm.
- void `mul_core_karatsuba` (IN bigint **x, IN bigint **y, OUT bigint **z)
Multiplies two binary integers using the Karatsuba multiplication algorithm.
- void `squ_single_word` (IN word A, OUT bigint **result)
Squares a single word and stores the result in a bigint structure.
- void `squ_tx` (IN bigint **x, OUT bigint **result)
Squares a bigint and stores the result in another bigint.
- void `squaring` (IN bigint **x, OUT bigint **result)
Squares a bigint if it is non-zero and stores the result.
- void `squ_karatsuba` (IN bigint **x, OUT bigint **z)

- Computes the square of a binary integer using the Karatsuba squaring algorithm.*
- void `bi_long_div` (IN bigint **x, IN bigint **y, OUT bigint **q, OUT bigint **r)
Performs binary long division on big integers.
- `word quotient` (IN word dividend1, IN word dividend0, IN word divisor)
Computes the quotient of a two-word dividend divided by a single-word divisor.
- void `general_div_core` (IN bigint **x, IN bigint **y, IN bigint **Q, IN bigint **R)
Performs division of two large integers using an optimized algorithm.
- void `L2R` (IN bigint **x, IN bigint **y, IN bigint **z, OUT bigint **M)
Computes modular exponentiation using the Left-to-Right binary method.
- void `R2L` (IN bigint **x, IN bigint **y, OUT bigint **z, IN bigint **M)
Computes modular exponentiation using the Right-to-Left binary method.
- void `exp_mod_montgomery` (IN bigint **x, IN bigint **y, OUT bigint **z, IN bigint **M)
Performs modular exponentiation using Montgomery multiplication.
- void `barret_reduction` (IN bigint **x, IN bigint **y, IN bigint **z, OUT bigint **result)
Performs Barrett reduction to reduce x modulo y .

4.1.1 Detailed Description

This file contains the function declarations for various arithmetic operations on big integers, such as addition, subtraction, multiplication, squaring, division, and modular exponentiation.

The operations are implemented for big integers and include both basic and optimized algorithms (e.g., Karatsuba multiplication, Barrett reduction, etc.). These functions handle various core arithmetic tasks required for large integer computations.

Operations include:

- Addition and subtraction of big integers
- Multiplication (with multiple algorithms such as Karatsuba)
- Squaring of big integers
- Division (both core and modular)
- Modular exponentiation and reduction (e.g., Montgomery and Barrett)

4.1.2 Function Documentation

4.1.2.1 `add()`

```
void add (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Adds two big integers `x` and `y`, and stores the result in `z`.

The function handles cases where one or both of the operands are zero, as well as when the signs of `x` and `y` are different. If the signs of `x` and `y` are different, it performs subtraction instead. If both have the same sign, it performs addition.

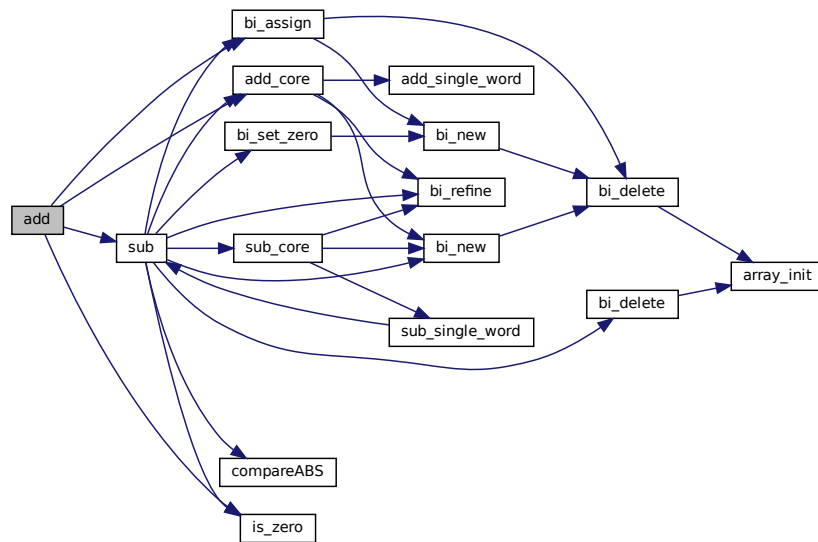
Parameters

in	x	Pointer to a pointer of the first bigint structure.
in	y	Pointer to a pointer of the second bigint structure.
out	z	Pointer to a pointer of the result bigint structure.

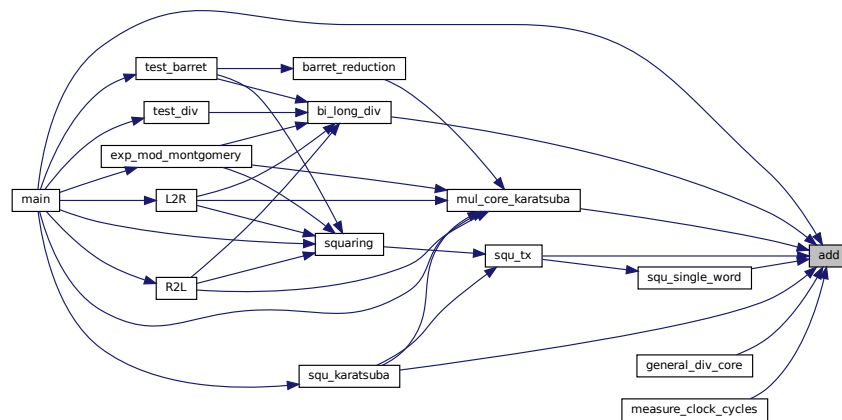
- If x is zero, the function assigns y to z.
- If y is zero, the function assigns x to z.
- If x and y have opposite signs, the function subtracts the smaller magnitude from the larger one.
- If x and y have the same sign, the function adds them and assigns the resulting sign to z.

Definition at line 79 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.2 add_core()

```
void add_core (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Adds two big integers and stores the result in a third bigint.

This function takes two big integers represented as `bigint` structures, adds them together, and stores the resulting bigint in a third `bigint`. It ensures that both numbers have the same length by padding with zeros if necessary. The addition is performed in reverse order to handle carry appropriately.

Parameters

in	<i>x</i>	Pointer to the first bigint to add.
in	<i>y</i>	Pointer to the second bigint to add.
out	<i>z</i>	Pointer to the result bigint where the sum will be stored.

This function performs the following steps:

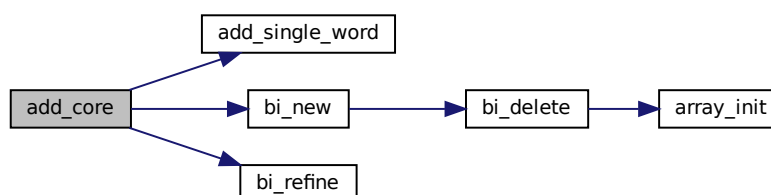
- Determines the lengths of the input big integers.
- Allocates memory for the result bigint based on the maximum length of the inputs.
- Iteratively adds each corresponding word from the two input big integers, starting from the least significant word, while handling any carry from previous additions.
- Stores the result in the appropriate position in the result bigint.
- Handles any carry that may remain after the final addition.
- Calls `bi_refine` to remove any leading zeros from the result.

Note

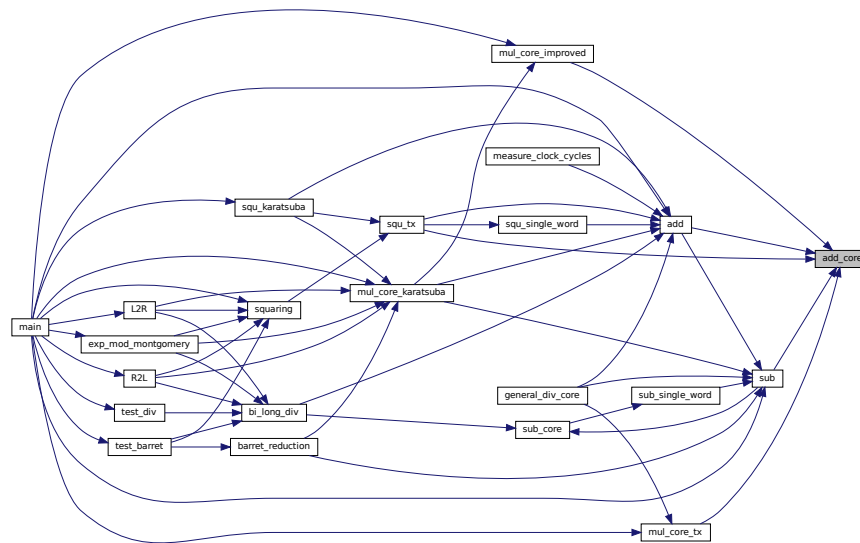
The input big integers should be properly initialized and should not be NULL. The result bigint will be allocated and must be freed by the caller when no longer needed.

Definition at line 46 of file `arithmetic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.3 add_single_word()

```
void add_single_word (
    IN word A,
    IN word B,
    IN word carry_in,
    OUT word * carry_out,
    OUT word * result )
```

Adds two words with carry handling.

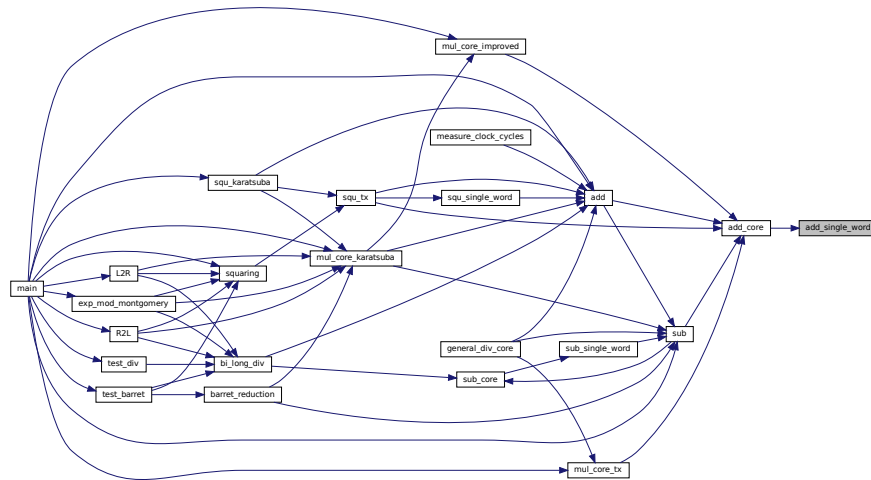
This function performs addition on two words (A and B) and incorporates a carry from a previous addition. It checks for overflow and sets the carry_out accordingly.

Parameters

in	<i>A</i>	First word to add.
in	<i>B</i>	Second word to add.
in	<i>carry_in</i>	Carry input from a previous addition.
out	<i>carry_out</i>	Pointer to store the carry output.
out	<i>result</i>	Pointer to store the result of the addition.

Definition at line 28 of file arithmetic.c.

Here is the caller graph for this function:



4.1.2.4 barret_reduction()

```

void barret_reduction (
    IN bigint ** x,
    IN bigint ** y,
    IN bigint ** z,
    OUT bigint ** result )

```

Performs Barrett reduction to reduce x modulo y .

This function computes the result of $(x \bmod y)$ using the Barrett reduction method. Barrett reduction is efficient for modular operations, especially when dealing with repeated modular reductions during large integer arithmetic (e.g., in cryptographic algorithms). It involves approximating the inverse of y modulo a power of two and using it to reduce x modulo y .

Parameters

in	x	A double pointer to a <code>bigint</code> representing the dividend.
in	y	A double pointer to a <code>bigint</code> representing the divisor.
in	z	A double pointer to a <code>bigint</code> representing the precomputed value used in the reduction.
out	$result$	A double pointer to a <code>bigint</code> where the result of the reduction will be stored.

Precondition

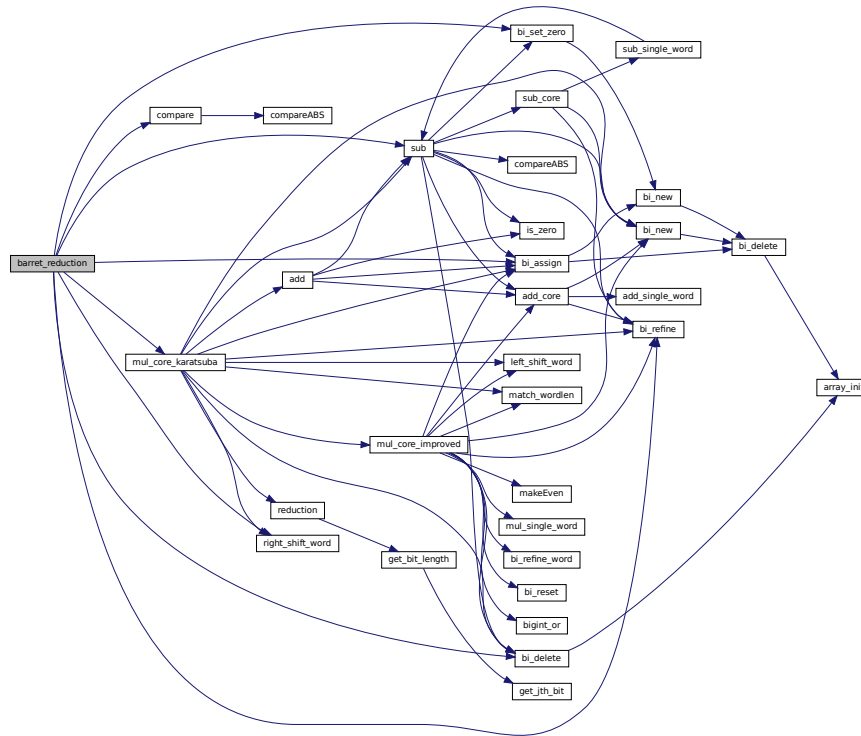
Both x and y must point to valid, initialized `bigint` objects. y must not be zero.

Postcondition

The result of $(x \bmod y)$ is stored in the location pointed to by $result$.

The function utilizes an approximation method to compute the modular result more efficiently. The reduction steps are performed using several intermediate variables to prevent overflow and optimize calculations.

Here is the call graph for this function:



```
void bi_long_div (
    IN bigint ** x,
    IN bigint ** y,
```

```
OUT bigint ** q,
OUT bigint ** r )
```

Performs binary long division on big integers.

This function calculates the quotient (q) and remainder (r) when dividing bigint $*x$ by bigint $*y$. It handles special cases for zero and negative values and manages sign adjustments for the result.

Parameters

in, out	x	Pointer to the dividend bigint. This value is read-only in the function.
in	y	Pointer to the divisor bigint. This value is read-only in the function.
out	q	Pointer to the quotient bigint, which will store the result of the division.
out	r	Pointer to the remainder bigint, which will store the remainder after division.

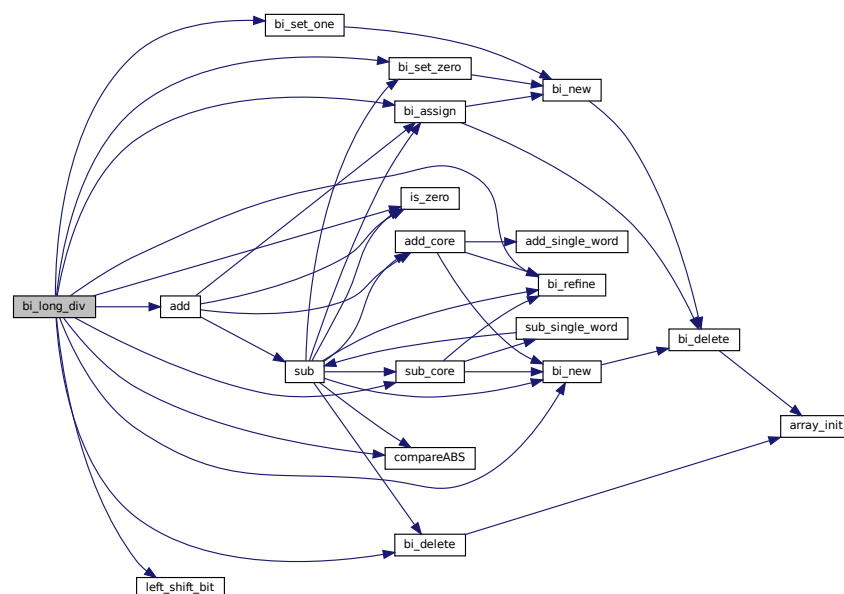
- If the divisor $*y$ is zero, the function triggers an error since division by zero is undefined.
- If $*x$ is zero, both quotient $*q$ and remainder $*r$ are set to zero.
- If the absolute value of $*x$ is smaller than $*y$, $*q$ is set to zero and $*r$ to $*x$.
- If the absolute value of $*x$ equals $*y$, $*q$ is set to one and $*r$ to zero, with sign adjustments if necessary.
- The main division loop shifts bits from $*x$ into $*r$, checks if $*r \geq *y$, and updates $*q$ and $*r$ accordingly.
- For negative dividends, the remainder $*r$ is adjusted to ensure the remainder sign follows division rules.

Note

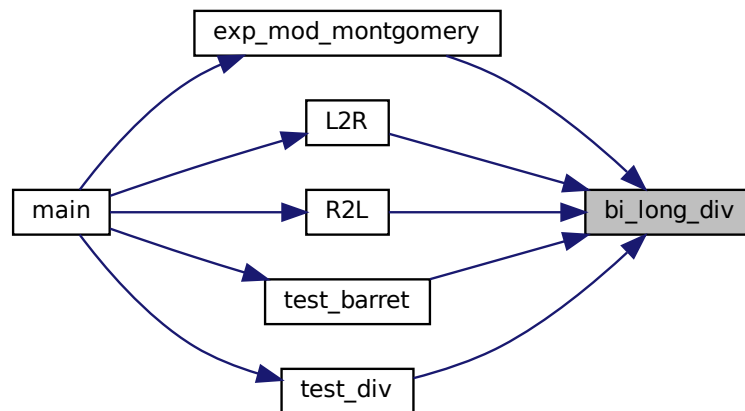
- The function uses helper functions like `bi_set_zero`, `bi_assign`, `add`, `sub_core`, and `left_shift_bit` to perform operations on bigint structures.
- The sign of the quotient $*q$ and remainder $*r$ is adjusted based on the signs of $*x$ and $*y$.
- The function assumes bigint pointers x , y , q , and r are properly initialized before calling.

Definition at line 623 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.6 bigint_or()

```

void bigint_or (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
  
```

Performs a bitwise OR operation on two binary integers.

This function computes the bitwise OR of two `bigint` objects `x` and `y`, and stores the result in `z`. It handles word-level operations and ensures that the result's word length and sign are properly set.

Parameters

in	<code>x</code>	A double pointer to a <code>bigint</code> representing the first operand.
in	<code>y</code>	A double pointer to a <code>bigint</code> representing the second operand.
out	<code>z</code>	A double pointer to a <code>bigint</code> where the result of the bitwise OR operation will be stored.

Precondition

`x` and `y` must point to valid `bigint` objects with properly initialized word lengths and data arrays. `z` must be properly allocated to store the resulting `bigint`.

Postcondition

The bitwise OR result of `(*x)` and `(*y)` is stored in `(*z)`. The word length of `(*z)` is set to the maximum of the word lengths of `(*x)` and `(*y)`. If the signs of `(*x)` and `(*y)` differ, the sign of `(*z)` is set to `NEGATIVE`.

Note

This function assumes that the `bigint` structure has attributes `a` (an array of words), `wordlen` (length of the array), and `sign` (the sign of the integer).

4.1.2.7 exp_mod_montgomery()

```
void exp_mod_montgomery (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z,
    IN bigint ** M )
```

Performs modular exponentiation using Montgomery multiplication.

This function computes $(z = x^y \bmod M)$ using the Montgomery multiplication method. It iterates over the bits of the exponent `y` from the most significant bit to the least significant bit, maintaining intermediate modular reductions to ensure efficient computation.

Parameters

in	<code>x</code>	A double pointer to a <code>bigint</code> representing the base.
in	<code>y</code>	A double pointer to a <code>bigint</code> representing the exponent.
out	<code>z</code>	A double pointer to a <code>bigint</code> where the result will be stored.
in	<code>M</code>	A double pointer to a <code>bigint</code> representing the modulus.

Precondition

Both `x`, `y`, and `M` must point to valid, initialized `bigint` objects. `M` must be non-zero.

Postcondition

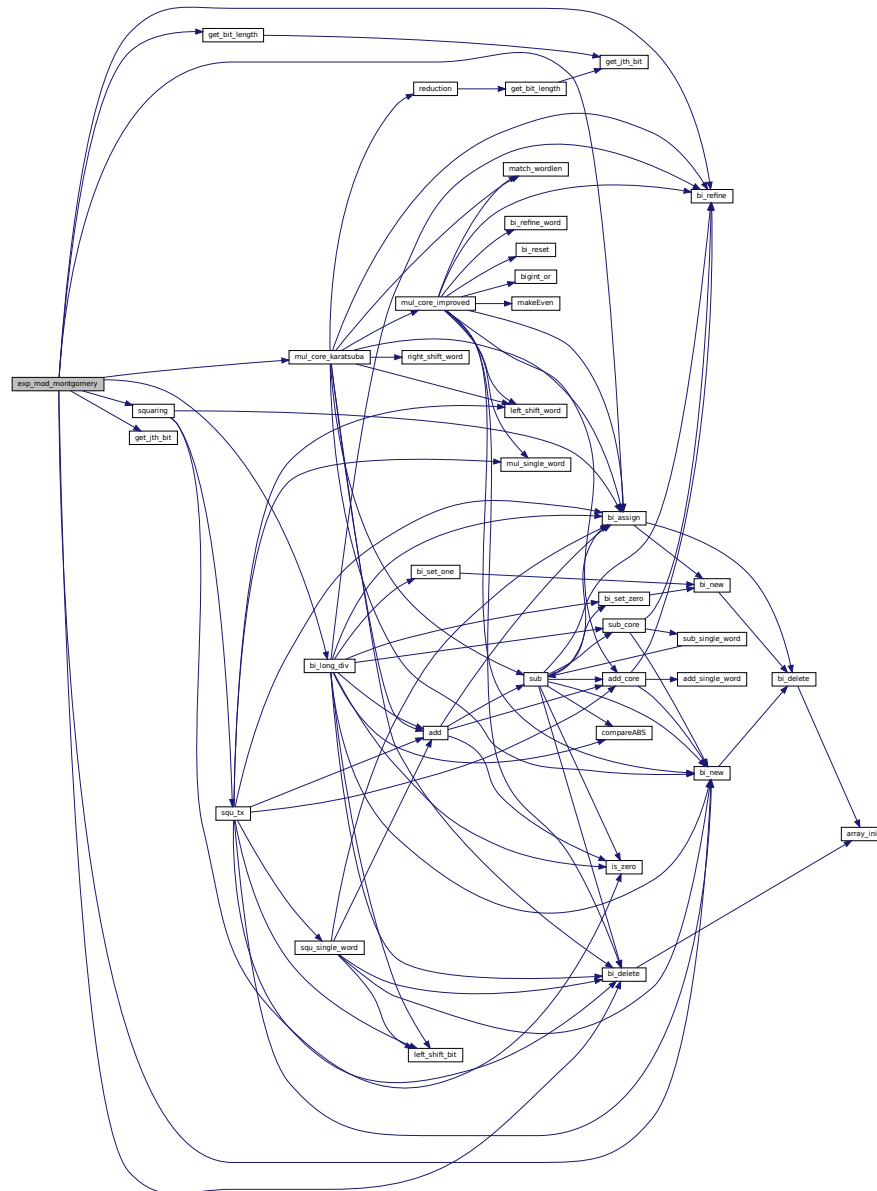
The result $(z = x^y \bmod M)$ is stored in the location pointed to by `z`.

Note

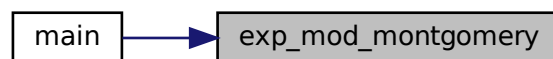
This implementation uses Montgomery multiplication for efficient modular reduction at each step.

Definition at line 913 of file `arithmetic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.8 `general_div_core()`

```
void general_div_core (
    IN bigint ** x,
    IN bigint ** y,
    IN bigint ** Q,
    IN bigint ** R )
```

Performs division of two large integers using an optimized algorithm.

This function calculates the quotient (Q) and remainder (R) when a large integer x is divided by y . It handles cases where the word lengths of x and y differ by at most one and adjusts results to ensure the remainder (R) is non-negative. This function also refines results to account for any adjustments due to negative remainders.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the dividend.
in	y	A double pointer to a <code>bigint</code> representing the divisor.
out	Q	A double pointer to a <code>bigint</code> where the quotient will be stored.
out	R	A double pointer to a <code>bigint</code> where the remainder will be stored.

Precondition

Both x and y must point to valid, initialized `bigint` objects, and y must not be zero. Q and R must be properly allocated to store the resulting values.

Postcondition

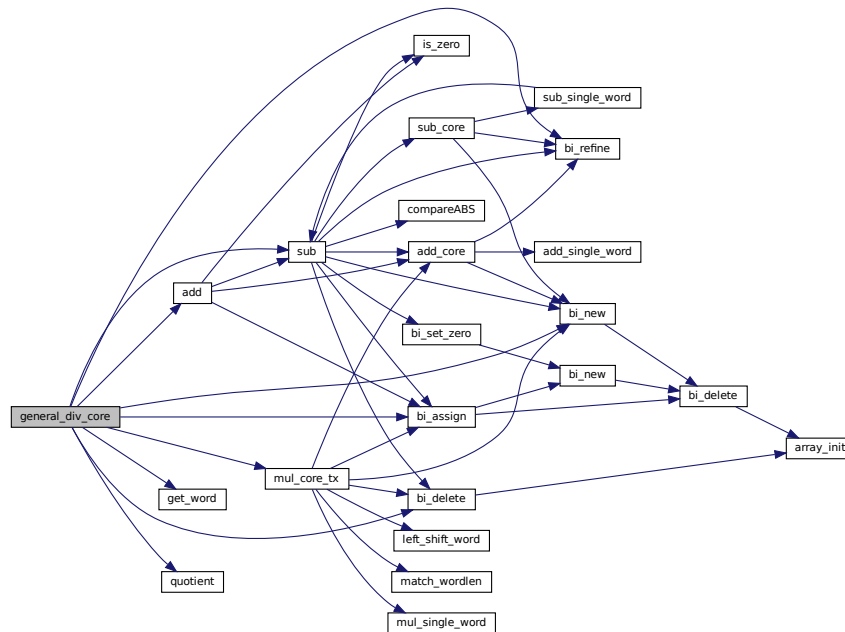
The quotient and remainder of x / y are stored in Q and R , respectively. The remainder R is guaranteed to be non-negative.

Note

This function uses helper functions such as `mul_core_tx` for multiplication, `sub` for subtraction, and `add` for addition. Memory management and refinement (`bi_refine`) are handled to ensure intermediate results are properly cleaned.

Definition at line 769 of file `arithmetic.c`.

Here is the call graph for this function:



4.1.2.9 L2R()

```
void L2R (
    IN bigint ** x,
    IN bigint ** y,
    IN bigint ** z,
    OUT bigint ** M )
```

Computes modular exponentiation using the Left-to-Right binary method.

This function calculates $(z = x^y \bmod M)$ efficiently using a left-to-right binary exponentiation algorithm. It processes the bits of y from the most significant to the least significant bit, squaring the intermediate result and conditionally multiplying based on the value of the current bit.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the base.
in	y	A double pointer to a <code>bigint</code> representing the exponent.
out	z	A double pointer to a <code>bigint</code> where the result will be stored.
in	M	A double pointer to a <code>bigint</code> representing the modulus.

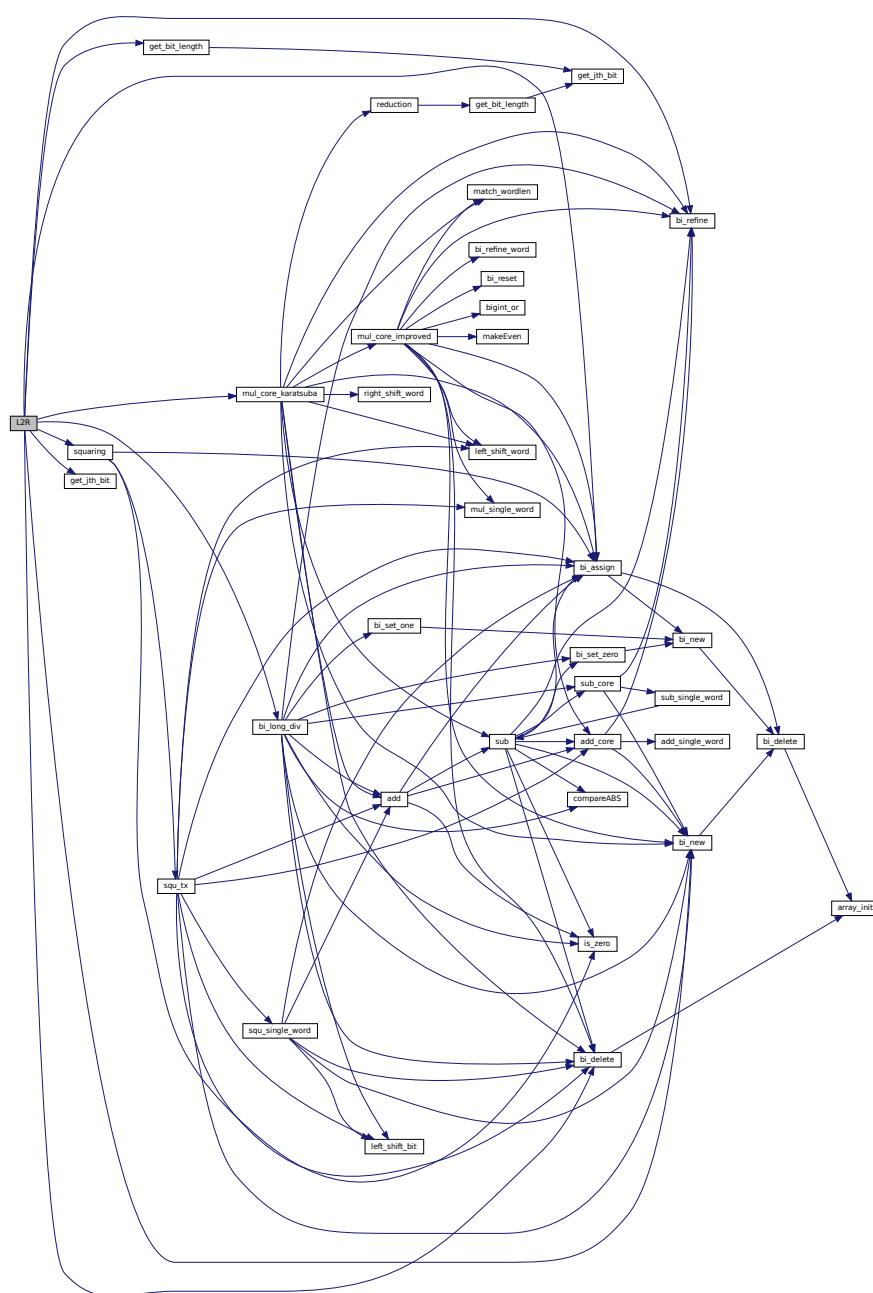
Precondition

Both x , y , and M must point to valid, initialized `bigint` objects. M must be non-zero.

Note

This function uses modular reduction during every squaring and multiplication step to ensure that intermediate results remain manageable in size.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.10 mul_core_improved()

```

void mul_core_improved (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
  
```

Multiplies two binary integers using an improved Textbook Multiplication algorithm.

Wrapper function that performs optimized multiplication of two BINTs pointed to by `x` and `y`, and stores the result in `z`. This function improves the standard Textbook Multiplication by aligning word lengths, splitting calculations into smaller chunks, and handling shifts efficiently.

Parameters

in	<code>x</code>	A double pointer to a BINT representing the first operand.
in	<code>y</code>	A double pointer to a BINT representing the second operand.
out	<code>z</code>	A double pointer to a BINT where the result will be stored.

Precondition

Both `x` and `y` must point to valid, initialized BINT objects. `z` must be properly allocated to store the resulting BINT.

Postcondition

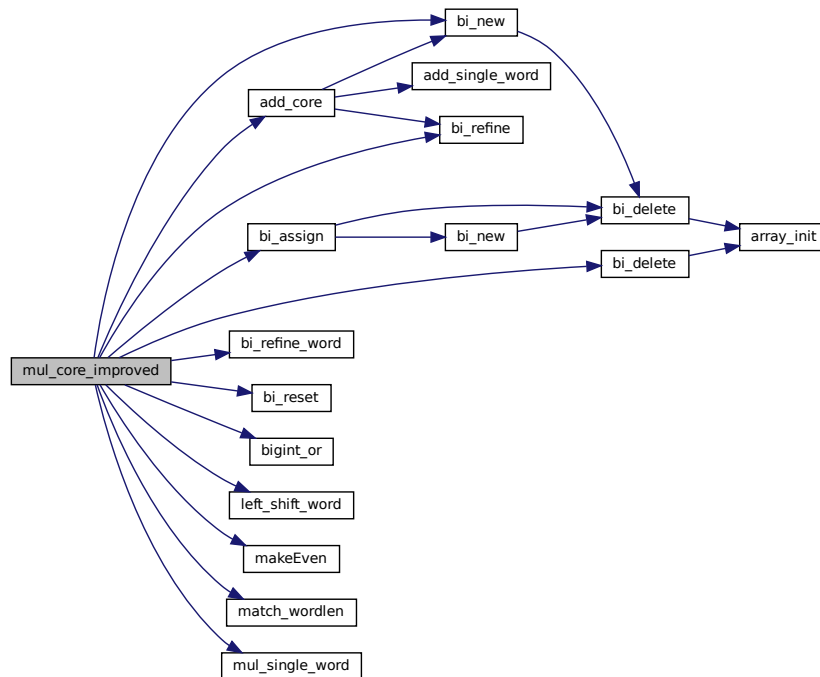
The result of multiplication is stored in the location pointed to by `z`.

Note

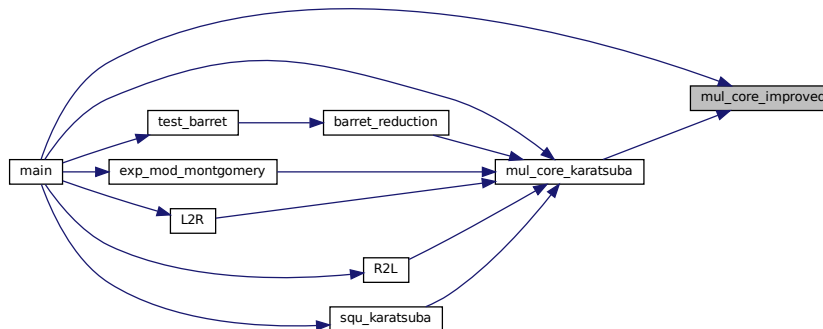
This function uses multiple helper functions, such as `mul_single_word`, `add_core`, and `left_↔shift_word`, to manage BINT arithmetic and memory efficiently.

Definition at line 326 of file `arithmetic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.11 mul_core_karatsuba()

```

void mul_core_karatsuba (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
  
```

Multiplies two binary integers using the Karatsuba multiplication algorithm.

This function performs efficient multiplication of two binary integers (x and y) using the divide-and-conquer Karatsuba algorithm. It recursively splits the inputs into higher and lower parts, computes intermediate products, and combines them to calculate the final result. For small input sizes, it falls back to an improved standard multiplication (`mul_core_improved`) for efficiency.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the first operand.
in	y	A double pointer to a <code>bigint</code> representing the second operand.
out	z	A double pointer to a <code>bigint</code> where the result of the multiplication will be stored.

Precondition

Both x and y must point to valid, initialized `bigint` objects. z must be properly allocated to store the resulting `bigint`.

Postcondition

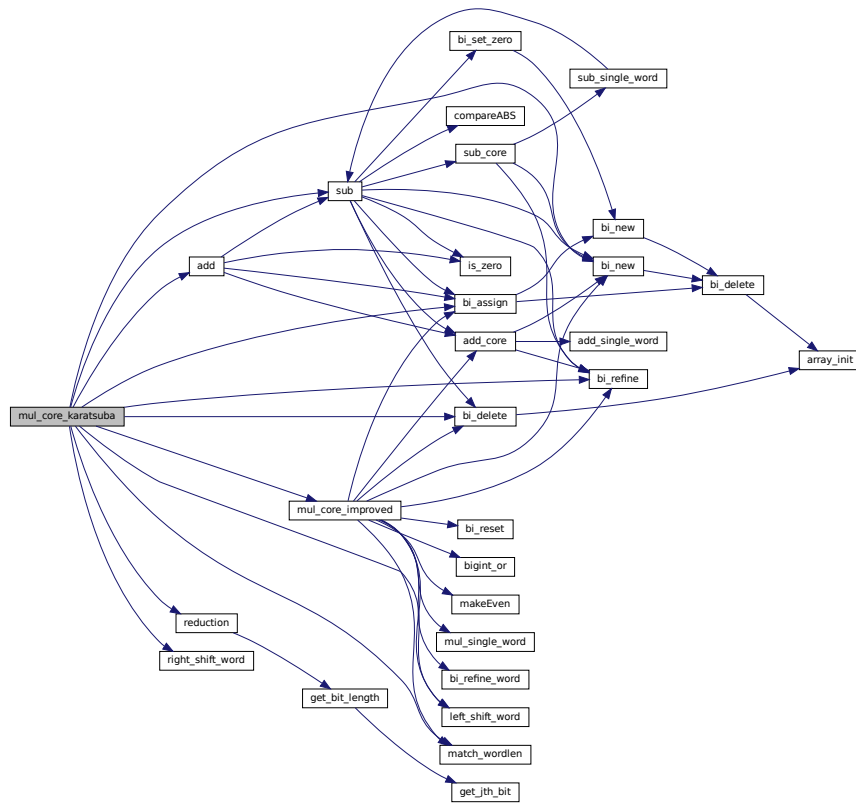
The result of the multiplication is stored in the location pointed to by z . Both x and y are refined back to their original states.

Note

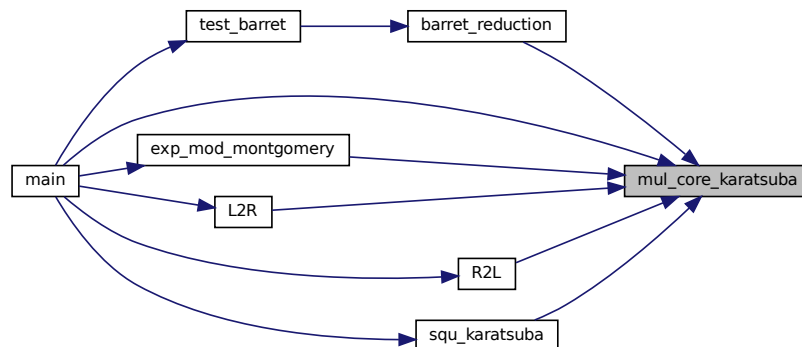
This function employs memory-efficient techniques for intermediate calculations, and uses helper functions such as `right_shift_word`, `reduction`, and `add` to manage the binary integer arithmetic.

Definition at line 399 of file `arithmetic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.12 mul_core_tx()

```

void mul_core_tx (
    IN bigint ** x,

```

```

    IN bigint ** y,
    OUT bigint ** z )

```

Multiplies two big integers and stores the result in the output.

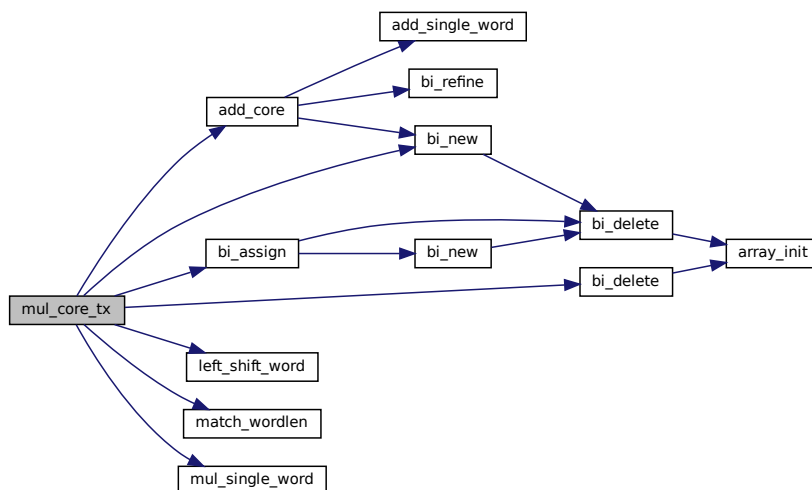
This function performs multiplication of two big integers, x and y , and stores the result in z . It adjusts for the sign of the result and handles multiplication at the word level with shifting and addition.

Parameters

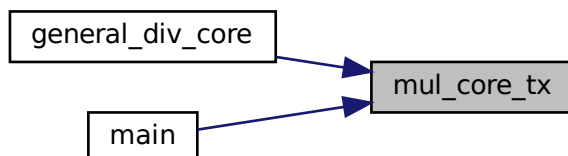
in	x	Pointer to a pointer of the first bigint operand.
in	y	Pointer to a pointer of the second bigint operand.
out	z	Pointer to a pointer of the resulting bigint product.

Definition at line 295 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.13 mul_single_word()

```
void mul_single_word (
    IN word A,
    IN word B,
    OUT bigint ** result )
```

Multiplies two single words and stores the result in a bigint structure.

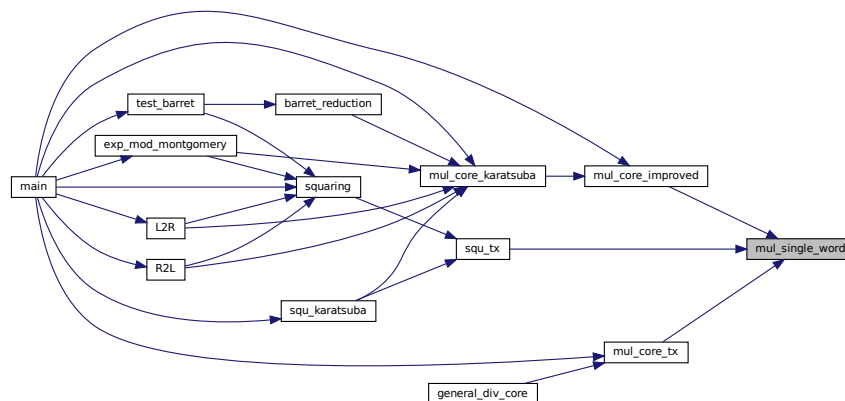
This function performs multiplication on two input words, A and B , and stores the result as a two-word bigint in the `result` parameter. It breaks down the multiplication into half-word operations to manage potential overflow in high and low parts separately, allowing for more precise handling of large word multiplications.

Parameters

in	<i>A</i>	The first input word.
in	<i>B</i>	The second input word.
out	<i>result</i>	Pointer to a bigint structure where the result will be stored. The bigint structure is expected to have space for two words (<code>a[0]</code> for the lower part and <code>a[1]</code> for the upper part).

Definition at line 266 of file arithmetic.c.

Here is the caller graph for this function:



4.1.2.14 quotient()

```
word quotient (
    IN word dividend1,
    IN word dividend0,
    IN word divisor )
```

Computes the quotient of a two-word dividend divided by a single-word divisor.

This function performs high-precision division using a two-word dividend (`dividend1` as the high word and `dividend0` as the low word) and a single-word divisor. The algorithm calculates the quotient (Q) and ensures that intermediate remainders (R) are handled appropriately during bit-by-bit processing.

Parameters

in	<i>dividend1</i>	The higher significant word of the two-word dividend.
in	<i>dividend0</i>	The lower significant word of the two-word dividend.
in	<i>divisor</i>	A single-word divisor.

Returns

The quotient resulting from the division of the two-word dividend by the divisor.

Precondition

`divisor` must be non-zero to avoid division by zero errors.

Postcondition

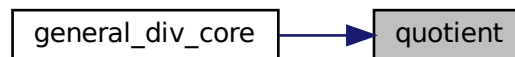
The returned quotient is the result of the division, and intermediate calculations ensure correctness.

Note

This function processes the division bit-by-bit, starting from the most significant bits and adjusting the remainder (R) and quotient (Q) iteratively.

Definition at line 739 of file arithmetic.c.

Here is the caller graph for this function:

**4.1.2.15 R2L()**

```

void R2L (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z,
    IN bigint ** M )
  
```

Computes modular exponentiation using the Right-to-Left binary method.

This function calculates $(z = x^y \bmod M)$ using the right-to-left binary exponentiation algorithm. It iterates over the bits of y from the least significant to the most significant, updating the intermediate result and modular reduction efficiently at each step.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the base.
in	y	A double pointer to a <code>bigint</code> representing the exponent.
out	z	A double pointer to a <code>bigint</code> where the result will be stored.
in	M	A double pointer to a <code>bigint</code> representing the modulus.

Precondition

Both x , y , and M must point to valid, initialized `bigint` objects. M must be non-zero.

Postcondition

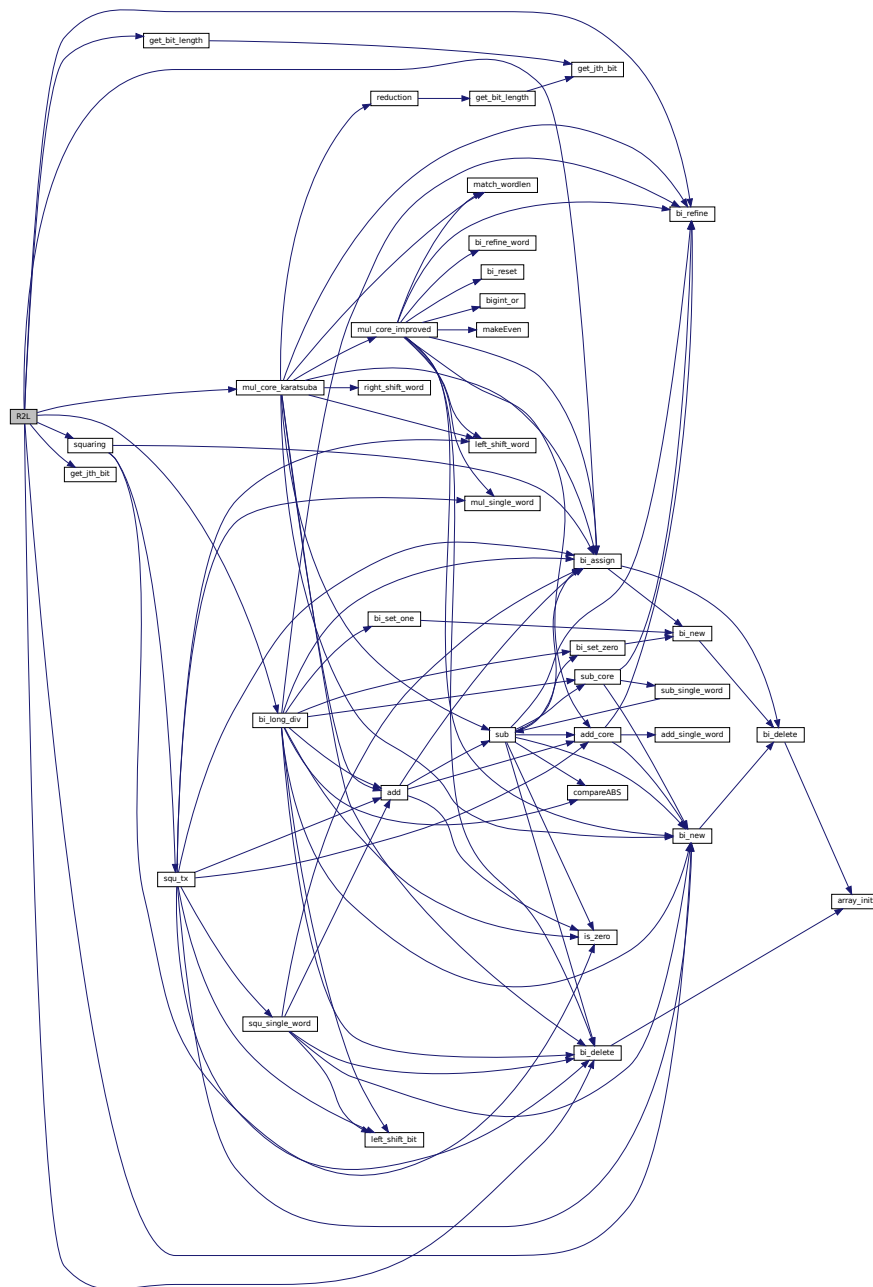
The result ($z = x^y \bmod M$) is stored in the location pointed to by z .

Note

This function employs modular reduction during each multiplication and squaring step to maintain manageable intermediate values and optimize performance.

Definition at line 877 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.16 `squ_karatsuba()`

```
void squ_karatsuba (
    IN bigint ** x,
    OUT bigint ** z )
```

Computes the square of a binary integer using the Karatsuba squaring algorithm.

This function efficiently computes the square of a binary integer `x` by utilizing the Karatsuba algorithm, which reduces the number of multiplications compared to traditional squaring methods. For small input sizes, the function falls back to a standard squaring algorithm (`squ_tx`) for performance.

Parameters

in	<code>x</code>	A double pointer to a <code>bigint</code> representing the binary integer to be squared.
out	<code>z</code>	A double pointer to a <code>bigint</code> where the result of the squaring operation will be stored.

Precondition

`x` must point to a valid, initialized `bigint` object. `z` must be properly allocated to store the resulting `bigint`.

Postcondition

The result of squaring `x` is stored in the location pointed to by `z`. The input `x` remains unchanged after the operation.

4.1.2.17 `squ_single_word()`

```
void squ_single_word (
    IN word A,
    OUT bigint ** result )
```

Squares a single word and stores the result in a bigint structure.

This function computes the square of a single word `A` and stores the result in a bigint structure pointed to by `result`.

Parameters

in	<code>A</code>	The single word to be squared.
out	<code>result</code>	A pointer to the bigint structure to store the squared result.

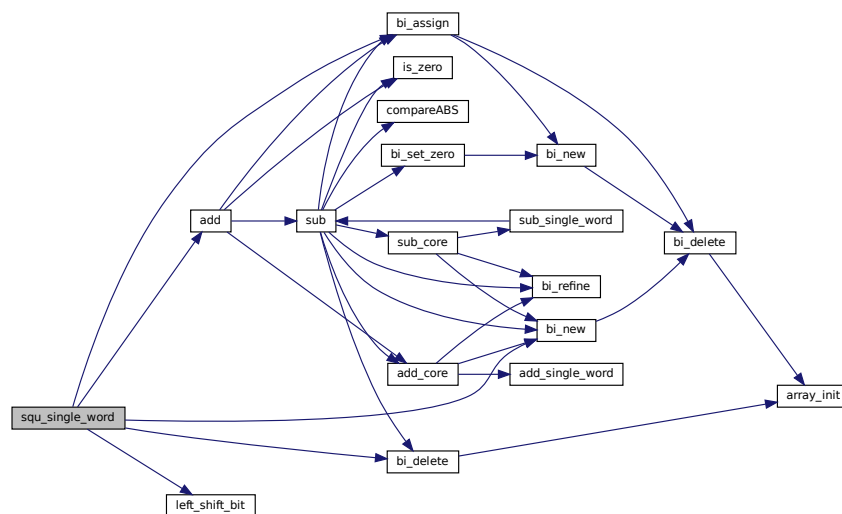
The squaring is done using a method that divides `A` into two halves: `A1` (upper bits) and `A0` (lower bits). The square is calculated as:

- $(C[0] = A0 * A0)$ (lower bits)
- $(C[1] = A1 * A1)$ (upper bits)
- The cross multiplication ($T = A0 * A1$), left-shifted by $w + 1$ bits

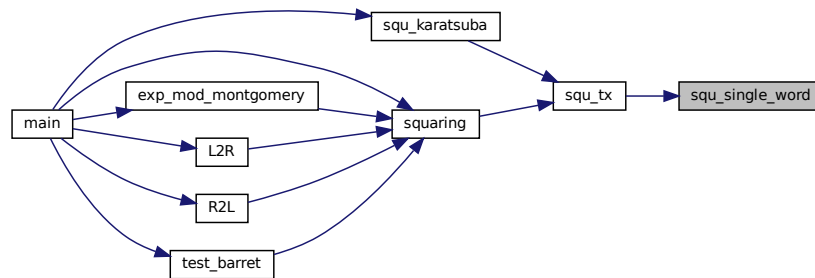
These intermediate results are combined to get the full square result.

Definition at line 477 of file `arithmetic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.18 squ_tx()

```

void squ_tx (
    IN bigint ** x,
    OUT bigint ** result )
  
```

Squares a bigint and stores the result in another bigint.

This function performs the squaring of a bigint x using the method of single-word squaring combined with cross products. The result is stored in the bigint structure pointed to by *result*.

Parameters

in	<i>x</i>	A pointer to the bigint to be squared.
out	<i>result</i>	A pointer to the bigint structure where the squared result will be stored.

The squaring method splits the process into:

- Squaring individual words in x and shifting accordingly.
- Calculating the cross products between words and summing them with left shifts.

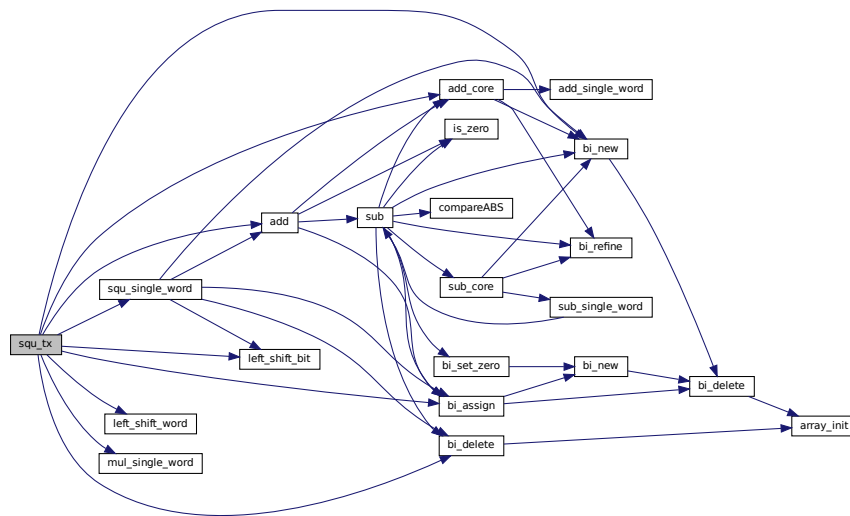
The function handles each word in x (up to t words) by:

- Calculating the square of each word j and adding the result to $C1$.
- Calculating the cross product for word pairs (j, i) where $i > j$ and accumulating in $C2$.

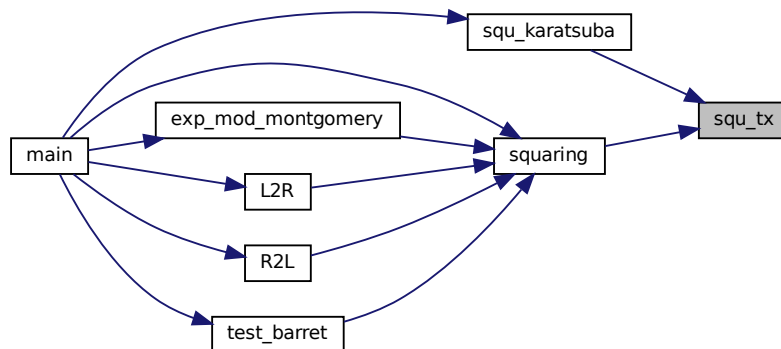
After processing, $C2$ is left-shifted by one bit to account for the doubling effect in cross terms.

Definition at line 510 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.19 squaring()

```

void squaring (
    IN bigint ** x,
    OUT bigint ** result )
  
```

Squares a bigint if it is non-zero and stores the result.

This function squares the bigint `x` and stores the result in `result`. If `x` is zero, it directly assigns zero to `result` and sets its sign to non-negative. Otherwise, it calls the `SQUC` function to compute the square of `x`.

Parameters

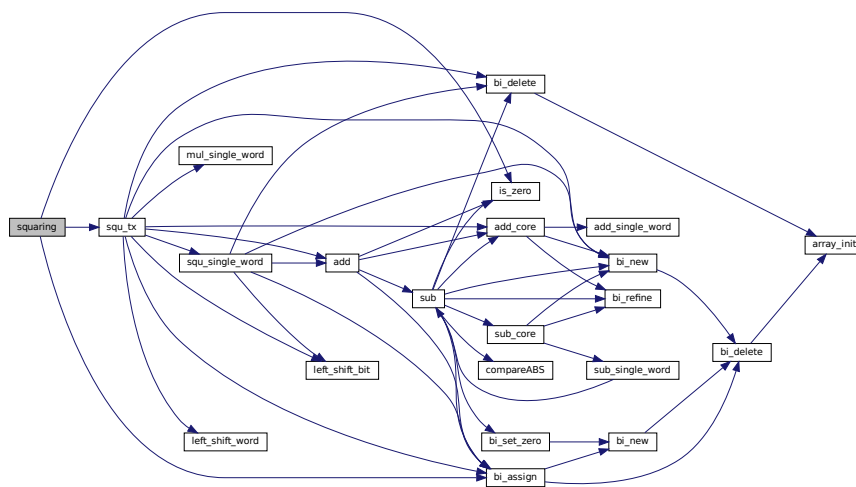
in	<i>x</i>	A pointer to the bigint to be squared.
out	<i>result</i>	A pointer to the bigint structure where the squared result will be stored.

The function first checks if *x* is zero:

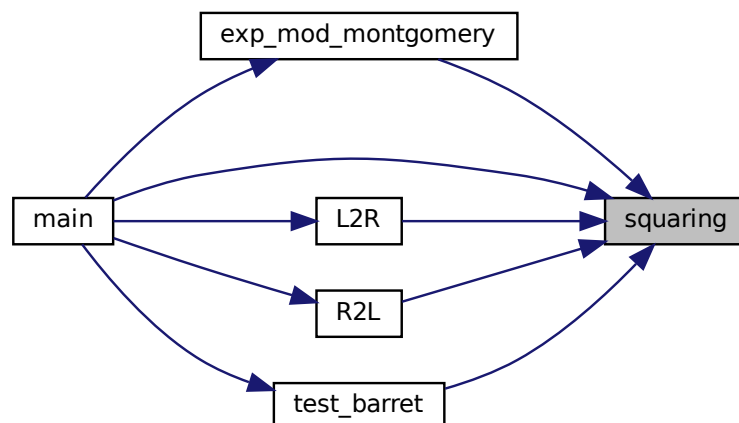
- If *x* is zero, *result* is assigned the value of zero with a non-negative sign.
- Otherwise, it uses *SQUC* to perform the squaring.

Definition at line 560 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.20 sub()

```
void sub (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Subtracts big integer y from big integer x and stores the result in z.

This function handles various cases depending on the signs and values of x and y, including when either operand is zero, when their absolute values are equal, and when their signs differ.

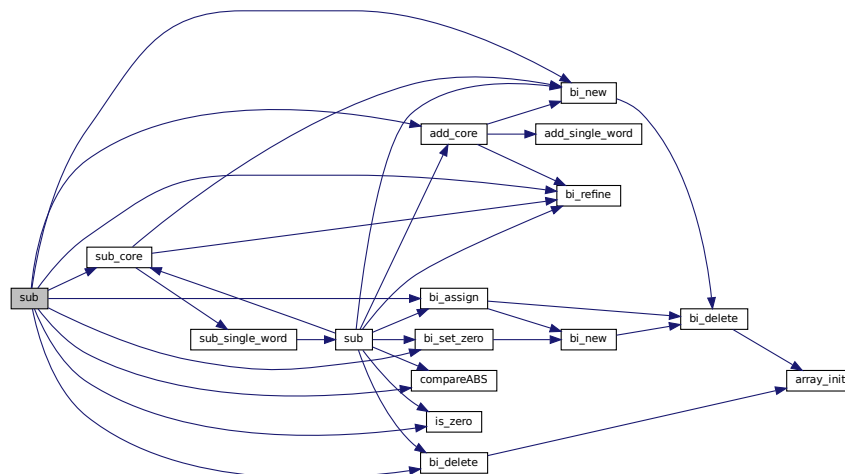
Parameters

in	x	Pointer to a pointer of the first bigint structure (minuend).
in	y	Pointer to a pointer of the second bigint structure (subtrahend).
out	z	Pointer to a pointer of the result bigint structure.

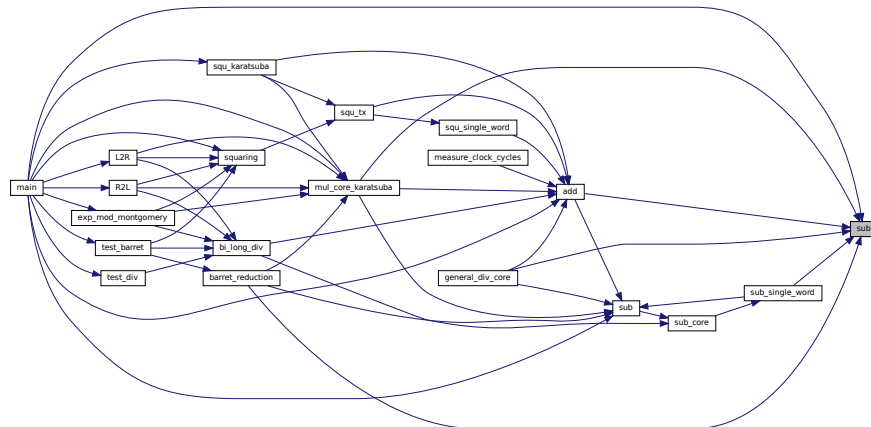
- If x is zero, the function sets z to -y.
- If y is zero, the function sets z to x.
- If the absolute values of x and y are equal:
 - If x and y have the same sign, z is set to zero.
 - If x and y have opposite signs, their absolute values are added, and z has the sign of x.
- If x and y are both positive, the function subtracts the smaller magnitude from the larger one and assigns the appropriate sign to z.
- If x and y are both negative, it subtracts the smaller magnitude from the larger and assigns a negative sign if x has a larger magnitude, or a positive sign if y is larger.
- If x and y have opposite signs, their absolute values are added and z has the sign of x.

Definition at line 166 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.21 sub_core()

```

void sub_core (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
  
```

Performs subtraction of two big integers.

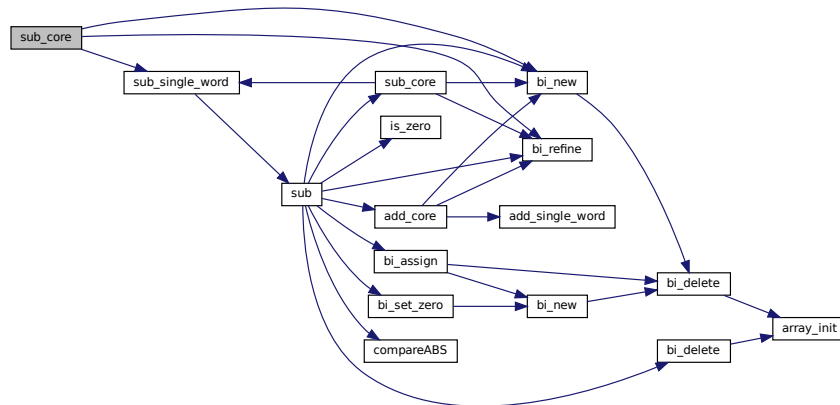
This function subtracts the big integer y from the big integer x and stores the result in z. It handles borrowing during subtraction and manages the necessary memory for the result.

Parameters

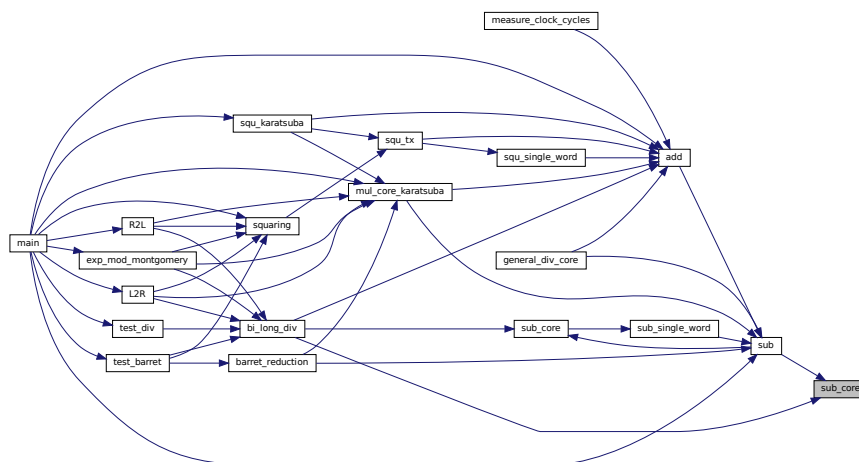
x	Pointer to the first bigint, the minuend.
y	Pointer to the second bigint, the subtrahend.
z	Pointer to store the result of the subtraction.

Definition at line 139 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.22 sub_single_word()

```

void sub_single_word (
    IN word A,
    IN word B,
    IN word borrow_in,
    OUT word * borrow_out,
    OUT word * result )

```

Performs subtraction of B from A and manages borrowing.

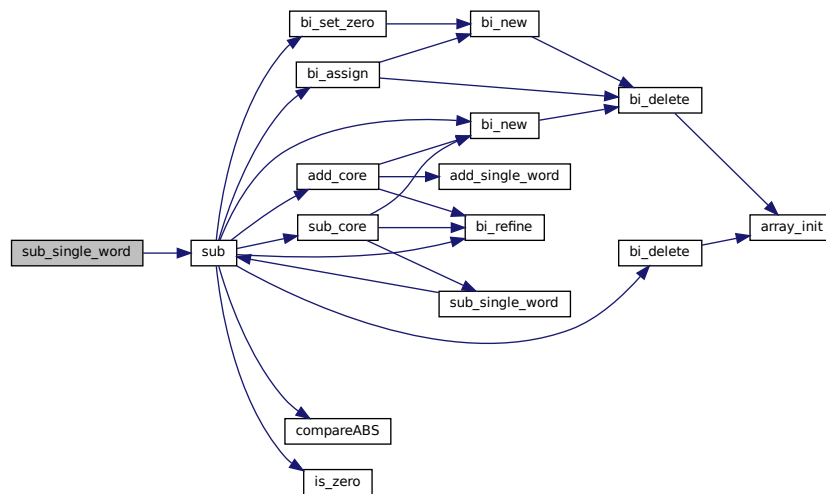
This function subtracts the second word (B) from the first word (A), taking into account any previous borrowing. It updates the result and the borrow status accordingly.

Parameters

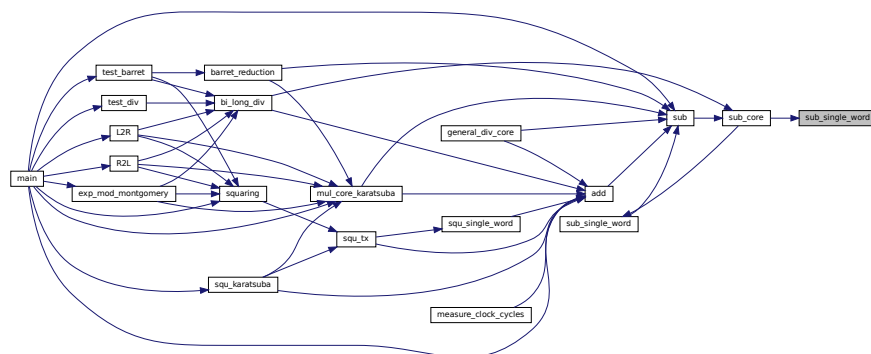
<i>A</i>	The first word, the value from which to subtract.
<i>B</i>	The second word, the value to be subtracted.
<i>borrow_in</i>	The previous borrow value.
<i>borrow_out</i>	Pointer to store the resulting borrow status.
<i>result</i>	Pointer to store the subtraction result.

Definition at line 123 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.2 include/bigint.h File Reference

This file defines the data structures and macros for representing and manipulating big integers.

- `#define` `MAXIMUM`(`x1`, `x2`) (`x1 > x2 ? x1 : x2`)
Returns the maximum of two values.
- `#define` `MINIMUM`(`x1`, `x2`) (`x1 < x2 ? x1 : x2`)
Returns the minimum of two values.

Typedefs

- `typedef` `u32 word`
Defines the word type based on the `WORD_BITLEN`.

4.2.1 Detailed Description

This file defines the data structures and macros for representing and manipulating big integers.

It provides a flexible data type definition for large integers (big integers) using conditional compilation based on the word size (8, 32, or 64 bits). The `bigint` structure is used to represent big integers, which consist of a sign, the length of the number in words, and a pointer to the array holding the actual big integer value.

Key components:

- Conditional compilation to define the `word` type based on the word size.
- The `bigint` structure to represent a big integer with a sign, word length, and the integer's value.
- Macros for calculating the maximum and minimum of two values.

The `bigint` structure allows for flexible handling of large integer arithmetic.

4.2.2 Macro Definition Documentation

4.2.2.1 MAXIMUM

```
#define MAXIMUM(  
    x1,  
    x2 ) (x1 > x2 ? x1 : x2)
```

Returns the maximum of two values.

A macro to compute the greater of two values `x1` and `x2`.

Parameters

<code>x1</code>	The first value.
<code>x2</code>	The second value.

Returns

The maximum of `x1` and `x2`.

Note

Both `x1` and `x2` should be of the same data type and comparable.

Definition at line 81 of file `bigint.h`.

4.2.2.2 MINIMUM

```
#define MINIMUM(  
    x1,  
    x2 ) (x1 < x2 ? x1 : x2)
```

Returns the minimum of two values.

A macro to compute the smaller of two values `x1` and `x2`.

Parameters

<code>x1</code>	The first value.
<code>x2</code>	The second value.

Returns

The minimum of `x1` and `x2`.

Note

Both `x1` and `x2` should be of the same data type and comparable.

Definition at line 91 of file `bigint.h`.

4.2.2.3 ONE

```
#define ONE (word)1
```

Macro for representing the constant one (1) in the `word` type.

Casts the value 1 to the current `word` type.

Definition at line 51 of file `bigint.h`.

4.2.2.4 WORD_BITLEN

```
#define WORD_BITLEN 32
```

Defines the bit length of a word.

Specifies the number of bits in a single word used for the `bigint` data type.

Note

This macro controls the word size, which impacts memory usage and computation speed.

Definition at line 29 of file `bigint.h`.

4.2.2.5 ZERO

```
#define ZERO (word)0
```

Macro for representing the constant zero (0) in the `word` type.

Casts the value 0 to the current `word` type.

Definition at line 57 of file `bigint.h`.

4.2.3 Typedef Documentation

4.2.3.1 word

```
typedef u32 word
```

Defines the word type based on the `WORD_BITLEN`.

Allows flexibility in choosing the word size for the `bigint` structure.

- If `WORD_BITLEN` is 8, the word type is `u8` (8-bit unsigned integer).
- If `WORD_BITLEN` is 32, the word type is `u32` (32-bit unsigned integer).
- For any other `WORD_BITLEN`, the word type defaults to `u64` (64-bit unsigned integer).

Note

Ensure that `WORD_BITLEN` is properly defined to avoid compilation errors. Word is defined as a 32-bit unsigned integer.

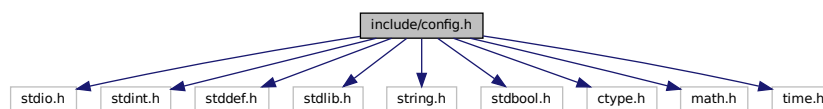
Definition at line 42 of file `bigint.h`.

4.3 include/config.h File Reference

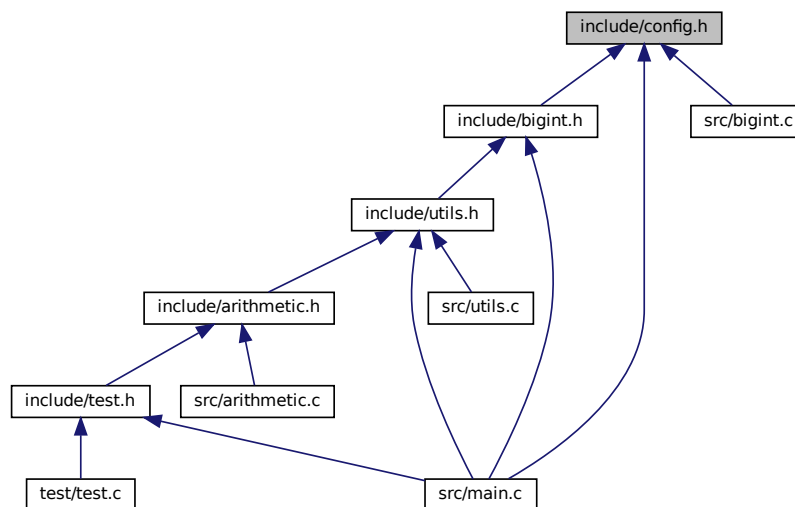
This file contains configuration settings, type definitions, and necessary includes for the project.

```
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <ctype.h>
#include <math.h>
#include <time.h>
```

Include dependency graph for config.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define IN`
Defines input/output direction macros for function parameters.
- `#define OUT`
- `#define INOUT`
- `#define NON_NEGATIVE 0`
Sign definitions for bigint values.
- `#define NEGATIVE 1`
- `#define FLAG 20`
Defines a generic flag constant.

Typedefs

- typedef uint8_t [u8](#)
Type aliases for unsigned integers.
- typedef uint32_t [u32](#)
- typedef uint64_t [u64](#)
- typedef int8_t [i8](#)
Type aliases for signed integers.
- typedef int32_t [i32](#)
- typedef int64_t [i64](#)

4.3.1 Detailed Description

This file contains configuration settings, type definitions, and necessary includes for the project.

It includes standard library headers for functionality like input/output, memory management, and mathematical operations. Conditional compilation is used to include specific headers based on the target architecture (e.g., x86 or x86_64). It also defines custom type aliases and symbolic constants for use across the project.

Key components:

- Includes common standard libraries like `stdio.h`, `stdint.h`, and `stdlib.h`.
- Conditional inclusion of architecture-specific headers for optimizations (e.g., `x86intrin.h`).
- Type definitions for `u8`, `u32`, `u64`, `i8`, `i32`, and `i64` to represent various data types.
- Macros defining input (`IN`), output (`OUT`), and input/output (`INOUT`) argument conventions.
- Constants to represent non-negative and negative values, along with a `FLAG` definition.

4.3.2 Macro Definition Documentation

4.3.2.1 FLAG

```
#define FLAG 20
```

Defines a generic flag constant.

This macro defines a constant value `FLAG` with a default value of 20.

Note

The purpose of this flag may vary depending on the specific implementation.

Definition at line 73 of file `config.h`.

4.3.2.2 IN

```
#define IN
```

Defines input/output direction macros for function parameters.

These macros provide clarity about the purpose of function parameters:

- **IN**: Indicates that the parameter is used as input and will not be modified.
- **OUT**: Indicates that the parameter is used as output and will be modified by the function.
- **INOUT**: Indicates that the parameter is both input and output, meaning it will be modified.

Note

These are purely for documentation and readability purposes and do not affect the code behavior.

Definition at line 55 of file config.h.

4.3.2.3 INOUT

```
#define INOUT
```

Definition at line 57 of file config.h.

4.3.2.4 NEGATIVE

```
#define NEGATIVE 1
```

Definition at line 66 of file config.h.

4.3.2.5 NON_NEGATIVE

```
#define NON_NEGATIVE 0
```

Sign definitions for bigint values.

These macros define constants to represent the sign of a bigint:

- **NON_NEGATIVE**: Represents non-negative integers (0 or positive values).
- **NEGATIVE**: Represents negative integers.

Definition at line 65 of file config.h.

4.3.2.6 OUT

```
#define OUT
```

Definition at line 56 of file config.h.

4.3.3 Typedef Documentation

4.3.3.1 i32

```
typedef int32_t i32
```

Definition at line 94 of file config.h.

4.3.3.2 i64

```
typedef int64_t i64
```

Definition at line 95 of file config.h.

4.3.3.3 i8

```
typedef int8_t i8
```

Type aliases for signed integers.

Provides platform-independent type definitions for signed integers:

- `i8`: 8-bit signed integer (alias for `int8_t`).
- `i32`: 32-bit signed integer (alias for `int32_t`).
- `i64`: 64-bit signed integer (alias for `int64_t`).

Definition at line 93 of file config.h.

4.3.3.4 u32

```
typedef uint32_t u32
```

Definition at line 83 of file config.h.

4.3.3.5 u64

```
typedef uint64_t u64
```

Definition at line 84 of file config.h.

4.3.3.6 u8

```
typedef uint8_t u8
```

Type aliases for unsigned integers.

Provides platform-independent type definitions for unsigned integers:

- u8: 8-bit unsigned integer (alias for uint8_t).
- u32: 32-bit unsigned integer (alias for uint32_t).
- u64: 64-bit unsigned integer (alias for uint64_t).

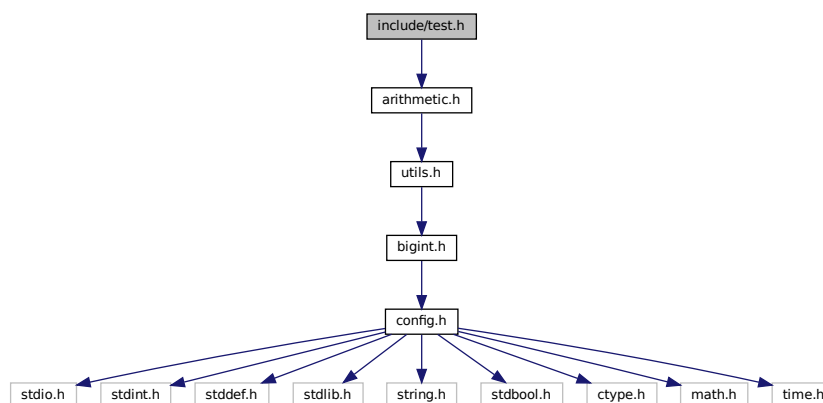
Definition at line 82 of file config.h.

4.4 include/test.h File Reference

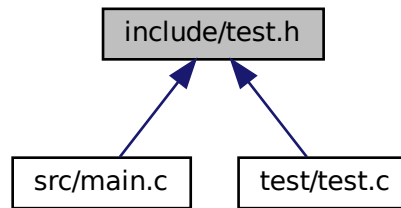
This file declares the functions and definitions used for testing and measuring the performance of various arithmetic operations in the BigInt library.

```
#include "arithmetic.h"
```

Include dependency graph for test.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define TEST_CASE 10000`
Number of test cases for testing functions.
- `#define MAX_BIT_LEN 256`
*Maximum bit length for *bigint* operations.*
- `#define TIME(start, end) ((double)((end) - (start))) / CLOCKS_PER_SEC`
Calculates the time difference between two clock values.

Functions

- `void print_bi_hex_py (IN const bigint *x)`
*Prints a *bigint* in hexadecimal format (Python-style).*
- `void test_basic_operation (void(*operation)(IN bigint **, IN bigint **, OUT bigint **), const char *operator, const char *operate)`
*Tests basic arithmetic operations on randomly generated *bigint* values.*
- `void test_exp_mod (void(*operation)(IN bigint **, IN bigint **, OUT bigint **, IN bigint **), const char *operate)`
Tests modular exponentiation with randomly generated values.
- `void test_div (const char *operate)`
Tests division by verifying the quotient and remainder.
- `void test_barret (const char *operate)`
Tests Barrett reduction for modular arithmetic.
- `void test_squ (void(*operation)(IN bigint **, OUT bigint **), const char *operate)`
*Tests the squaring operation on randomly generated *bigint* values.*
- `void measure_cycles (void(*func)(IN bigint **, IN bigint **, OUT bigint **), IN bigint **x, IN bigint **y, OUT bigint **Z)`
Measures the average number of CPU cycles for a specific operation.
- `void measure_clock_cycles ()`
Measures and prints the clock cycles for various operations.

4.4.1 Detailed Description

This file declares the functions and definitions used for testing and measuring the performance of various arithmetic operations in the `BigInt` library.

It includes function prototypes for testing basic arithmetic operations, exponentiation, division, squaring, and modular operations. Additionally, it provides functions for measuring the execution time and clock cycles for these operations.

Key components:

- Definitions like `TEST_CASE` and `MAX_BIT_LEN` for configuring test parameters.
- Test functions for basic arithmetic operations, exponentiation, division, squaring, and Barret reduction.
- Performance measurement functions that evaluate execution time and clock cycles.

The file is intended to be used in unit tests and performance benchmarking for the `BigInt` library's arithmetic operations.

4.4.2 Macro Definition Documentation

4.4.2.1 `MAX_BIT_LEN`

```
#define MAX_BIT_LEN 256
```

Maximum bit length for `bigint` operations.

This macro sets the upper limit on the bit length of the numbers generated for testing or operations. It determines the maximum size of the `bigint` structure's data array.

Note

Adjusting `MAX_BIT_LEN` affects memory usage and computational overhead. Larger values allow for testing larger integers but require more resources.

Definition at line 39 of file `test.h`.

4.4.2.2 `TEST_CASE`

```
#define TEST_CASE 10000
```

Number of test cases for testing functions.

This macro defines the total number of iterations or test cases that will be executed during the testing phase.

Note

Increasing the value of `TEST_CASE` can lead to more thorough testing but may significantly increase execution time.

Definition at line 29 of file `test.h`.

4.4.2.3 TIME

```
#define TIME(  
    start,  
    end ) ((double)((end) - (start))) / CLOCKS_PER_SEC
```

Calculates the time difference between two clock values.

This macro computes the elapsed time in seconds by subtracting the starting clock value (*start*) from the ending clock value (*end*) and dividing by `CLOCKS_PER_SEC`.

Parameters

<i>start</i>	The clock value at the start of the timed operation.
<i>end</i>	The clock value at the end of the timed operation.

Returns

The elapsed time in seconds as a `double`.

Note

This macro relies on the standard library macro `CLOCKS_PER_SEC` to convert clock ticks into seconds.

Warning

Ensure that *start* and *end* are valid `clock_t` values obtained from `clock()` or similar functions.

Definition at line 54 of file `test.h`.

4.4.3 Function Documentation

4.4.3.1 `measure_clock_cycles()`

```
void measure_clock_cycles ( )
```

Measures and prints the clock cycles for various operations.

This function executes different arithmetic operations multiple times and measures the average CPU clock cycles required for each operation.

Precondition

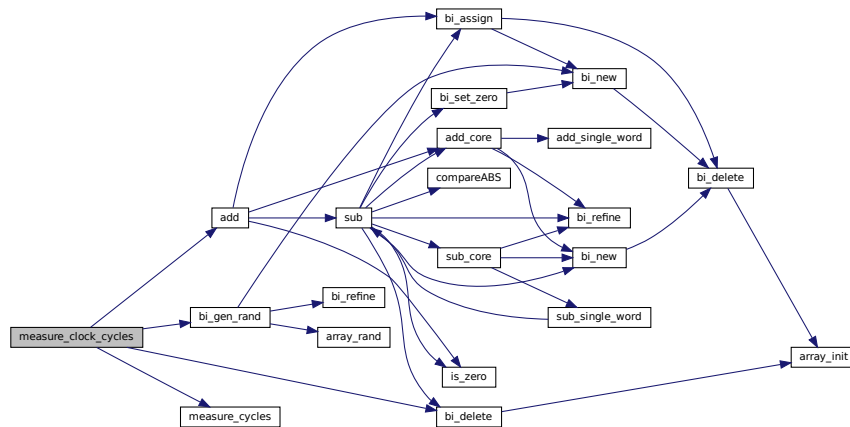
All operations to be measured must have properly defined function implementations.

Postcondition

The average clock cycles for each operation are printed to the standard output.

Definition at line 247 of file test.c.

Here is the call graph for this function:

**4.4.3.2 measure_cycles()**

```

void measure_cycles (
    void(*) (IN bigint **, IN bigint **, OUT bigint **) func,
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
  
```

Measures the average number of CPU cycles for a specific operation.

This function times the execution of a specified operation over multiple iterations to compute the average number of CPU cycles consumed.

Parameters

in	<i>func</i>	A function pointer to the operation being measured.
in	<i>x</i>	A pointer to the first <code>bigint</code> input.
in	<i>y</i>	A pointer to the second <code>bigint</code> input.
out	<i>z</i>	A pointer to the <code>bigint</code> where the result will be stored.

Precondition

`func` must be a valid function pointer, and all input/output `bigint` pointers must be initialized.

Postcondition

The average CPU cycles are printed to the standard output.

Definition at line 229 of file test.c.

Here is the caller graph for this function:

**4.4.3.3 print_bi_hex_py()**

```
void print_bi_hex_py (
    IN const bigint * x )
```

Prints a bigint in hexadecimal format (Python-style).

This function prints the hexadecimal representation of the given `bigint` object. The format is similar to Python's hexadecimal literals, prefixed with "0x". If the number is negative, a "-" sign is prepended.

Parameters

in	x	A pointer to the <code>bigint</code> to be printed.
----	---	---

Precondition

`x` must point to a valid `bigint` object with an initialized array `a` and word length.

Postcondition

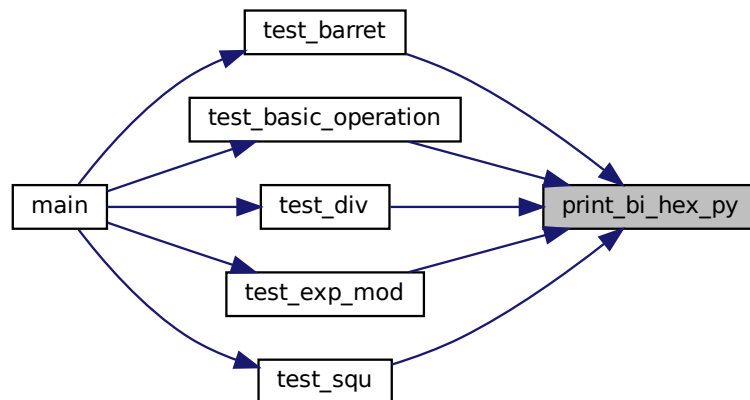
The hexadecimal representation of the `bigint` is printed to the standard output.

Note

This function handles different word lengths based on the `WORD_BITLEN` macro.

Definition at line 3 of file test.c.

Here is the caller graph for this function:



4.4.3.4 test_barret()

```
void test_barret (
    const char * operate )
```

Tests Barrett reduction for modular arithmetic.

This function verifies the Barrett reduction method for efficient modular reduction. It generates random `bigint` values and precomputes necessary constants for Barrett reduction.

Parameters

<code>in</code>	<code>operate</code>	A string representing the name of the operation (used in output).
-----------------	----------------------	---

Precondition

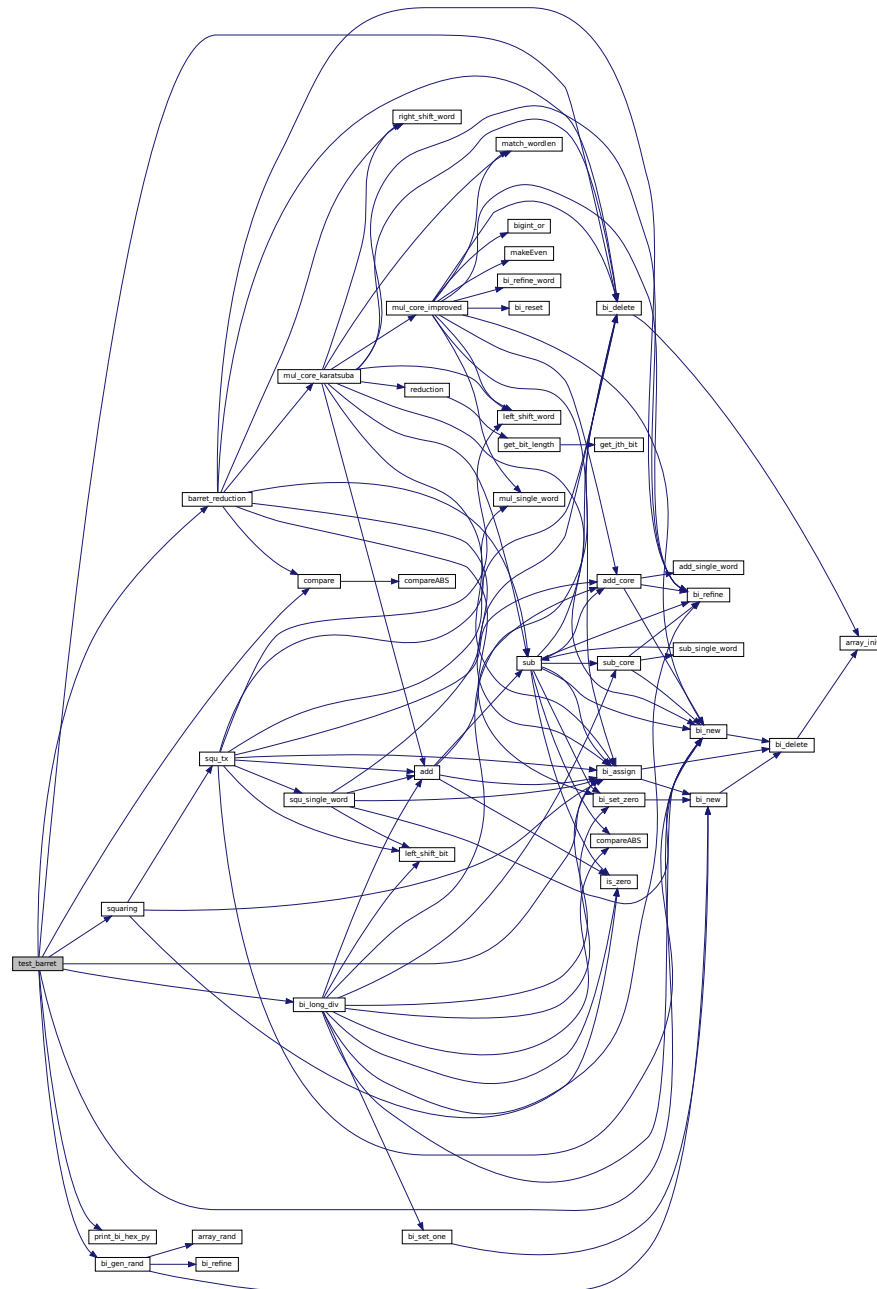
Input values must be valid `bigint` objects, and the modulus must be non-zero.

Postcondition

The reduction results are printed, verifying correctness.

Definition at line 165 of file `test.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.3.5 test_basic_operation()

```
void test_basic_operation (
    void(*) (IN bigint **, IN bigint **, OUT bigint **) operation,
    const char * operator,
    const char * operate )
```

Tests basic arithmetic operations on randomly generated `bigint` values.

This function generates random `bigint` values, performs the specified operation, and prints the inputs and results in the format: `x [operator] y == z`.

Parameters

in	<i>operation</i>	A function pointer to the arithmetic operation being tested.
in	<i>operator</i>	A string representing the symbol of the operation (e.g., "+", "-", "*").
in	<i>operate</i>	A string representing the name of the operation (used in output).

Precondition

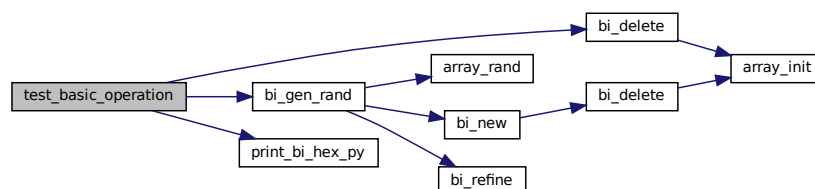
`operation` must point to a valid arithmetic function accepting input and output `bigint` pointers.

Postcondition

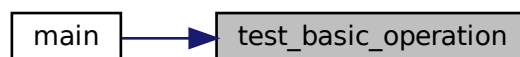
The inputs, operation, and results are printed to the standard output.

Definition at line 61 of file `test.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.3.6 test_div()

```
void test_div (
    const char * operate )
```

Tests division by verifying the quotient and remainder.

This function generates random `bigint` values, computes the quotient and remainder using long division, and verifies that the equation $x == q * y + r$ holds.

Parameters

in	<i>operate</i>	A string representing the name of the operation (used in output).
----	----------------	---

Precondition

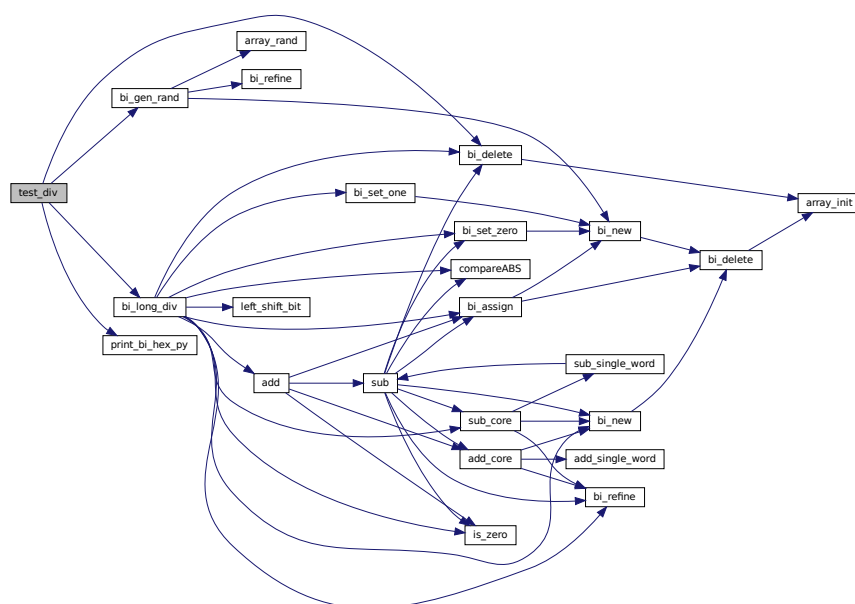
The `bigint` inputs must be non-negative, and the divisor `y` must not be zero.

Postcondition

The division operation and verification results are printed to the standard output.

Definition at line 90 of file test.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.3.7 test_exp_mod()

```

void test_exp_mod (
    void(*) (IN bigint **, IN bigint **, OUT bigint **, IN bigint **) operation,
    const char * operate )
  
```

Tests modular exponentiation with randomly generated values.

This function computes $(x^y) \% \text{mod}$ using the specified operation and prints the result.

Parameters

in	<i>operation</i>	A function pointer to the modular exponentiation function being tested.
in	<i>operate</i>	A string representing the name of the operation (used in output).

Precondition

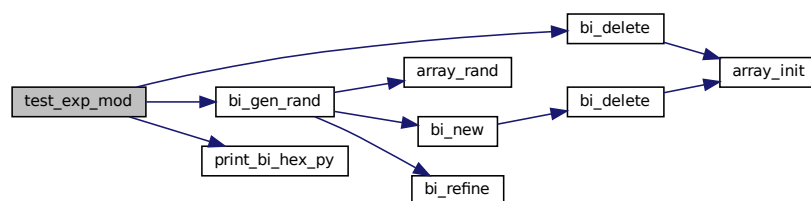
operation must point to a valid modular exponentiation function accepting input and output *bigint* pointers.

Postcondition

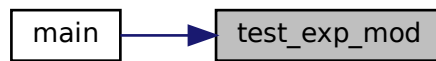
The inputs and the modular exponentiation result are printed to the standard output.

Definition at line 124 of file test.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.3.8 test_squ()

```

void test_squ (
    void(*) (IN bigint **, OUT bigint **) operation,
    const char * operate )
  
```

Tests the squaring operation on randomly generated `bigint` values.

This function generates random `bigint` values, computes their square using the given operation, and prints the inputs and results for verification.

Parameters

in	<i>operation</i>	A function pointer to the squaring operation being tested.
in	<i>operate</i>	A string representing the name of the operation (used in output).

Precondition

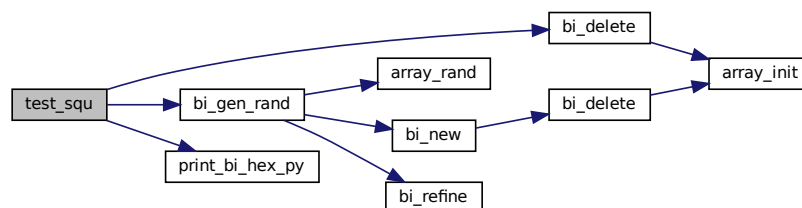
`operation` must point to a valid squaring function accepting input and output `bigint` pointers.

Postcondition

The inputs and squared results are printed to the standard output.

Definition at line 34 of file `test.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

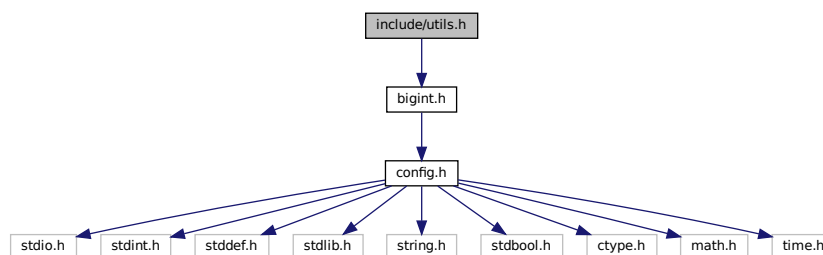


4.5 include/utls.h File Reference

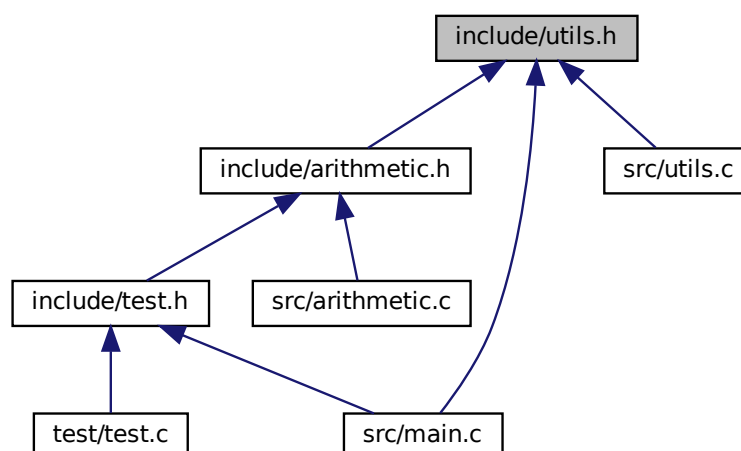
This file provides utility functions for managing and manipulating big integers (bigint type), including memory allocation, bitwise operations, shifting, and random number generation.

```
#include "bigint.h"
```

Include dependency graph for utls.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define MEM_ALLOCATION_FAIL` `fprintf(stderr, "Error: Memory allocation failed at %s:%d\n", __FILE__, __LINE__);`
Reports a memory allocation failure.
- `#define SET_STRING_FAIL` `fprintf(stderr, "Error: Invalid input for string or base at %s:%d\n", __FILE__, __LINE__);`
Reports a failure when setting a string or base.
- `#define SET_ARRAY_FAIL` `fprintf(stderr, "Error: Set array failed at %s:%d\n", __FILE__, __LINE__);`
Reports a failure when setting an array.
- `#define INVAILD_DATA` `fprintf(stderr, "Error: Invalid data at %s:%d\n", __FILE__, __LINE__);`
Reports invalid data input.

Functions

- `void bi_new (IN bigint **x, IN int wordlen)`
Allocates memory for a new bigint structure.
- `void array_init (IN word *array, IN int length)`
Initializes an array with a given length to a default value (0x00000000).
- `void bi_delete (INOUT bigint **x)`
Frees the memory occupied by a bigint structure.
- `void bi_show_hex (IN bigint *x)`
Displays the hexadecimal representation of a bigint.
- `int bi_set_by_array (OUT bigint **x, IN int sign, IN word *a, IN int wordlen)`
Initializes a bigint structure using an array of words and sets its sign.
- `int bi_set_by_string (OUT bigint **x, IN int sign, IN char *str, IN int base)`
Initializes a bigint structure from a character string representing a number.
- `void bi_refine (INOUT bigint *x)`
Refines a bigint by removing leading zeros and adjusting its length.
- `void bi_assign (OUT bigint **dest, IN bigint *src)`
Assigns the value of one bigint to another.
- `void bi_gen_rand (OUT bigint **x, IN int sign, IN int wordlen)`
Generates a random bigint with a specified sign and word length.
- `void array_rand (OUT word *dst, IN int wordlen)`
Fills an array with random bytes.
- `void bi_set_one (OUT bigint **x)`
Sets a bigint to the value of one.
- `void bi_set_zero (OUT bigint **x)`
Sets a bigint to the value of zero.
- `int compareABS (IN bigint *x, IN bigint *y)`
Compares the absolute values of two bigint numbers.
- `int compare (IN bigint *x, IN bigint *y)`
Compare two bigint numbers, taking into account their signs.
- `int get_bit_length (IN bigint *x)`
Calculate the bit length of a given bigint.
- `bool get_jth_bit (IN bigint *x, IN int j)`
Get the value of the j-th bit from a bigint.
- `void right_shift_bit (INOUT bigint *x, IN int shift)`
Perform a right shift operation on a bigint.
- `void left_shift_bit (bigint *pptrBint, int shift_amount)`

- Perform a left shift operation on a bigint.*

 - void `reduction` (IN `bigint **x`, IN int `r`)
- Perform a reduction operation on a bigint.*

 - void `right_shift_word` (INOUT `bigint **x`, IN int `shift_words`)
- Right-shifts a bigint by a specified number of words.*

 - void `left_shift_word` (INOUT `bigint **x`, IN int `shift_words`)
- Left-shifts a bigint by a specified number of words.*

 - int `is_zero` (IN `bigint *x`)
- Checks if a bigint is zero.*

 - void `swap_bigint` (INOUT `bigint **x`, INOUT `bigint **y`)
- Swaps the values of two bigint pointers.*

 - void `bi_refine_word` (IN `bigint *x`, IN int `num_words`)
- Refines the word length of a bigint by removing a specified number of words.*

 - void `makeEven` (INOUT `bigint *x`)
- Ensures that the word length of a bigint is even.*

 - void `match_wordlen` (INOUT `bigint *x`, INOUT `bigint *y`)
- Matches the word lengths of two bigint objects by resizing them to the larger of their current lengths.*

 - void `bi_reset` (INOUT `bigint *x`)
- Resets all words in a bigint to zero.*

 - `word get_word` (IN `bigint *x`, IN int `m_th`)
- Returns the value of the m-th word from a bigint.*

4.5.1 Detailed Description

This file provides utility functions for managing and manipulating big integers (`bigint` type), including memory allocation, bitwise operations, shifting, and random number generation.

It includes functions for:

- Memory management for big integers (`bi_new`, `bi_delete`, etc.)
- Arithmetic and bitwise operations (e.g., shifting, setting bits, comparing values)
- Conversion between different formats (e.g., array, string)
- Random number generation for big integers

These utility functions are essential for the core functionality of the `BigInt` library.

4.5.2 Macro Definition Documentation

4.5.2.1 INVAILD_DATA

```
#define INVAILD_DATA fprintf(stderr, "Error: Invalid data at %s:%d\n", __FILE__, __LINE__);
```

Reports invalid data input.

This macro prints an error message to `stderr` when an invalid data input or state is detected, including the file name and line number.

Note

Use this macro in functions where invalid data might cause undefined behavior or logic errors.

Warning

Ensure that input data is validated before processing to avoid triggering this macro unnecessarily.

Definition at line 57 of file `utils.h`.

4.5.2.2 MEM_ALLOCATION_FAIL

```
#define MEM_ALLOCATION_FAIL fprintf(stderr, "Error: Memory allocation failed at %s:%d\n", __FILE__, __LINE__);
```

Reports a memory allocation failure.

This macro prints an error message to `stderr` when memory allocation fails, including the file name and line number where the failure occurred.

Note

This macro is intended to assist in debugging memory issues.

Warning

Ensure that memory allocation functions (e.g., `malloc`, `calloc`) are checked for NULL return values before using this macro.

Definition at line 27 of file `utils.h`.

4.5.2.3 SET_ARRAY_FAIL

```
#define SET_ARRAY_FAIL fprintf(stderr, "Error: Set array failed at %s:%d\n", __FILE__, __LINE__);
```

Reports a failure when setting an array.

This macro prints an error message to `stderr` if an error occurs while setting up or initializing an array, including the file name and line number.

Note

Common scenarios include invalid indices or failed assignments during array initialization.

Warning

Verify that the array is properly allocated and accessed within bounds before using this macro.

Definition at line 47 of file `utils.h`.

4.5.2.4 SET_STRING_FAIL

```
#define SET_STRING_FAIL fprintf(stderr, "Error: Invalid input for string or base at %s:%d\n", __FILE__, __LINE__);
```

Reports a failure when setting a string or base.

This macro prints an error message to `stderr` when an invalid string or base input is encountered in the program, including the file name and line number.

Note

Use this macro in functions where input validation for strings or bases is required.

Warning

Ensure that the input is properly validated before calling functions that rely on strings or bases.

Definition at line 37 of file `utils.h`.

4.5.3 Function Documentation

4.5.3.1 array_init()

```
void array_init (
    IN word * array,
    IN int length )
```

Initializes an array with a given length to a default value (0x00000000).

This function sets all elements of the input array to 0. It is typically used to clear or reset an array before performing other operations like big integer addition or multiplication.

Parameters

in	<i>array</i>	The pointer to the array to be initialized.
in	<i>length</i>	The number of elements (length) of the array to initialize.

Returns

void This function does not return any value.

Note

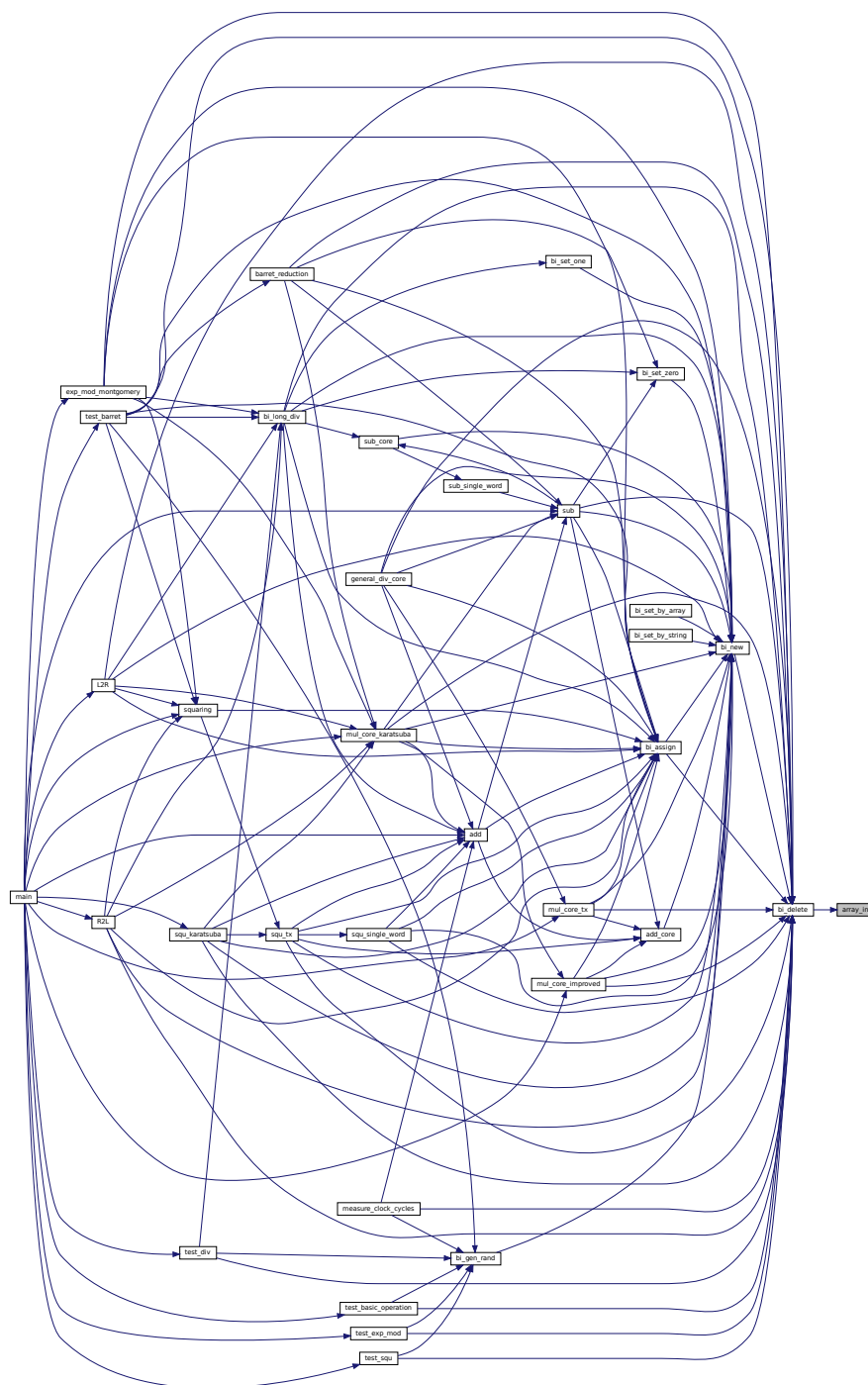
Ensure that the array has enough allocated memory to hold `length` elements.

Warning

This function assumes the array is writable and properly allocated.

Definition at line 28 of file `utils.c`.

Here is the caller graph for this function:



4.5.3.2 array_rand()

```
void array_rand (
    OUT word * dst,
    IN int wordlen )
```

Fills an array with random bytes.

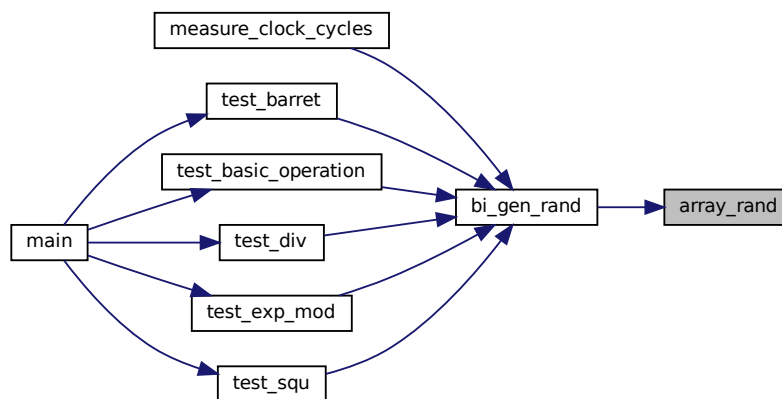
This function populates the specified array with random bytes using the rand function.

Parameters

out	<i>dst</i>	Pointer to the array to be filled with random values.
in	<i>wordlen</i>	The number of words to fill.

Definition at line 215 of file utils.c.

Here is the caller graph for this function:



4.5.3.3 bi_assign()

```

void bi_assign (
    OUT bigint ** dest,
    IN bigint * src )

```

Assigns the value of one bigint to another.

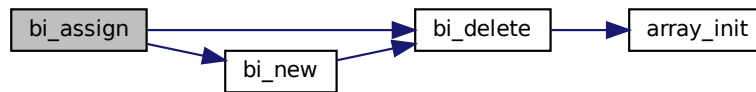
This function deletes the existing destination bigint, creates a new bigint of the same word length as the source, and copies the sign, word length, and value from the source bigint.

Parameters

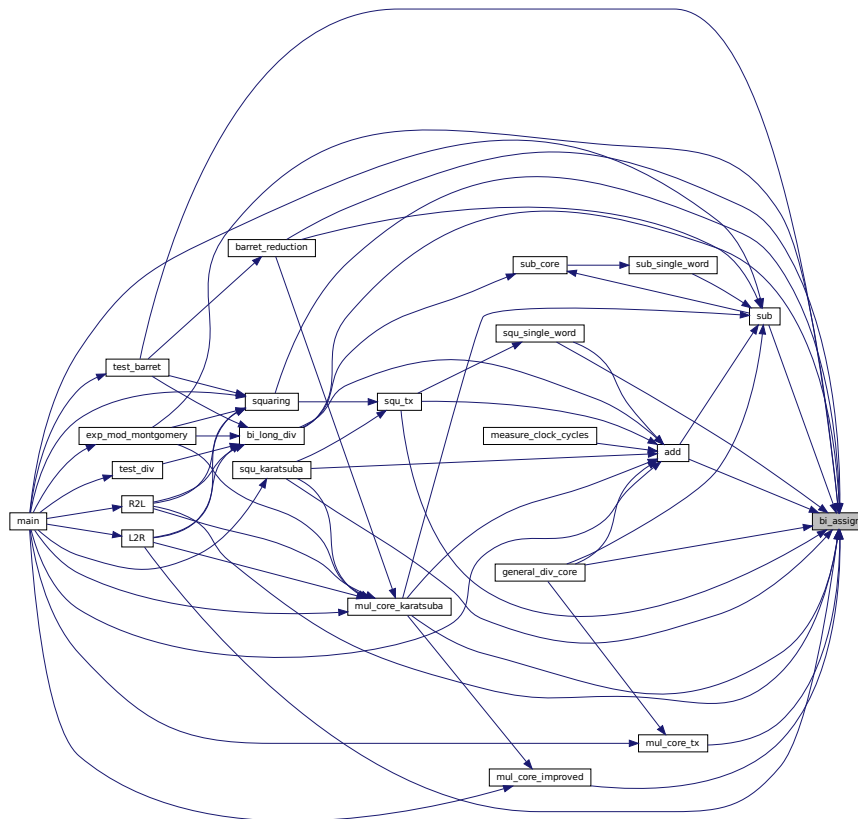
out	<i>dest</i>	Pointer to the destination bigint to be assigned to.
in	<i>src</i>	Pointer to the source bigint to copy from.

Definition at line 189 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.4 bi_delete()

```
void bi_delete (
    INOUT bigint ** x )
```

Frees the memory occupied by a bigint structure.

This function releases the memory allocated for the bigint structure and sets the pointer to NULL to avoid dangling references.

Parameters

in, out	x	Pointer to the bigint structure to be deleted.
---------	---	--

Returns

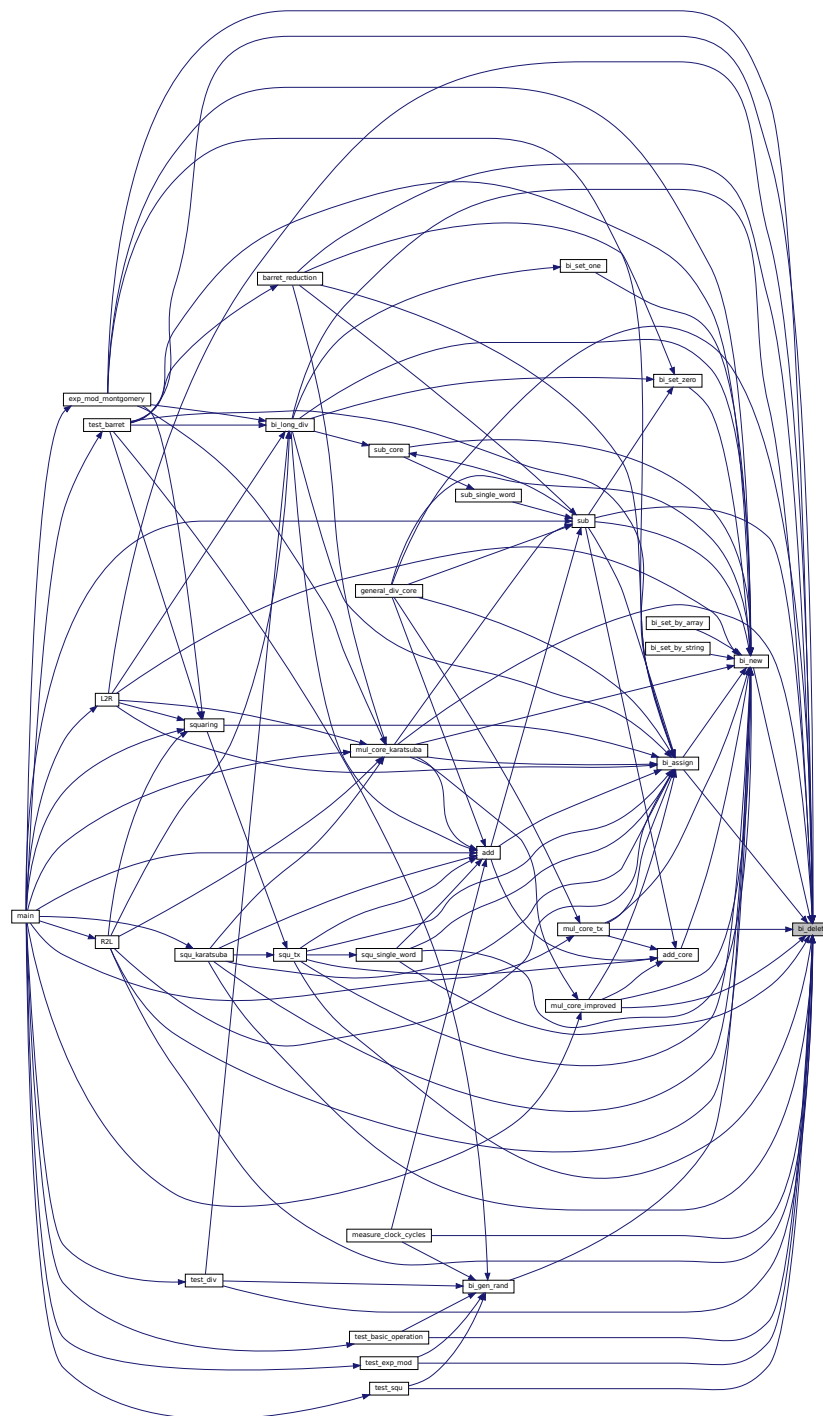
void

Definition at line 35 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.5 bi_gen_rand()

```
void bi_gen_rand (
    OUT bigint ** x,
```

```

IN int sign,
IN int wordlen )

```

Generates a random bigint with a specified sign and word length.

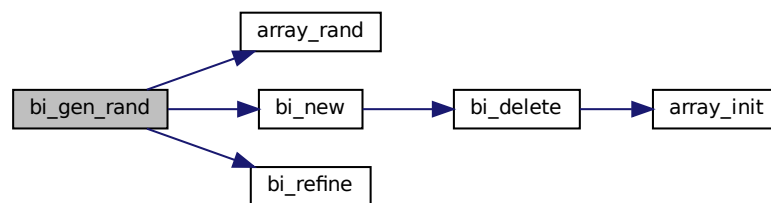
This function creates a new bigint, sets its sign, generates random values for its word array, and then refines it to remove any leading zeros.

Parameters

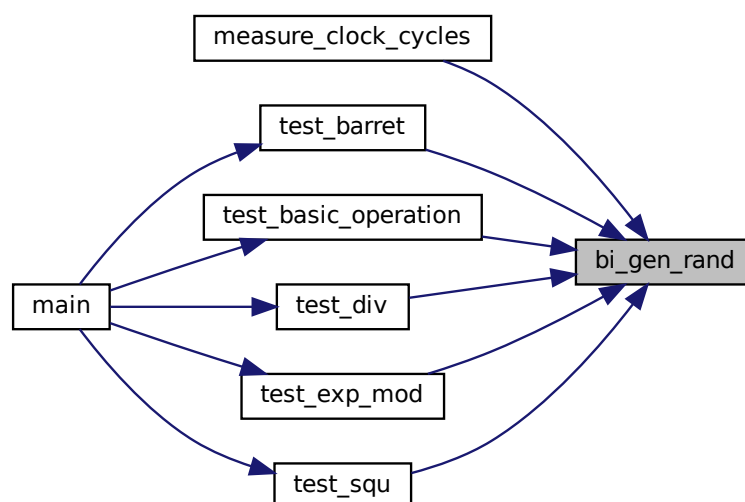
out	<i>x</i>	Pointer to the bigint to be generated.
in	<i>sign</i>	The sign of the bigint (NEGATIVE or NON_NEGATIVE).
in	<i>wordlen</i>	The desired length of the word array.

Definition at line 206 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.6 bi_new()

```
void bi_new (
    IN bigint ** x,
    IN int wordlen )
```

Allocates memory for a new bigint structure.

This function creates a new bigint structure and initializes it with the specified word length. The pointer to the newly created bigint is stored in the provided pointer to a pointer (x).

Parameters

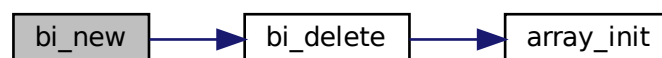
out	<i>x</i>	Pointer to the bigint structure to be initialized.
in	<i>wordlen</i>	The number of words to allocate for the bigint.

Returns

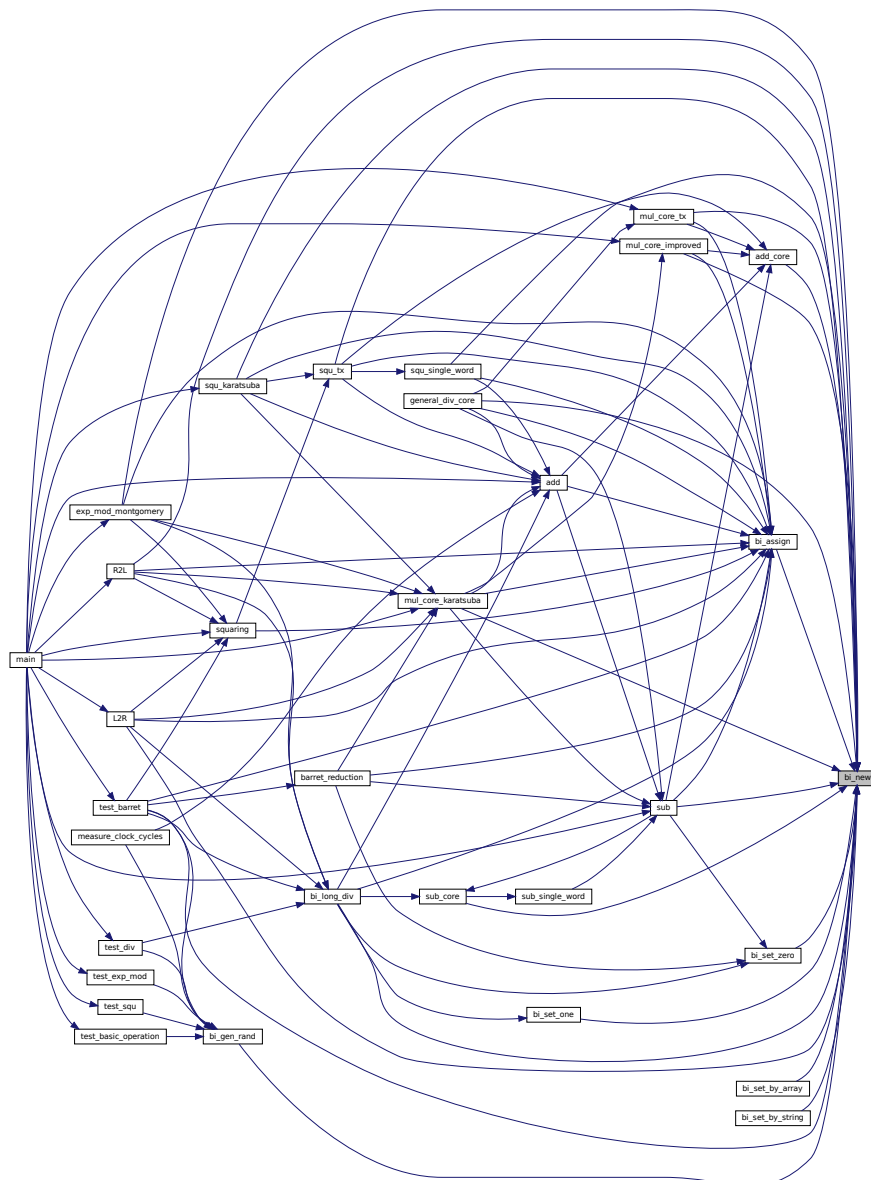
void

Definition at line 5 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.7 bi_refine()

```
void bi_refine (
    INOUT bigint * x )
```

Refines a bigint by removing leading zeros and adjusting its length.

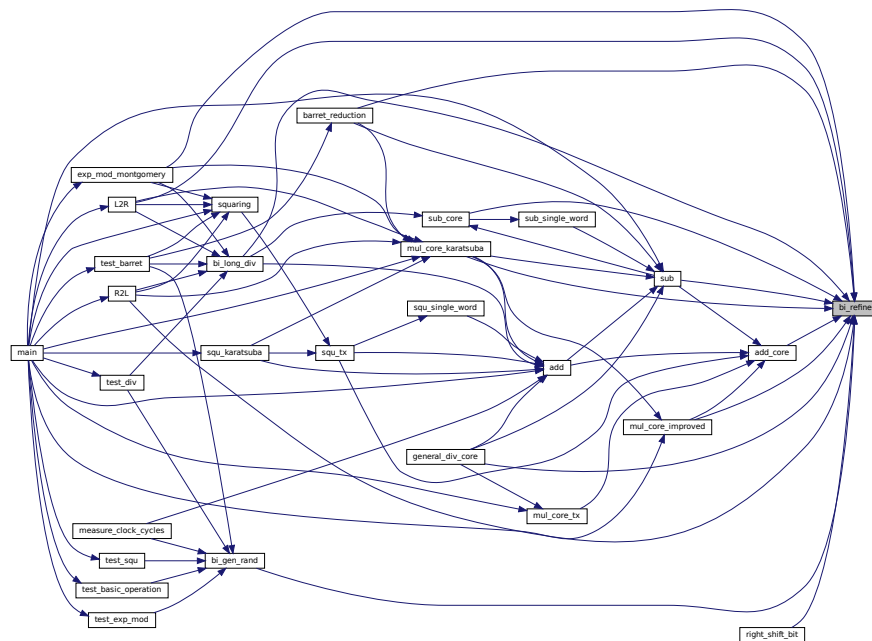
This function modifies the given bigint by removing any leading zeros, updating its word length accordingly, and ensuring that the sign is set correctly if the bigint becomes zero.

Parameters

in, out	<i>x</i>	Pointer to the bigint to be refined (input/output).
---------	----------	---

Definition at line 139 of file utils.c.

Here is the caller graph for this function:



4.5.3.8 bi_refine_word()

```

void bi_refine_word (
    IN bigint * x,
    IN int num_words )

```

Refines the word length of a `bigint` by removing a specified number of words.

This function adjusts the word length of a `bigint` object `x` by reducing it by `num_words`. It reallocates memory if necessary and ensures the integrity of the `bigint` structure. If the result represents zero (single word with value 0), the sign is reset to `NON_NEGATIVE`.

Parameters

in	<i>x</i>	A pointer to the <code>bigint</code> object to be refined.
in	<i>num_words</i>	The number of words to remove from the end of the <code>bigint</code> .

Precondition

`x` must be a valid pointer to an initialized `bigint` object. `num_words` should not exceed the current word length of `x`.

Postcondition

The `bigint`'s word length is reduced by `num_words`, and its memory is resized accordingly. If the resulting `bigint` represents zero, its sign is reset to `NON_NEGATIVE`.

Note

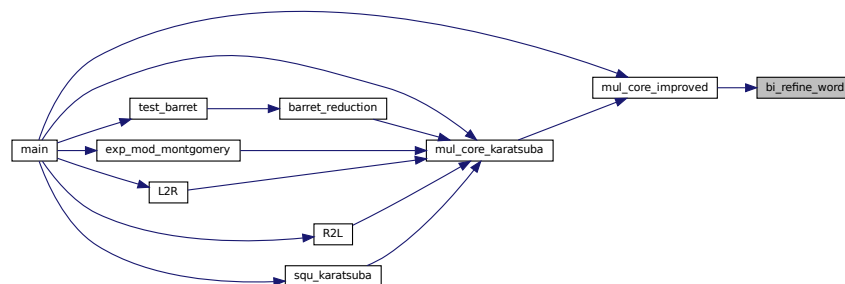
- If `x` is `NULL`, the function prints an error message (`INVALID_DATA`) and returns without performing any operation.
- The function assumes that the `bigint` structure has attributes `a` (an array of words), `wordlen` (length of the array), and `sign` (the sign of the integer).
- Memory reallocation is handled using `realloc`. Ensure proper error handling for memory operations if extended.

Warning

- If `num_words` is greater than or equal to `x->wordlen`, the behavior may be undefined.
- Always ensure that the `bigint` object `x` is valid before calling this function.

Definition at line 167 of file `utils.c`.

Here is the caller graph for this function:

**4.5.3.9 bi_reset()**

```
void bi_reset (
    INOUT bigint * x )
```

Resets all words in a `bigint` to zero.

This function iterates through all words in the given `bigint` object `x` and sets each word to zero. It does not alter the word length of the `bigint`, only the values of the words.

Parameters

in, out	<i>x</i>	A pointer to the <code>bigint</code> object to be reset.
---------	----------	--

Precondition

x must be a valid pointer to an initialized `bigint` object.

Postcondition

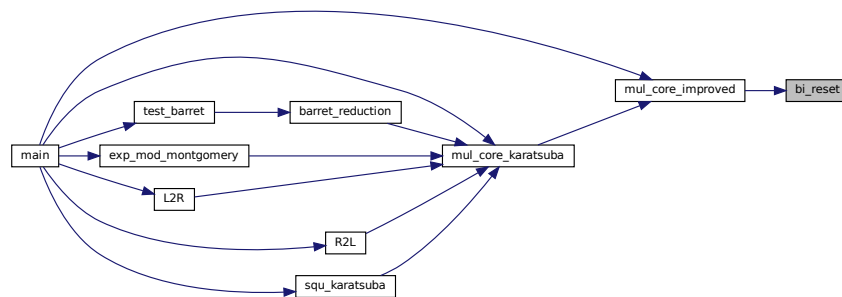
All words in the `bigint` pointed to by *x* are set to zero, but the word length remains unchanged.

Note

This function assumes that the `bigint` structure has an attribute `a` (an array of words) and `wordlen` (the number of words).

Definition at line 581 of file `utils.c`.

Here is the caller graph for this function:



4.5.3.10 bi_set_by_array()

```

int bi_set_by_array (
    OUT bigint ** x,
    IN int sign,
    IN word * a,
    IN int wordlen )

```

Initializes a `bigint` structure using an array of words and sets its sign.

Parameters

out	<i>x</i>	Pointer to the <code>bigint</code> structure to be initialized.
in	<i>sign</i>	The sign of the <code>bigint</code> (NEGATIVE or NON_NEGATIVE).
in	<i>a</i>	Array of words to set the value of the <code>bigint</code> .
in	<i>wordlen</i>	Length of the word array.

Returns

int Returns 0 on success, non-zero on failure.

Definition at line 80 of file utils.c.

Here is the call graph for this function:

**4.5.3.11 bi_set_by_string()**

```

int bi_set_by_string (
    OUT bigint ** x,
    IN int sign,
    IN char * str,
    IN int base )
  
```

Initializes a bigint structure from a character string representing a number.

Parameters

out	<i>x</i>	Pointer to the bigint structure to be initialized (output).
in	<i>sign</i>	The sign of the bigint (NEGATIVE or NON_NEGATIVE) (input).
in	<i>str</i>	The string representation of the number to be converted (input).
in	<i>base</i>	The base of the number system (e.g., 10 for decimal) (input).

Returns

int Returns 0 on success, non-zero on failure.

Definition at line 98 of file utils.c.

Here is the call graph for this function:



4.5.3.12 bi_set_one()

```
void bi_set_one (
    OUT bigint ** x )
```

Sets a bigint to the value of one.

This function initializes a new bigint structure and sets its value to one. The sign of the bigint is set to NON_NEGATIVE.

Parameters

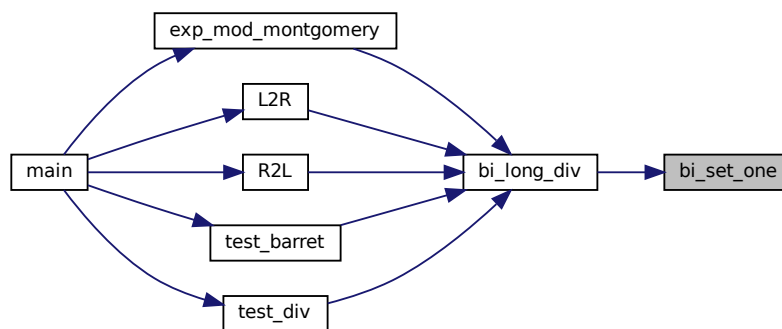
out	x	Pointer to the bigint structure to be initialized and set to one.
-----	---	---

Definition at line 227 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.13 bi_set_zero()

```
void bi_set_zero (
    OUT bigint ** x )
```

Sets a bigint to the value of zero.

This function initializes a new bigint structure and sets its value to zero. The sign of the bigint is set to NON_NEGATIVE.

Parameters

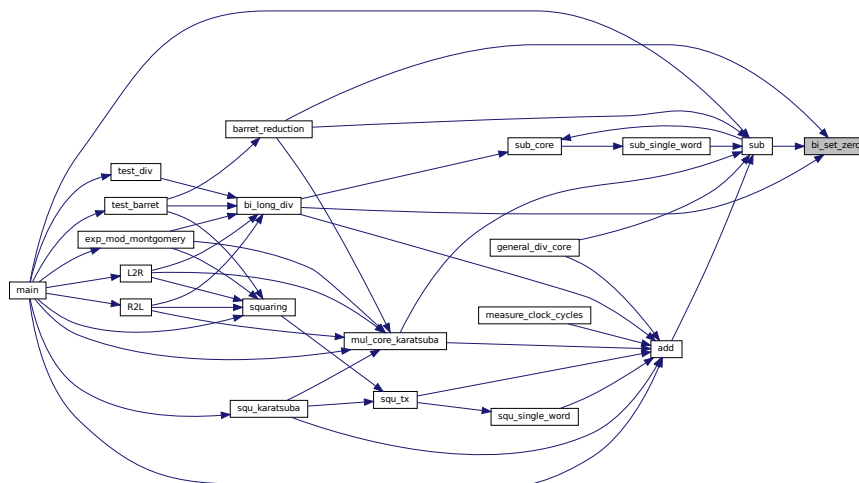
out	x	Pointer to the bigint structure to be initialized and set to zero.
-----	---	--

Definition at line 234 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**4.5.3.14 bi_show_hex()**

```
void bi_show_hex (
    IN bigint * x )
```

Displays the hexadecimal representation of a bigint.

This function prints the value of the bigint in hexadecimal format. It also handles the sign of the bigint and checks for invalid inputs.

Parameters

in	x	Pointer to the bigint to be displayed. If NULL, an error message is printed.
----	---	--

Definition at line 49 of file utils.c.

4.5.3.15 compare()

```
int compare (  
    IN bigint * x,  
    IN bigint * y )
```

Compare two bigint numbers, taking into account their signs.

This function compares two bigint numbers, considering both their magnitudes and signs.

Parameters

in	x	Pointer to the first bigint number (input).
in	y	Pointer to the second bigint number (input).

Returns

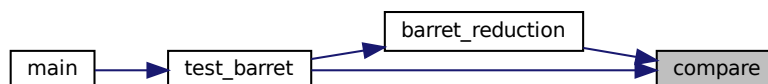
1 if $x > y$, -1 if $x < y$, or 0 if $x == y$.

Definition at line 268 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.16 compareABS()

```
int compareABS (
    IN bigint * x,
    IN bigint * y )
```

Compares the absolute values of two bigint numbers.

This function compares the absolute values of two `bigint` numbers `x` and `y`. The sign of `x` and `y` is ignored, and only the magnitudes are compared.

Parameters

in	<code>x</code>	Pointer to the first bigint number.
in	<code>y</code>	Pointer to the second bigint number.

Returns

int

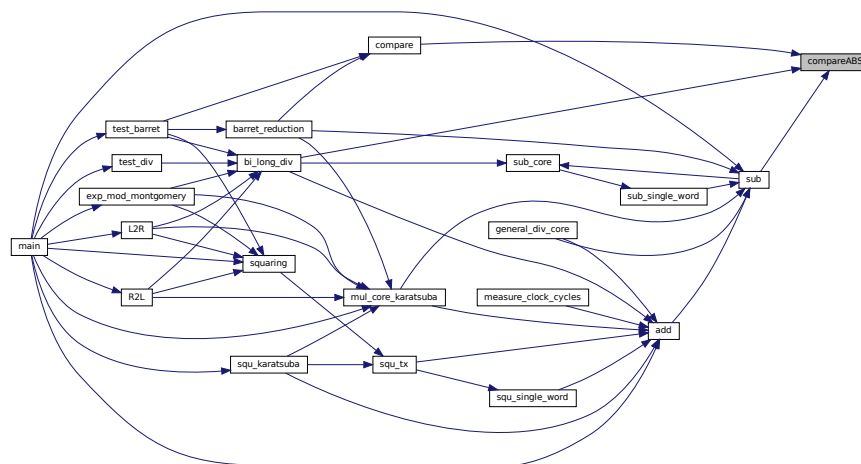
- Returns 1 if $|x| > |y|$
- Returns 0 if $|x| < |y|$
- Returns -1 if $|x| == |y|$

Note

Assumes that `x` and `y` are valid bigint pointers with non-null `a` arrays.

Definition at line 241 of file `utils.c`.

Here is the caller graph for this function:



4.5.3.17 get_bit_length()

```
int get_bit_length (
    IN bigint * x )
```

Calculate the bit length of a given bigint.

This function calculates the bit length of the absolute value of a bigint. It accounts for the number of leading zeros in the last word and adds an extra bit for the sign if the bigint is negative.

Parameters

in	x	A pointer to the bigint structure (input).
----	---	--

Returns

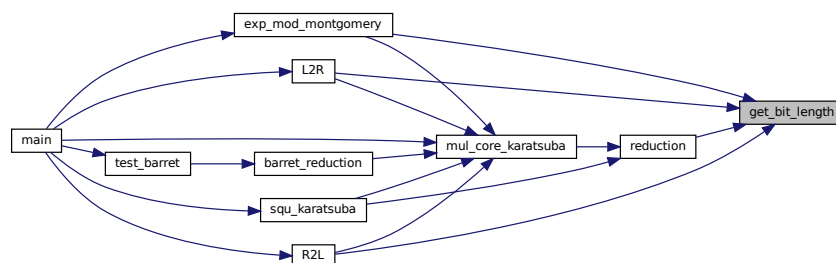
The bit length of the bigint. If there are no words, returns 0. If the bigint is negative, an extra bit for the sign is included.

Definition at line 289 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.18 get_jth_bit()

```
bool get_jth_bit (
    IN bigint * x,
    IN int j )
```

Get the value of the j -th bit from a `bigint`.

This function retrieves the value of the specified bit (j) from the given `bigint`. The indexing is 0-based, where the least significant bit is at index 0.

Parameters

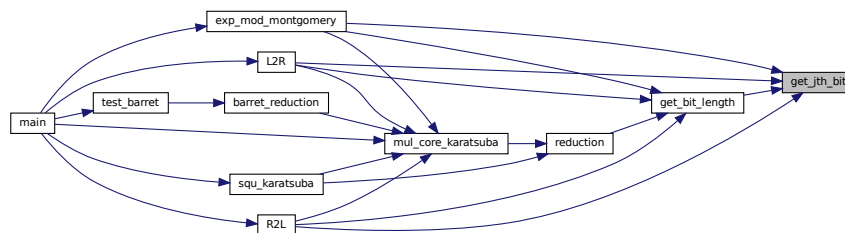
in	x	Pointer to the <code>bigint</code> structure from which the bit will be retrieved (input).
in	j	The index of the bit to retrieve (0-based) (input).

Returns

Returns 1 if the bit is set, 0 if the bit is not set, or -1 if the input is invalid (e.g., NULL pointer, out of bounds).

Definition at line 301 of file `utils.c`.

Here is the caller graph for this function:



4.5.3.19 get_word()

```
word get_word (
    IN bigint * x,
    IN int m_th )
```

Returns the value of the m -th word from a `bigint`.

This function retrieves the word at the specified index `m_th` from the `bigint`'s internal array of words. If the index is out of bounds, the function returns zero.

Parameters

in	x	A pointer to the <code>bigint</code> object from which the word will be retrieved.
in	m_{th}	The index of the word to retrieve.
	$_{th}$	

Returns

The value of the *m*-th word, or 0 if the index is out of bounds.

Precondition

x must be a valid pointer to an initialized `bigint` object.

Postcondition

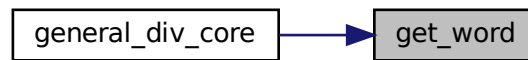
The function returns the value of the specified word in the `bigint`.

Note

This function assumes that the `bigint` structure has an attribute *a* (an array of words) and *wordlen* (the number of words).

Definition at line 587 of file `utils.c`.

Here is the caller graph for this function:

**4.5.3.20 is_zero()**

```
int is_zero (
    IN bigint * x )
```

Checks if a `bigint` is zero.

This function checks if all bits in the `bigint` *x* are zero. It iterates through each word and each bit in *x*→*a* to verify that no bit is set to 1.

Parameters

in	<i>x</i>	Pointer to the <code>bigint</code> structure to be checked.
----	----------	---

Returns

int Returns 1 if any bit in *x* is set to 1, indicating *x* is not zero. Returns 0 if all bits in *x* are zero.

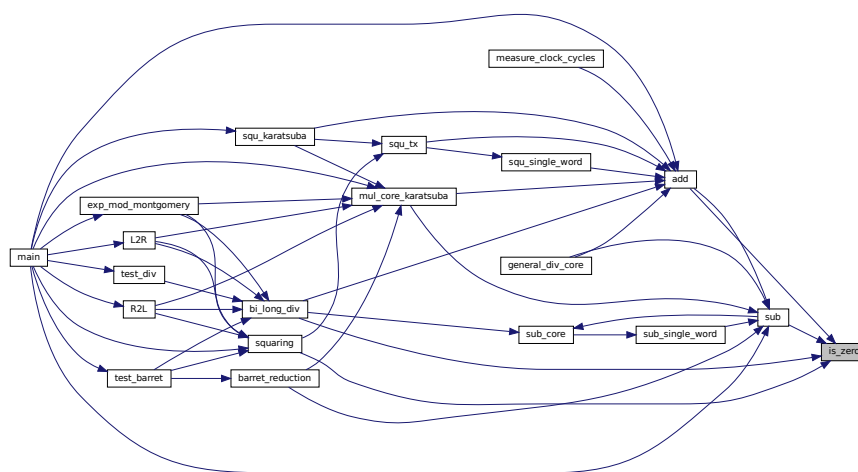
- The function uses bitwise operations to examine each bit of $x \rightarrow a$.
- If any bit in $x \rightarrow a$ is set to 1, the function returns immediately with a result of 1.
- If all bits are zero, the function returns 0, indicating that x represents zero.

Note

- The function assumes x is a valid pointer to a bigint structure.
- The use of an intermediate variable `temp` helps avoid modifying $x \rightarrow a[i]$ directly.

Definition at line 495 of file `utils.c`.

Here is the caller graph for this function:



4.5.3.21 left_shift_bit()

```
void left_shift_bit (
    bigint * pptrBint,
    int shift_amount )
```

Perform a left shift operation on a bigint.

This function shifts the bits of the given bigint to the left by a specified number of positions. It reallocates memory for the underlying array to accommodate the shift and updates the word length accordingly.

Parameters

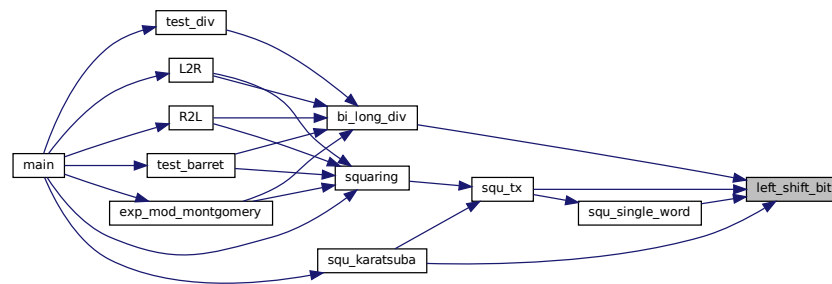
in, out	<i>x</i>	Pointer to the bigint to be shifted.
in	<i>shift</i>	The number of bits to shift to the left.

Note

The function handles memory allocation and will exit the program with an error message if memory allocation fails. It also removes any leading zero words after the shift operation.

Definition at line 340 of file utils.c.

Here is the caller graph for this function:

**4.5.3.22 left_shift_word()**

```

void left_shift_word (
    INOUT bigint ** x,
    IN int shift_words )

```

Left-shifts a bigint by a specified number of words.

This function shifts the bigint `x` to the left by the specified number of words (`shift_words`). It reallocates memory for the bigint array `(*x) -> a` to accommodate the shift and initializes any newly allocated lower-order words to zero.

Parameters

in, out	<code>x</code>	Double pointer to the bigint structure to be shifted.
in	<code>shift_words</code>	The number of words to shift <code>x</code> to the left.

Note

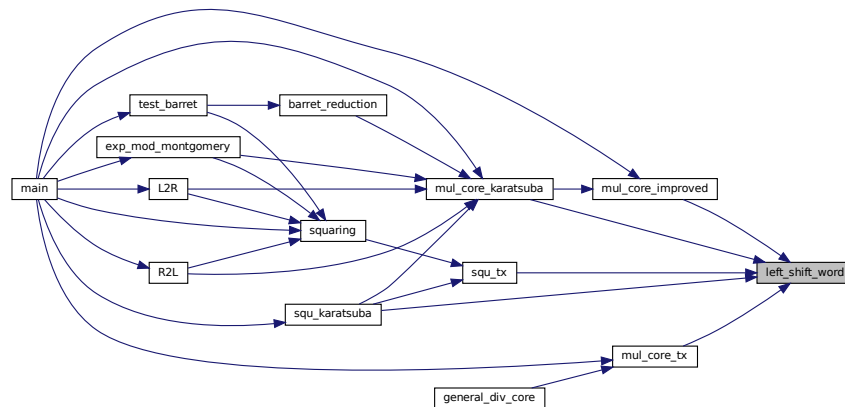
- If `shift_words` is negative, the function outputs an error message and calls `exit(1)`.
- The function reallocates memory for `(*x) -> a` to ensure there is enough space for the shift.
- The function increases the length of `(*x) -> a` by `shift_words`.
- Existing words in `(*x) -> a` are shifted to the left by `shift_words`.
- Newly added words are set to `ZERO`.
- The word length of `(*x)` is updated to the new length.

Warning

- On memory allocation failure, `exit(1)` is called.
- The function assumes `x` points to a valid `bigint` structure.

Definition at line 431 of file `utils.c`.

Here is the caller graph for this function:

**4.5.3.23 makeEven()**

```
void makeEven (
    INOUT bigint * x )
```

Ensures that the word length of a `bigint` is even.

This function checks if the word length of the given `bigint` object `x` is odd. If it is, the function increases the word length by one, reallocates memory to accommodate the new size, and initializes the newly added word to zero.

Parameters

<code>in, out</code>	<code>x</code>	A pointer to the <code>bigint</code> object to be adjusted. The structure is modified to ensure an even word length.
----------------------	----------------	--

Precondition

`x` must be a valid pointer to an initialized `bigint` object.

Postcondition

If the original word length of `x` is odd:

- The word length is increased by one.

- Memory is reallocated to match the new size.
- The new higher-order word is initialized to zero.

Note

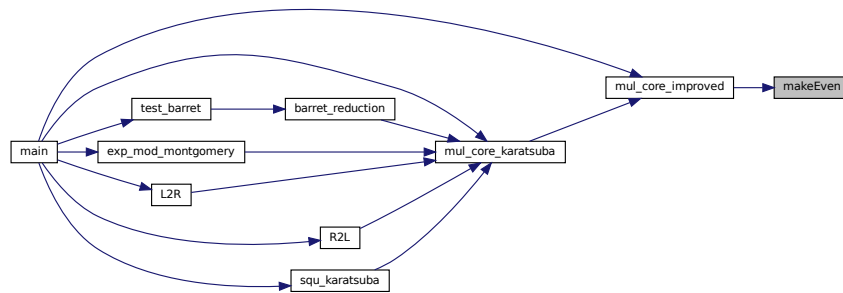
- If the word length of `x` is already even, no changes are made.
- The function assumes that the `bigint` structure has attributes `a` (an array of words) and `wordlen` (length of the array).
- Memory reallocation is handled using `realloc`. Ensure error handling for memory failures.

Warning

- If memory allocation fails during `realloc`, the program prints an error message (`MEM_ALLOCATION_FAIL`) and exits.
- This function modifies the memory directly, so ensure there are no conflicting references to the same `bigint`.

Definition at line 523 of file `utlis.c`.

Here is the caller graph for this function:



4.5.3.24 match_wordlen()

```

void match_wordlen (
    INOUT bigint * x,
    INOUT bigint * y )

```

Matches the word lengths of two `bigint` objects by resizing them to the larger of their current lengths.

This function adjusts the word lengths of two `bigint` objects `x` and `y` so that both have the same word length. If either `x` or `y` has a smaller word length than the maximum of the two, it reallocates memory and initializes the newly added words to zero.

Parameters

in, out	<i>x</i>	A pointer to the first <code>bigint</code> object. Its word length and memory may be modified to match the other <code>bigint</code> .
in, out	<i>y</i>	A pointer to the second <code>bigint</code> object. Its word length and memory may be modified to match the other <code>bigint</code> .

Precondition

Both *x* and *y* must point to valid, initialized `bigint` objects.

Postcondition

- The word lengths of *x* and *y* are equal and match the larger of their original word lengths.
- Memory is reallocated for either *x* or *y* (or both) if necessary.
- Any newly added words are initialized to zero.

Note

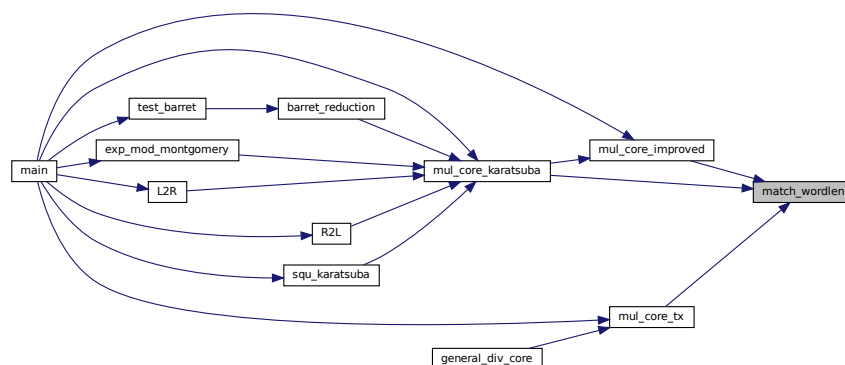
- The function assumes that the `bigint` structure has attributes `a` (an array of words) and `wordlen` (length of the array).
- Memory reallocation is performed using `realloc`. Proper error handling is in place to terminate the program if allocation fails.

Warning

- If memory allocation fails during `realloc`, the program prints an error message (`MEM_ALLOCATION_FAIL`) and exits.
- Ensure no conflicting references to the same `bigint` objects are active when calling this function.

Definition at line 541 of file `utils.c`.

Here is the caller graph for this function:



4.5.3.25 reduction()

```
void reduction (
    IN bigint ** x,
    IN int r )
```

Perform a reduction operation on a bigint.

This function reduces the given bigint A by shifting its bits to the right by a specified number of positions r. The result is stored in the provided bigint result. If r is greater than or equal to the total bit length of A, the result will be zero. The sign of the result will match the sign of A.

Parameters

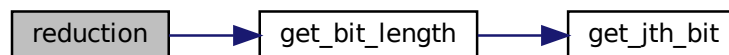
in	<i>x</i>	Pointer to the bigint to be reduced.
in	<i>r</i>	The number of bits to shift to the right.
out	<i>result</i>	Pointer to the bigint where the result will be stored.

Note

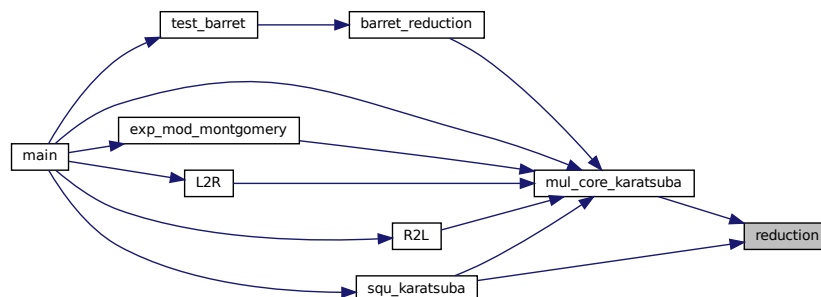
Memory for the result bigint is allocated inside this function. The caller is responsible for freeing the memory used by result.

Definition at line 380 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.26 right_shift_bit()

```
void right_shift_bit (
    INOUT bigint * x,
    IN int shift )
```

Perform a right shift operation on a bigint.

This function shifts the bits of the given bigint to the right by a specified number of positions. It reallocates memory for the underlying array to accommodate the shift and updates the word length accordingly. If the shift results in fewer than one word, the function ensures that at least one word remains.

Parameters

in, out	<i>x</i>	Pointer to the bigint to be shifted (input/output).
in	<i>shift</i>	The number of bits to shift to the right (input).

Note

The function handles memory allocation and will exit the program with an error message if memory allocation fails. It also removes any leading zero words after the shift operation.

Definition at line 309 of file utils.c.

Here is the call graph for this function:



4.5.3.27 right_shift_word()

```
void right_shift_word (
    INOUT bigint ** x,
    IN int shift_words )
```

Right-shifts a bigint by a specified number of words.

This function shifts the bigint *x* to the right by the specified number of words (*shift_words*). It reallocates memory for the bigint array *x*→*a* to accommodate the shift and initializes any newly allocated words to zero.

Parameters

in, out	<i>x</i>	Pointer to the bigint structure to be shifted.
in	<i>shift_words</i>	The number of words to shift <i>x</i> to the right.

Note

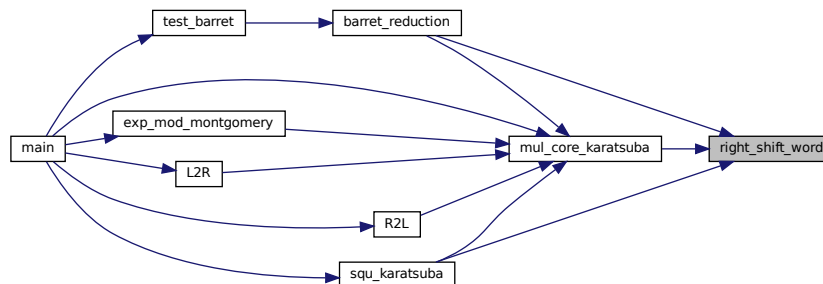
- If memory reallocation fails, the function outputs `MEM_ALLOCATION_FAIL` and calls `exit(1)`.
- The function assumes that `x` and `x->a` are valid pointers with allocated memory.
- The function increases the length of `x->a` by `shift_words`.
- After reallocation, newly added words in `x->a` are set to `ZERO`.
- The sign of `x` remains unchanged.

Warning

- `x->a` is reallocated, and its size is modified to include the shifted words.
- On memory allocation failure, `exit(1)` is called.

Definition at line 462 of file `utils.c`.

Here is the caller graph for this function:

**4.5.3.28 swap_bigint()**

```

void swap_bigint (
    INOUT bigint ** x,
    INOUT bigint ** y )

```

Swaps the values of two `bigint` pointers.

Exchanges the pointers of two `bigint` objects `x` and `y`. This effectively swaps the two `bigint` structures without copying their contents.

Parameters

in, out	<code>x</code>	A double pointer to the first <code>bigint</code> object.
in, out	<code>y</code>	A double pointer to the second <code>bigint</code> object.

Precondition

`x` and `y` must be valid, initialized double pointers to `bigint` objects.

Postcondition

The values of `*x` and `*y` are swapped. The `bigint` objects they point to are unchanged, but the pointers themselves are exchanged.

Note

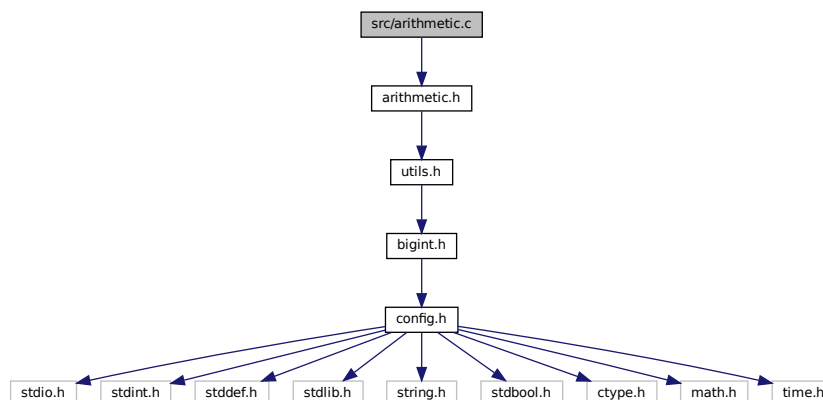
This function operates only on the pointers and does not modify the actual contents of the `bigint` objects.

Definition at line 515 of file `utils.c`.

4.6 src/arithmetic.c File Reference

```
#include "arithmetic.h"
```

Include dependency graph for `arithmetic.c`:

**Functions**

- void `bigint_or` (`bigint **x`, `bigint **y`, `bigint **z`)
- void `add_single_word` (`IN word A`, `IN word B`, `IN word carry_in`, `OUT word *carry_out`, `OUT word *result`)
Adds two words with carry handling.
- void `add_core` (`IN bigint **x`, `IN bigint **y`, `OUT bigint **z`)
Adds two big integers and stores the result in a third bigint.
- void `add` (`IN bigint **x`, `IN bigint **y`, `OUT bigint **z`)
Adds two big integers `x` and `y`, and stores the result in `z`.
- void `sub_single_word` (`IN word A`, `IN word B`, `IN word borrow_in`, `OUT word *borrow_out`, `OUT word *result`)
Performs subtraction of `B` from `A` and manages borrowing.
- void `sub_core` (`IN bigint **x`, `IN bigint **y`, `OUT bigint **z`)
Performs subtraction of two big integers.

- void `sub` (IN bigint **x, IN bigint **y, OUT bigint **z)
Subtracts big integer y from big integer x and stores the result in z.
- void `mul_single_word` (IN word A, IN word B, OUT bigint **result)
Multiplies two single words and stores the result in a bigint structure.
- void `mul_core_tx` (IN bigint **x, IN bigint **y, OUT bigint **z)
Multiplies two big integers and stores the result in the output.
- void `mul_core_improved` (IN bigint **x, IN bigint **y, OUT bigint **z)
Multiplies two binary integers using an improved Textbook Multiplication algorithm.
- void `mul_core_karatsuba` (IN bigint **x, IN bigint **y, OUT bigint **z)
Multiplies two binary integers using the Karatsuba multiplication algorithm.
- void `squ_single_word` (IN word A, OUT bigint **result)
Squares a single word and stores the result in a bigint structure.
- void `squ_tx` (IN bigint **x, OUT bigint **result)
Squares a bigint and stores the result in another bigint.
- void `squaring` (IN bigint **x, OUT bigint **result)
Squares a bigint if it is non-zero and stores the result.
- void `squ_karatsuba` (IN bigint **x, OUT bigint **z)
Computes the square of a binary integer using the Karatsuba squaring algorithm.
- void `bi_long_div` (IN bigint **x, IN bigint **y, OUT bigint **q, OUT bigint **r)
Performs binary long division on big integers.
- void `word_quotient` (IN word dividend1, IN word dividend0, IN word divisor)
Computes the quotient of a two-word dividend divided by a single-word divisor.
- void `general_div_core` (IN bigint **x, IN bigint **y, IN bigint **Q, IN bigint **R)
Performs division of two large integers using an optimized algorithm.
- void `L2R` (IN bigint **x, IN bigint **y, IN bigint **Z, OUT bigint **M)
Computes modular exponentiation using the Left-to-Right binary method.
- void `R2L` (IN bigint **x, IN bigint **y, OUT bigint **Z, IN bigint **M)
Computes modular exponentiation using the Right-to-Left binary method.
- void `exp_mod_montgomery` (IN bigint **x, IN bigint **y, OUT bigint **z, IN bigint **M)
Performs modular exponentiation using Montgomery multiplication.
- void `barret_reduction` (IN bigint **x, IN bigint **y, IN bigint **z, OUT bigint **result)
Performs Barrett reduction to reduce x modulo y .

4.6.1 Function Documentation

4.6.1.1 `add()`

```
void add (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Adds two big integers x and y, and stores the result in z.

The function handles cases where one or both of the operands are zero, as well as when the signs of x and y are different. If the signs of x and y are different, it performs subtraction instead. If both have the same sign, it performs addition.

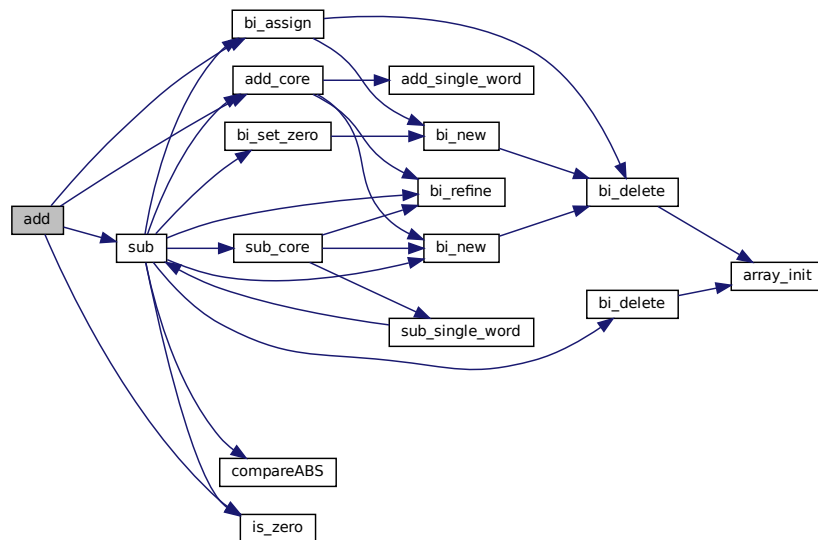
Parameters

in	x	Pointer to a pointer of the first bigint structure.
in	y	Pointer to a pointer of the second bigint structure.
out	z	Pointer to a pointer of the result bigint structure.

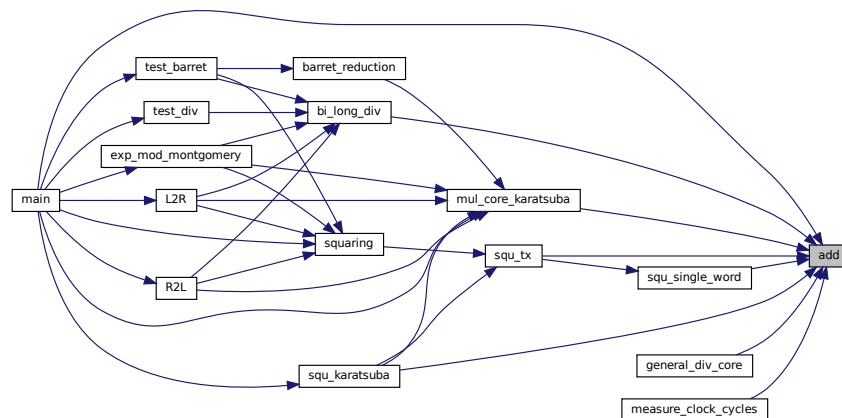
- If x is zero, the function assigns y to z.
- If y is zero, the function assigns x to z.
- If x and y have opposite signs, the function subtracts the smaller magnitude from the larger one.
- If x and y have the same sign, the function adds them and assigns the resulting sign to z.

Definition at line 79 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.2 add_core()

```
void add_core (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Adds two big integers and stores the result in a third bigint.

This function takes two big integers represented as `bigint` structures, adds them together, and stores the resulting `bigint` in a third `bigint`. It ensures that both numbers have the same length by padding with zeros if necessary. The addition is performed in reverse order to handle carry appropriately.

Parameters

in	x	Pointer to the first bigint to add.
in	y	Pointer to the second bigint to add.
out	z	Pointer to the result bigint where the sum will be stored.

This function performs the following steps:

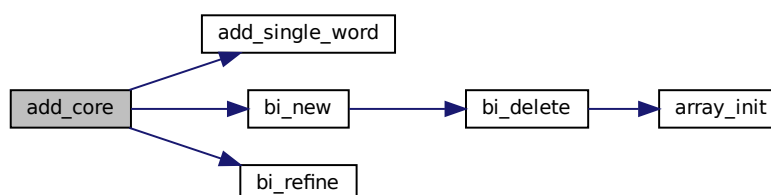
- Determines the lengths of the input big integers.
- Allocates memory for the result bigint based on the maximum length of the inputs.
- Iteratively adds each corresponding word from the two input big integers, starting from the least significant word, while handling any carry from previous additions.
- Stores the result in the appropriate position in the result bigint.
- Handles any carry that may remain after the final addition.
- Calls `bi_refine` to remove any leading zeros from the result.

Note

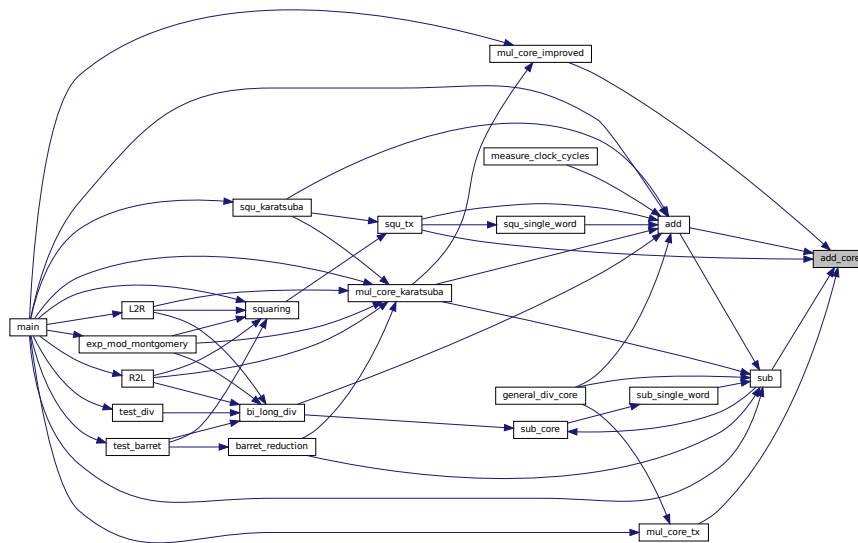
The input big integers should be properly initialized and should not be NULL. The result bigint will be allocated and must be freed by the caller when no longer needed.

Definition at line 46 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.3 add_single_word()

```
void add_single_word (
    IN word A,
    IN word B,
    IN word carry_in,
    OUT word * carry_out,
    OUT word * result )
```

Adds two words with carry handling.

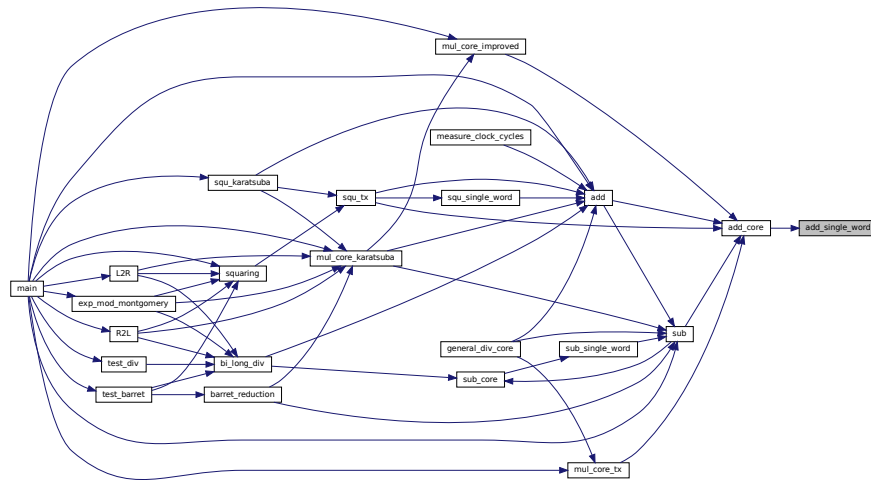
This function performs addition on two words (A and B) and incorporates a carry from a previous addition. It checks for overflow and sets the carry_out accordingly.

Parameters

in	<i>A</i>	First word to add.
in	<i>B</i>	Second word to add.
in	<i>carry_in</i>	Carry input from a previous addition.
out	<i>carry_out</i>	Pointer to store the carry output.
out	<i>result</i>	Pointer to store the result of the addition.

Definition at line 28 of file arithmetic.c.

Here is the caller graph for this function:



4.6.1.4 barret_reduction()

```

void barret_reduction (
    IN bigint ** x,
    IN bigint ** y,
    IN bigint ** z,
    OUT bigint ** result )

```

Performs Barrett reduction to reduce x modulo y .

This function computes the result of $(x \bmod y)$ using the Barrett reduction method. Barrett reduction is efficient for modular operations, especially when dealing with repeated modular reductions during large integer arithmetic (e.g., in cryptographic algorithms). It involves approximating the inverse of y modulo a power of two and using it to reduce x modulo y .

Parameters

in	x	A double pointer to a <code>bigint</code> representing the dividend.
in	y	A double pointer to a <code>bigint</code> representing the divisor.
in	z	A double pointer to a <code>bigint</code> representing the precomputed value used in the reduction.
out	$result$	A double pointer to a <code>bigint</code> where the result of the reduction will be stored.

Precondition

Both x and y must point to valid, initialized `bigint` objects. y must not be zero.

Postcondition

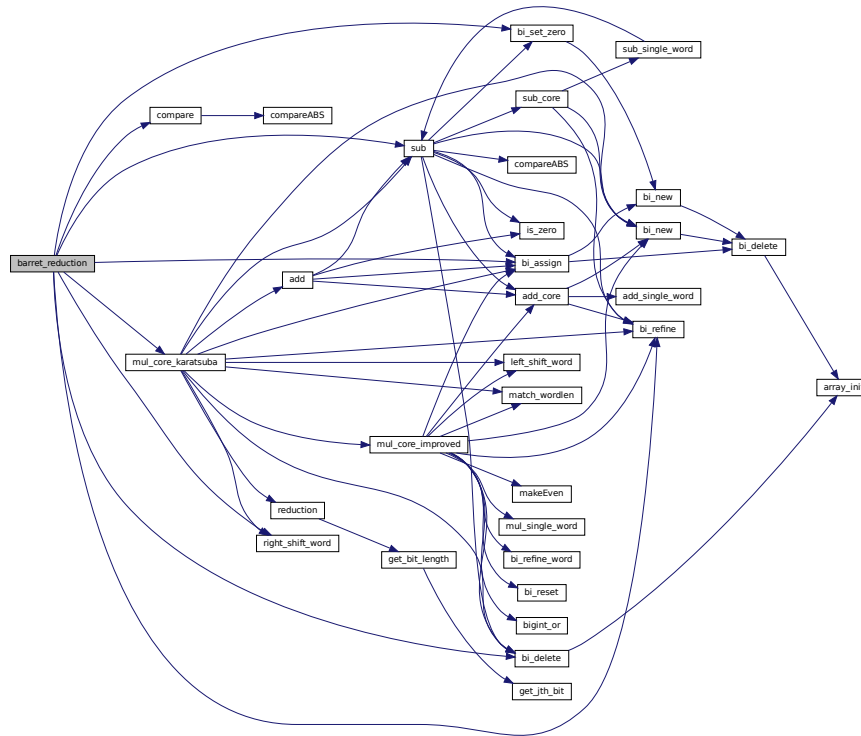
The result of $(x \bmod y)$ is stored in the location pointed to by $result$.

Note

The function utilizes an approximation method to compute the modular result more efficiently. The reduction steps are performed using several intermediate variables to prevent overflow and optimize calculations.

Definition at line 952 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.5 bi_long_div()

```

void bi_long_div (
    IN bigint ** x,
    IN bigint ** y,

```

```
OUT bigint ** q,
OUT bigint ** r )
```

Performs binary long division on big integers.

This function calculates the quotient (q) and remainder (r) when dividing bigint $*x$ by bigint $*y$. It handles special cases for zero and negative values and manages sign adjustments for the result.

Parameters

in, out	x	Pointer to the dividend bigint. This value is read-only in the function.
in	y	Pointer to the divisor bigint. This value is read-only in the function.
out	q	Pointer to the quotient bigint, which will store the result of the division.
out	r	Pointer to the remainder bigint, which will store the remainder after division.

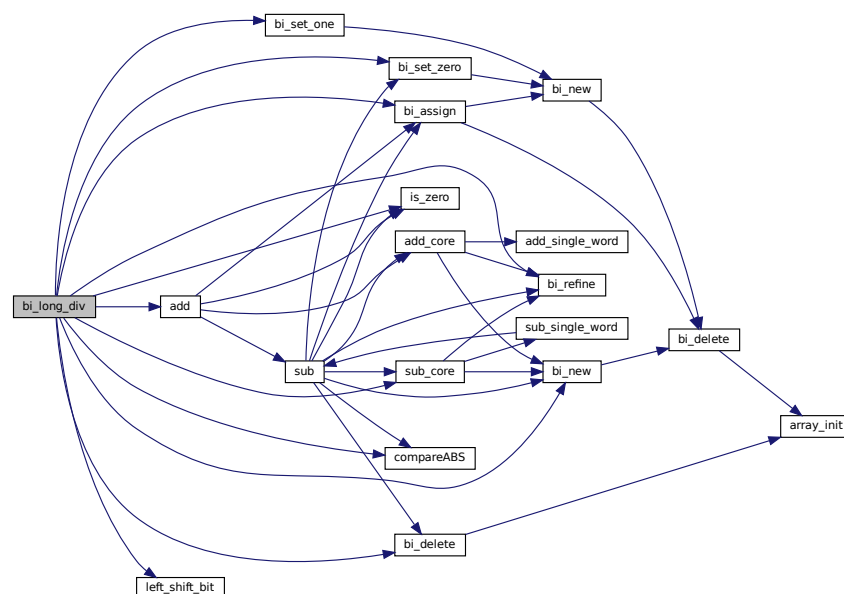
- If the divisor $*y$ is zero, the function triggers an error since division by zero is undefined.
- If $*x$ is zero, both quotient $*q$ and remainder $*r$ are set to zero.
- If the absolute value of $*x$ is smaller than $*y$, $*q$ is set to zero and $*r$ to $*x$.
- If the absolute value of $*x$ equals $*y$, $*q$ is set to one and $*r$ to zero, with sign adjustments if necessary.
- The main division loop shifts bits from $*x$ into $*r$, checks if $*r \geq *y$, and updates $*q$ and $*r$ accordingly.
- For negative dividends, the remainder $*r$ is adjusted to ensure the remainder sign follows division rules.

Note

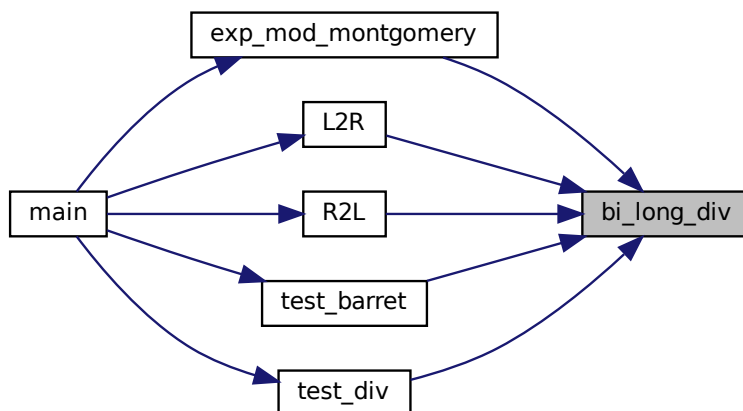
- The function uses helper functions like `bi_set_zero`, `bi_assign`, `add`, `sub_core`, and `left_shift_bit` to perform operations on bigint structures.
- The sign of the quotient $*q$ and remainder $*r$ is adjusted based on the signs of $*x$ and $*y$.
- The function assumes bigint pointers x , y , q , and r are properly initialized before calling.

Definition at line 623 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



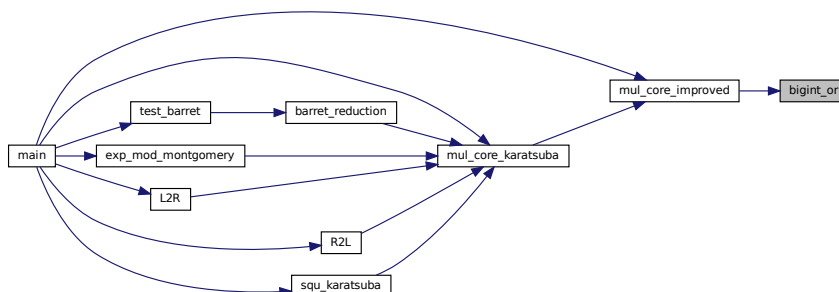
4.6.1.6 bigint_or()

```

void bigint_or (
    bigint ** x,
    bigint ** y,
    bigint ** z )
  
```

Definition at line 3 of file arithmetic.c.

Here is the caller graph for this function:



4.6.1.7 exp_mod_montgomery()

```
void exp_mod_montgomery (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z,
    IN bigint ** M )
```

Performs modular exponentiation using Montgomery multiplication.

This function computes $(z = x^y \bmod M)$ using the Montgomery multiplication method. It iterates over the bits of the exponent y from the most significant bit to the least significant bit, maintaining intermediate modular reductions to ensure efficient computation.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the base.
in	y	A double pointer to a <code>bigint</code> representing the exponent.
out	z	A double pointer to a <code>bigint</code> where the result will be stored.
in	M	A double pointer to a <code>bigint</code> representing the modulus.

Precondition

Both x , y , and M must point to valid, initialized `bigint` objects. M must be non-zero.

Postcondition

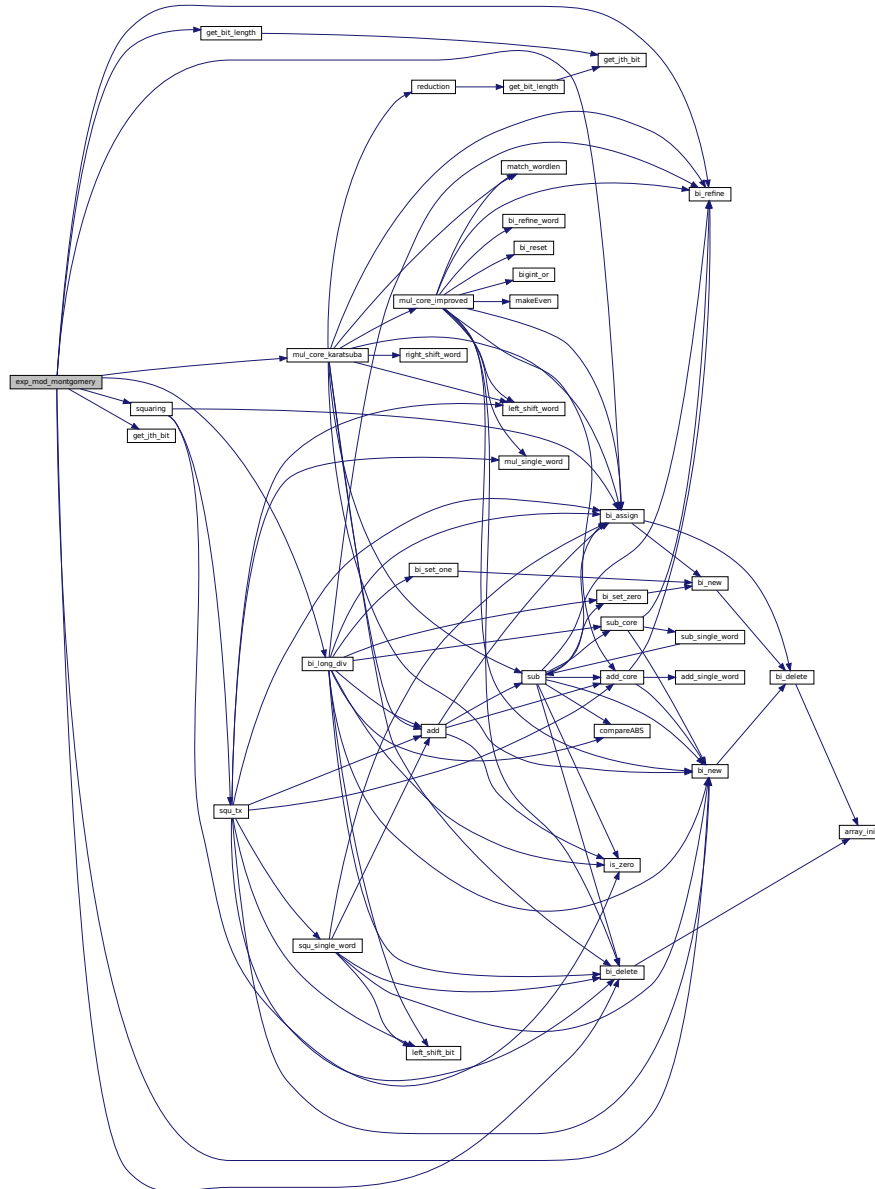
The result $(z = x^y \bmod M)$ is stored in the location pointed to by z .

Note

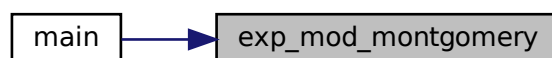
This implementation uses Montgomery multiplication for efficient modular reduction at each step.

Definition at line 913 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.8 general_div_core()

```
void general_div_core (
    IN bigint ** x,
    IN bigint ** y,
    IN bigint ** Q,
    IN bigint ** R )
```

Performs division of two large integers using an optimized algorithm.

This function calculates the quotient (Q) and remainder (R) when a large integer x is divided by y . It handles cases where the word lengths of x and y differ by at most one and adjusts results to ensure the remainder (R) is non-negative. This function also refines results to account for any adjustments due to negative remainders.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the dividend.
in	y	A double pointer to a <code>bigint</code> representing the divisor.
out	Q	A double pointer to a <code>bigint</code> where the quotient will be stored.
out	R	A double pointer to a <code>bigint</code> where the remainder will be stored.

Precondition

Both x and y must point to valid, initialized `bigint` objects, and y must not be zero. Q and R must be properly allocated to store the resulting values.

Postcondition

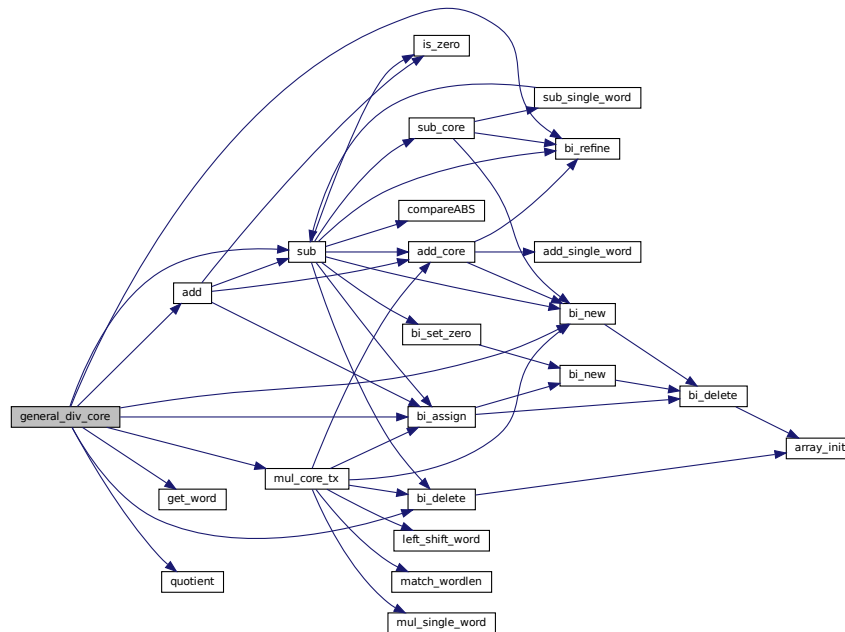
The quotient and remainder of x / y are stored in Q and R , respectively. The remainder R is guaranteed to be non-negative.

Note

This function uses helper functions such as `mul_core_tx` for multiplication, `sub` for subtraction, and `add` for addition. Memory management and refinement (`bi_refine`) are handled to ensure intermediate results are properly cleaned.

Definition at line 769 of file `arithmetic.c`.

Here is the call graph for this function:



4.6.1.9 L2R()

```
void L2R (
    IN bigint ** x,
    IN bigint ** y,
    IN bigint ** z,
    OUT bigint ** M )
```

Computes modular exponentiation using the Left-to-Right binary method.

This function calculates $(z = x^y \bmod M)$ efficiently using a left-to-right binary exponentiation algorithm. It processes the bits of y from the most significant to the least significant bit, squaring the intermediate result and conditionally multiplying based on the value of the current bit.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the base.
in	y	A double pointer to a <code>bigint</code> representing the exponent.
out	z	A double pointer to a <code>bigint</code> where the result will be stored.
in	M	A double pointer to a <code>bigint</code> representing the modulus.

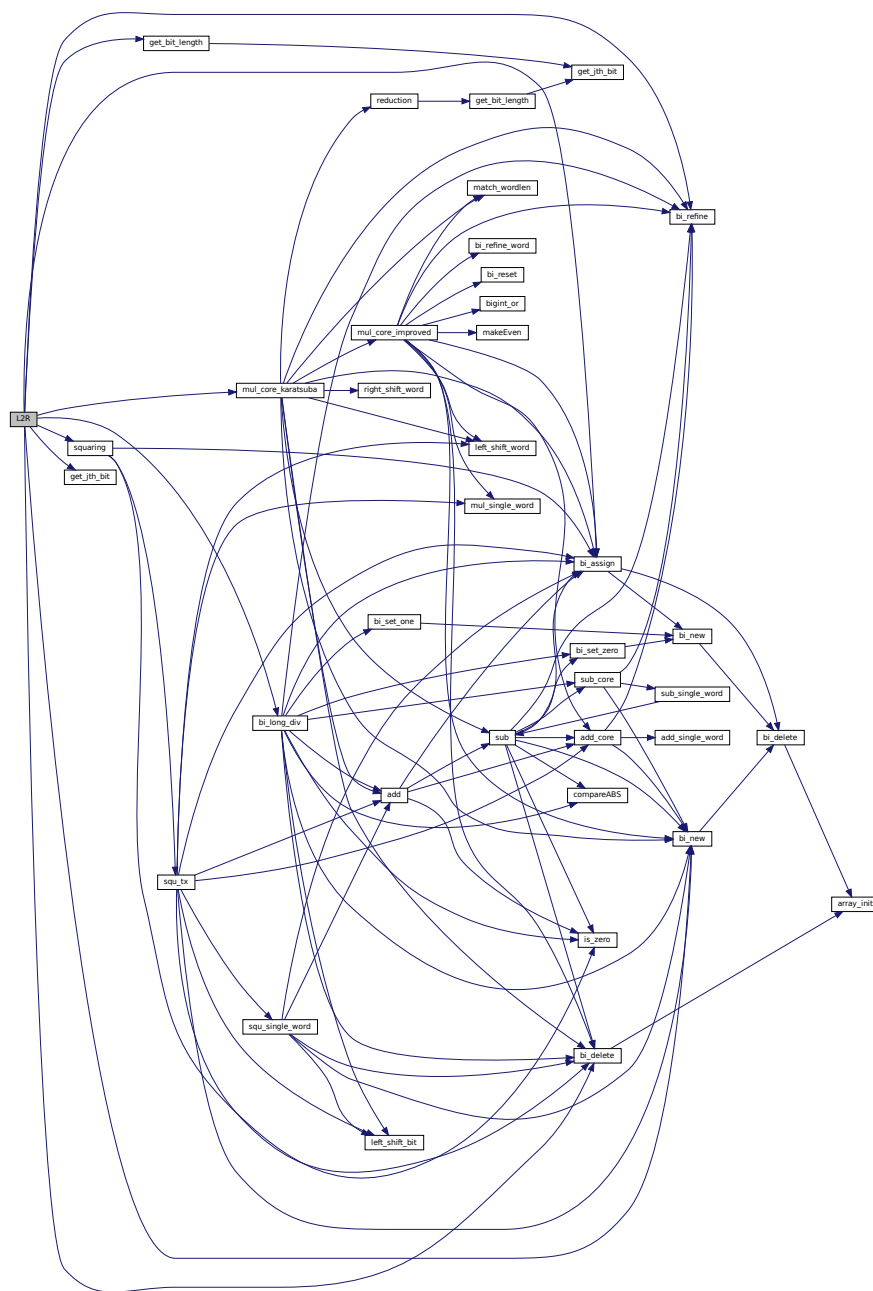
Precondition

Both x , y , and M must point to valid, initialized `bigint` objects. M must be non-zero.

The result ($z = x^y \bmod M$) is stored in the location pointed to by z .

This function uses modular reduction during every squaring and multiplication step to ensure that intermediate results remain manageable in size.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.10 mul_core_improved()

```

void mul_core_improved (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
  
```

Multiplies two binary integers using an improved Textbook Multiplication algorithm.

Wrapper function that performs optimized multiplication of two BINTs pointed to by `x` and `y`, and stores the result in `z`. This function improves the standard Textbook Multiplication by aligning word lengths, splitting calculations into smaller chunks, and handling shifts efficiently.

Parameters

in	<code>x</code>	A double pointer to a BINT representing the first operand.
in	<code>y</code>	A double pointer to a BINT representing the second operand.
out	<code>z</code>	A double pointer to a BINT where the result will be stored.

Precondition

Both `x` and `y` must point to valid, initialized BINT objects. `z` must be properly allocated to store the resulting BINT.

Postcondition

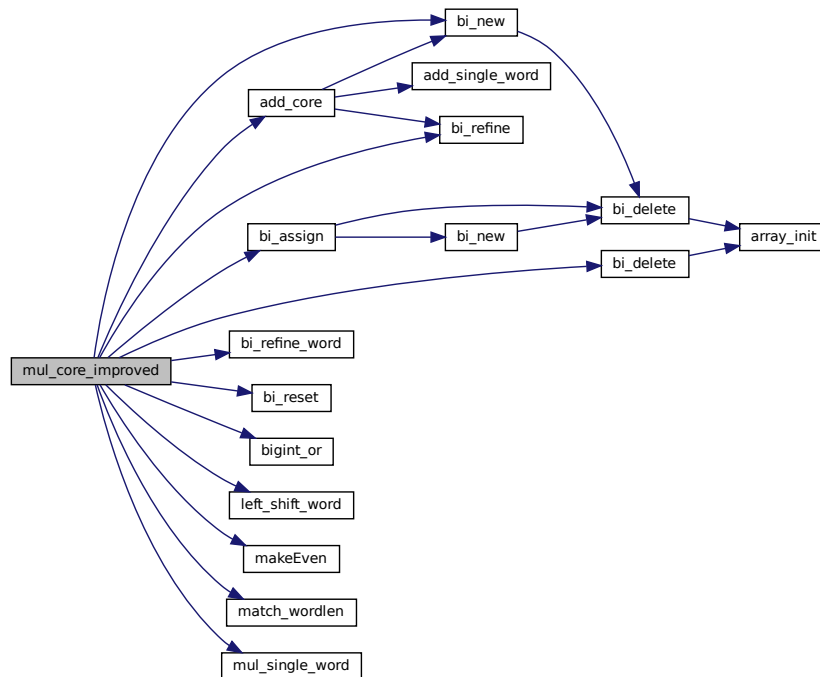
The result of multiplication is stored in the location pointed to by `z`.

Note

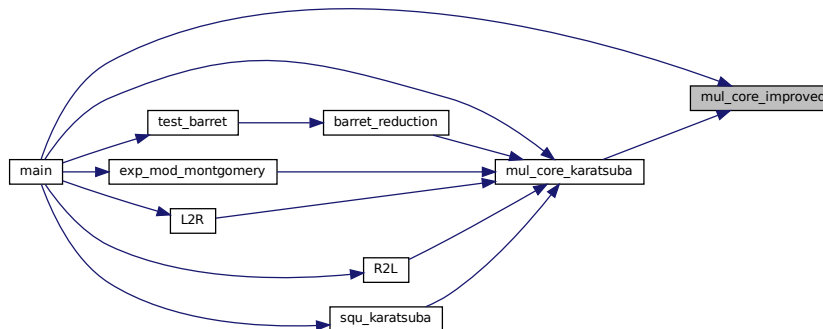
This function uses multiple helper functions, such as `mul_single_word`, `add_core`, and `left_↔shift_word`, to manage BINT arithmetic and memory efficiently.

Definition at line 326 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.11 mul_core_karatsuba()

```

void mul_core_karatsuba (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )

```

Multiplies two binary integers using the Karatsuba multiplication algorithm.

This function performs efficient multiplication of two binary integers (x and y) using the divide-and-conquer Karatsuba algorithm. It recursively splits the inputs into higher and lower parts, computes intermediate products, and combines them to calculate the final result. For small input sizes, it falls back to an improved standard multiplication (`mul_core_improved`) for efficiency.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the first operand.
in	y	A double pointer to a <code>bigint</code> representing the second operand.
out	z	A double pointer to a <code>bigint</code> where the result of the multiplication will be stored.

Precondition

Both x and y must point to valid, initialized `bigint` objects. z must be properly allocated to store the resulting `bigint`.

Postcondition

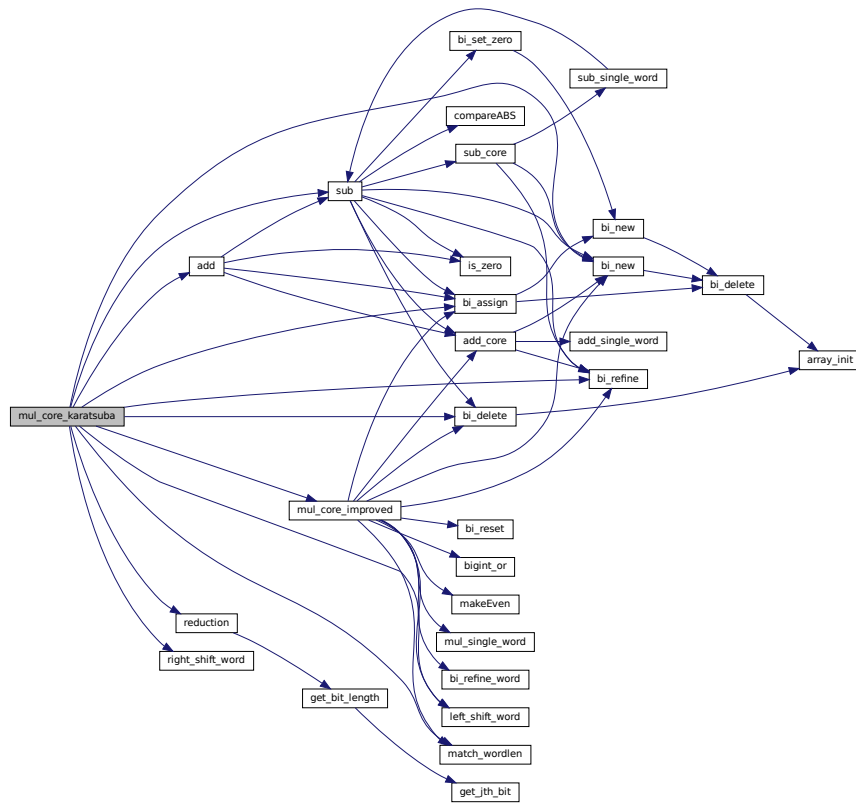
The result of the multiplication is stored in the location pointed to by z . Both x and y are refined back to their original states.

Note

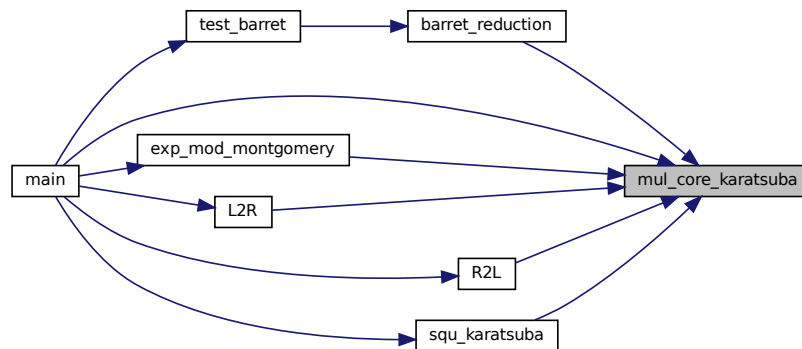
This function employs memory-efficient techniques for intermediate calculations, and uses helper functions such as `right_shift_word`, `reduction`, and `add` to manage the binary integer arithmetic.

Definition at line 399 of file `arithmetic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.12 mul_core_tx()

```
void mul_core_tx (
    IN bigint ** x,
```

```

    IN bigint ** y,
    OUT bigint ** z )

```

Multiplies two big integers and stores the result in the output.

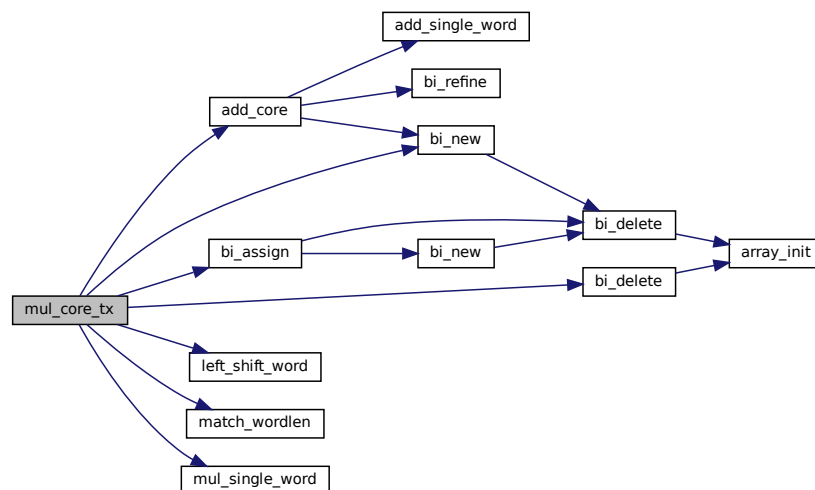
This function performs multiplication of two big integers, x and y , and stores the result in z . It adjusts for the sign of the result and handles multiplication at the word level with shifting and addition.

Parameters

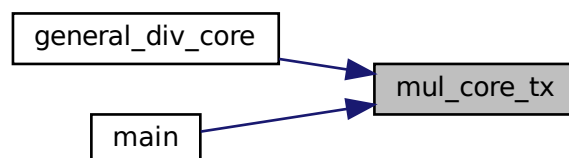
in	x	Pointer to a pointer of the first bigint operand.
in	y	Pointer to a pointer of the second bigint operand.
out	z	Pointer to a pointer of the resulting bigint product.

Definition at line 295 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.13 mul_single_word()

```
void mul_single_word (
    IN word A,
    IN word B,
    OUT bigint ** result )
```

Multiplies two single words and stores the result in a bigint structure.

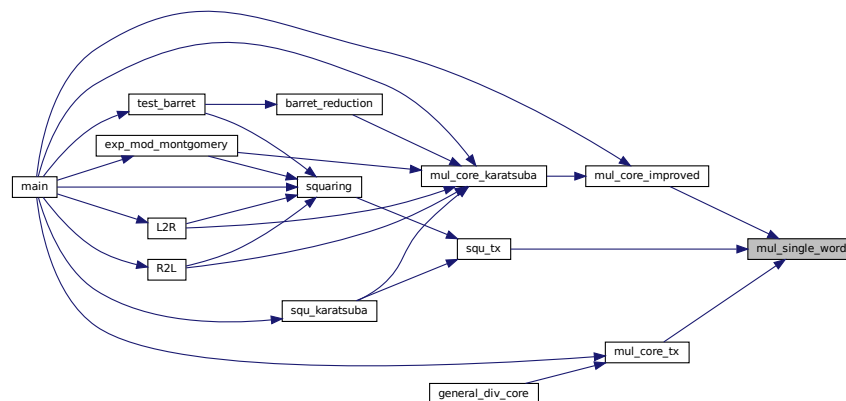
This function performs multiplication on two input words, *A* and *B*, and stores the result as a two-word bigint in the *result* parameter. It breaks down the multiplication into half-word operations to manage potential overflow in high and low parts separately, allowing for more precise handling of large word multiplications.

Parameters

in	<i>A</i>	The first input word.
in	<i>B</i>	The second input word.
out	<i>result</i>	Pointer to a bigint structure where the result will be stored. The bigint structure is expected to have space for two words (<i>a</i> [0] for the lower part and <i>a</i> [1] for the upper part).

Definition at line 266 of file arithmetic.c.

Here is the caller graph for this function:



4.6.1.14 quotient()

```
word quotient (
    IN word dividend1,
    IN word dividend0,
    IN word divisor )
```

Computes the quotient of a two-word dividend divided by a single-word divisor.

This function performs high-precision division using a two-word dividend (*dividend1* as the high word and *dividend0* as the low word) and a single-word divisor. The algorithm calculates the quotient (*Q*) and ensures that intermediate remainders (*R*) are handled appropriately during bit-by-bit processing.

Parameters

in	<i>dividend1</i>	The higher significant word of the two-word dividend.
in	<i>dividend0</i>	The lower significant word of the two-word dividend.
in	<i>divisor</i>	A single-word divisor.

Returns

The quotient resulting from the division of the two-word dividend by the divisor.

Precondition

`divisor` must be non-zero to avoid division by zero errors.

Postcondition

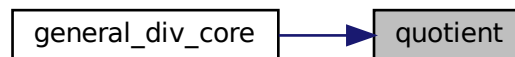
The returned quotient is the result of the division, and intermediate calculations ensure correctness.

Note

This function processes the division bit-by-bit, starting from the most significant bits and adjusting the remainder (R) and quotient (Q) iteratively.

Definition at line 739 of file arithmetic.c.

Here is the caller graph for this function:

**4.6.1.15 R2L()**

```

void R2L (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z,
    IN bigint ** M )
  
```

Computes modular exponentiation using the Right-to-Left binary method.

This function calculates $(z = x^y \bmod M)$ using the right-to-left binary exponentiation algorithm. It iterates over the bits of `y` from the least significant to the most significant, updating the intermediate result and modular reduction efficiently at each step.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the base.
in	y	A double pointer to a <code>bigint</code> representing the exponent.
out	z	A double pointer to a <code>bigint</code> where the result will be stored.
in	M	A double pointer to a <code>bigint</code> representing the modulus.

Precondition

Both x , y , and M must point to valid, initialized `bigint` objects. M must be non-zero.

Postcondition

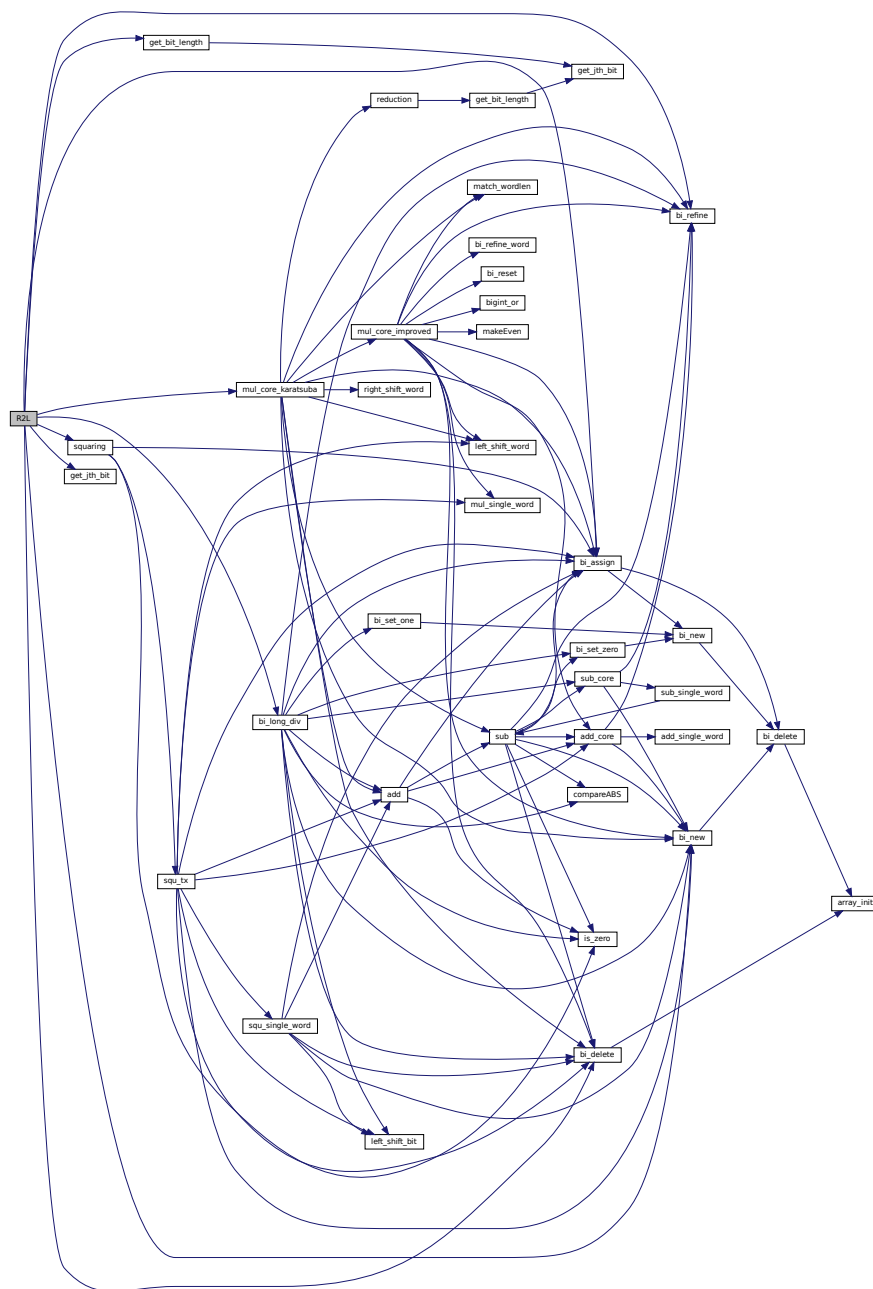
The result ($z = x^y \bmod M$) is stored in the location pointed to by z .

Note

This function employs modular reduction during each multiplication and squaring step to maintain manageable intermediate values and optimize performance.

Definition at line 877 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.16 `squ_karatsuba()`

```
void squ_karatsuba (
    IN bigint ** x,
    OUT bigint ** z )
```

Computes the square of a binary integer using the Karatsuba squaring algorithm.

This function efficiently computes the square of a binary integer `x` by utilizing the Karatsuba algorithm, which reduces the number of multiplications compared to traditional squaring methods. For small input sizes, the function falls back to a standard squaring algorithm (`squ_tx`) for performance.

Parameters

in	x	A double pointer to a <code>bigint</code> representing the binary integer to be squared.
out	z	A double pointer to a <code>bigint</code> where the result of the squaring operation will be stored.

Precondition

`x` must point to a valid, initialized `bigint` object. `z` must be properly allocated to store the resulting `bigint`.

Postcondition

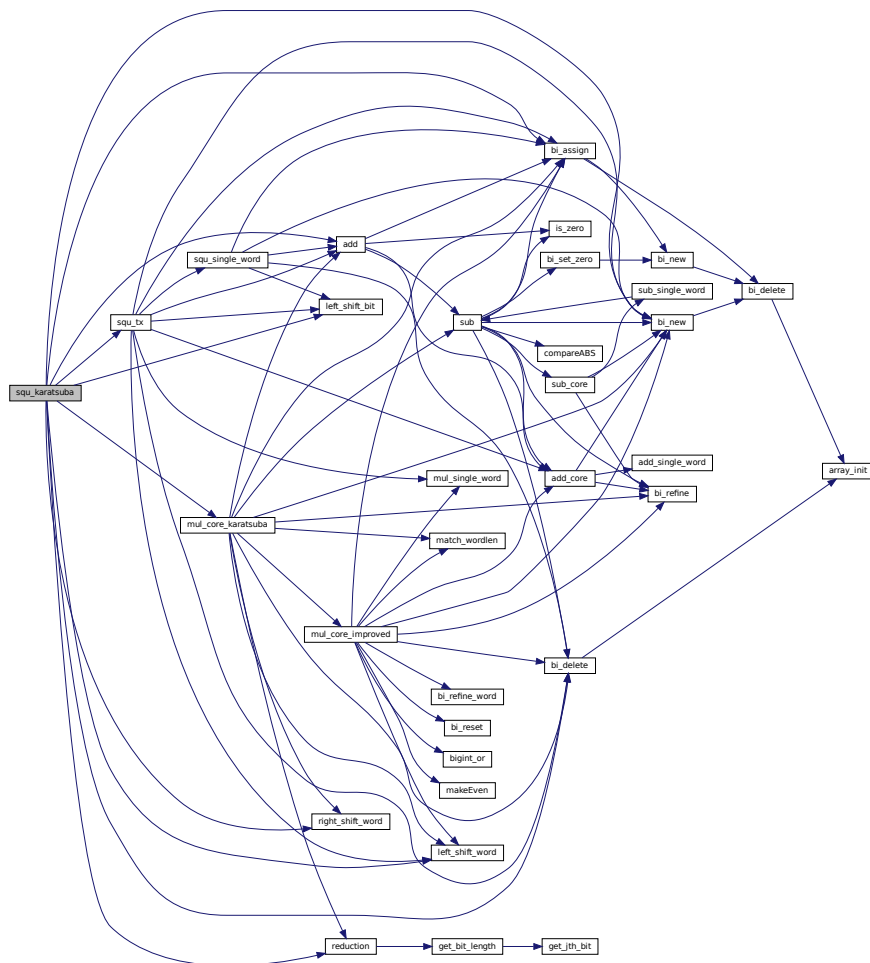
The result of squaring `x` is stored in the location pointed to by `z`. The input `x` remains unchanged after the operation.

Note

This function uses memory-efficient recursive computations and helper functions such as `left_shift_word`, `add`, and `mul_core_karatsuba` to manage intermediate results.

Definition at line 571 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.17 `squ_single_word()`

```
void squ_single_word (
    IN word A,
    OUT bigint ** result )
```

Squares a single word and stores the result in a bigint structure.

This function computes the square of a single word `A` and stores the result in a bigint structure pointed to by `result`.

Parameters

in	<code>A</code>	The single word to be squared.
out	<code>result</code>	A pointer to the bigint structure to store the squared result.

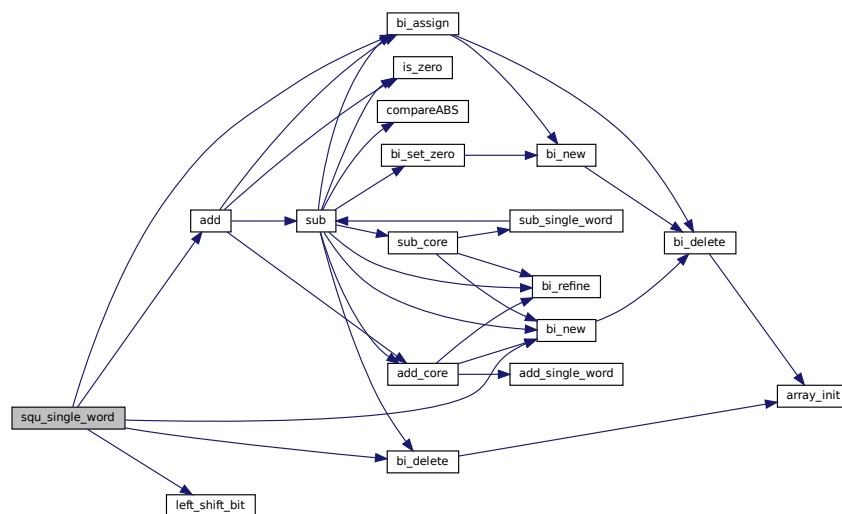
The squaring is done using a method that divides `A` into two halves: `A1` (upper bits) and `A0` (lower bits). The square is calculated as:

- $(C[0] = A0 * A0)$ (lower bits)
- $(C[1] = A1 * A1)$ (upper bits)
- The cross multiplication ($T = A0 * A1$), left-shifted by $w + 1$ bits

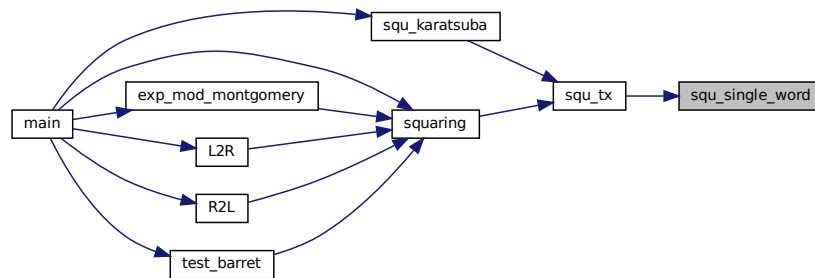
These intermediate results are combined to get the full square result.

Definition at line 477 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.18 squ_tx()

```

void squ_tx (
    IN bigint ** x,
    OUT bigint ** result )
  
```

Squares a bigint and stores the result in another bigint.

This function performs the squaring of a bigint x using the method of single-word squaring combined with cross products. The result is stored in the bigint structure pointed to by `result`.

Parameters

in	<code>x</code>	A pointer to the bigint to be squared.
out	<code>result</code>	A pointer to the bigint structure where the squared result will be stored.

The squaring method splits the process into:

- Squaring individual words in x and shifting accordingly.
- Calculating the cross products between words and summing them with left shifts.

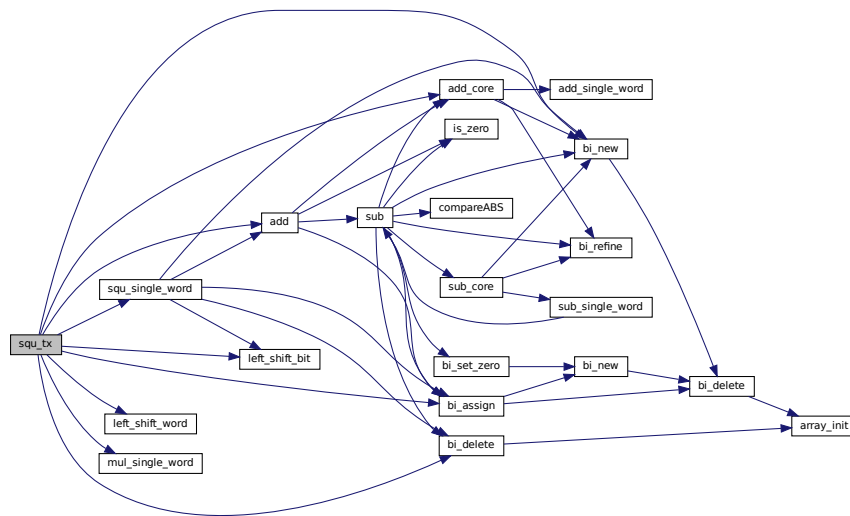
The function handles each word in x (up to t words) by:

- Calculating the square of each word j and adding the result to $C1$.
- Calculating the cross product for word pairs (j, i) where $i > j$ and accumulating in $C2$.

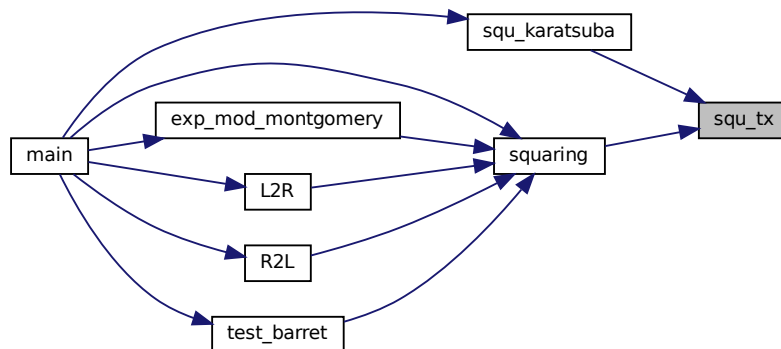
After processing, $C2$ is left-shifted by one bit to account for the doubling effect in cross terms.

Definition at line 510 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.19 squaring()

```
void squaring (
    IN bigint ** x,
    OUT bigint ** result )
```

Squares a bigint if it is non-zero and stores the result.

This function squares the bigint `x` and stores the result in `result`. If `x` is zero, it directly assigns zero to `result` and sets its sign to non-negative. Otherwise, it calls the `SQUC` function to compute the square of `x`.

4.6.1.20 sub()

```
void sub (
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Subtracts big integer y from big integer x and stores the result in z.

This function handles various cases depending on the signs and values of x and y, including when either operand is zero, when their absolute values are equal, and when their signs differ.

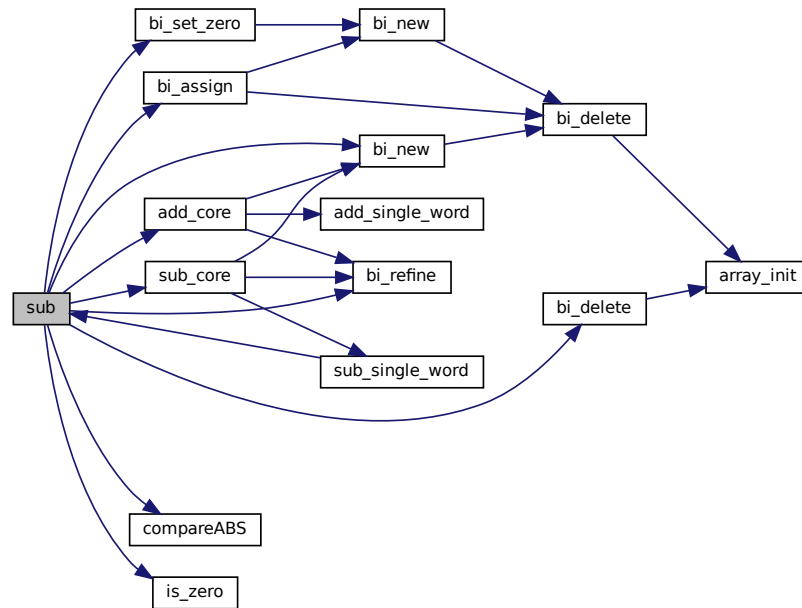
Parameters

in	x	Pointer to a pointer of the first bigint structure (minuend).
in	y	Pointer to a pointer of the second bigint structure (subtrahend).
out	z	Pointer to a pointer of the result bigint structure.

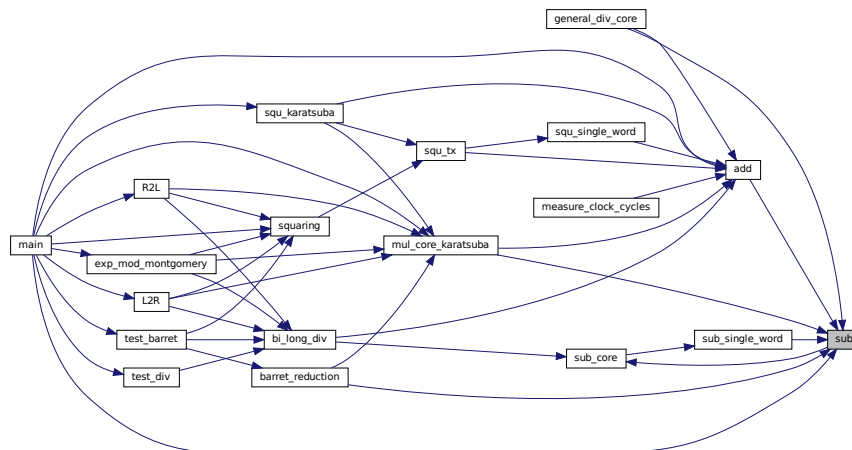
- If x is zero, the function sets z to -y.
- If y is zero, the function sets z to x.
- If the absolute values of x and y are equal:
 - If x and y have the same sign, z is set to zero.
 - If x and y have opposite signs, their absolute values are added, and z has the sign of x.
- If x and y are both positive, the function subtracts the smaller magnitude from the larger one and assigns the appropriate sign to z.
- If x and y are both negative, it subtracts the smaller magnitude from the larger and assigns a negative sign if x has a larger magnitude, or a positive sign if y is larger.
- If x and y have opposite signs, their absolute values are added and z has the sign of x.

Definition at line 166 of file arithmetic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.21 sub_core()

```

void sub_core (
    IN bigint ** x,

```

```

IN bigint ** y,
OUT bigint ** z )

```

Performs subtraction of two big integers.

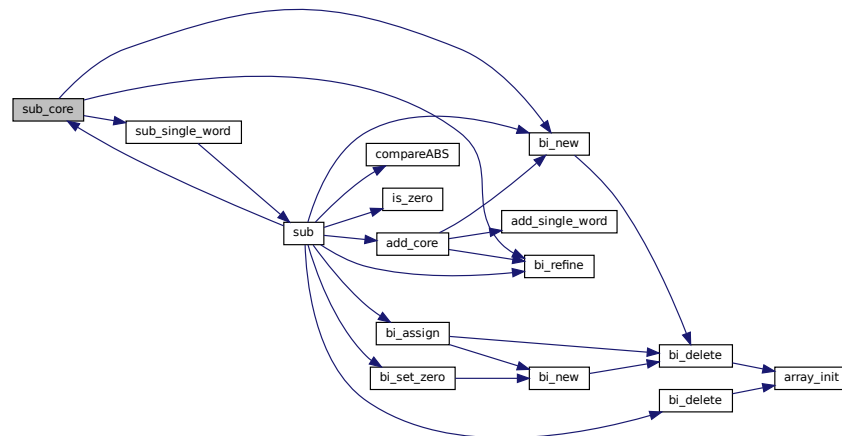
This function subtracts the big integer *y* from the big integer *x* and stores the result in *z*. It handles borrowing during subtraction and manages the necessary memory for the result.

Parameters

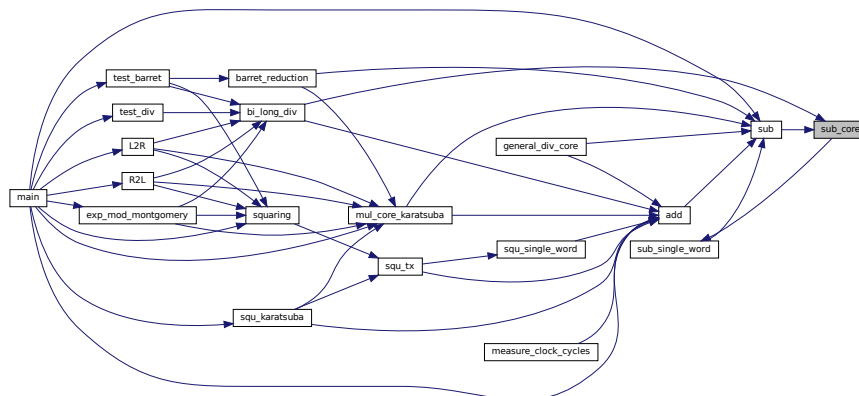
<i>x</i>	Pointer to the first bigint, the minuend.
<i>y</i>	Pointer to the second bigint, the subtrahend.
<i>z</i>	Pointer to store the result of the subtraction.

Definition at line 139 of file arithmic.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.1.22 sub_single_word()

```
void sub_single_word (
    IN word A,
    IN word B,
    IN word borrow_in,
    OUT word * borrow_out,
    OUT word * result )
```

Performs subtraction of B from A and manages borrowing.

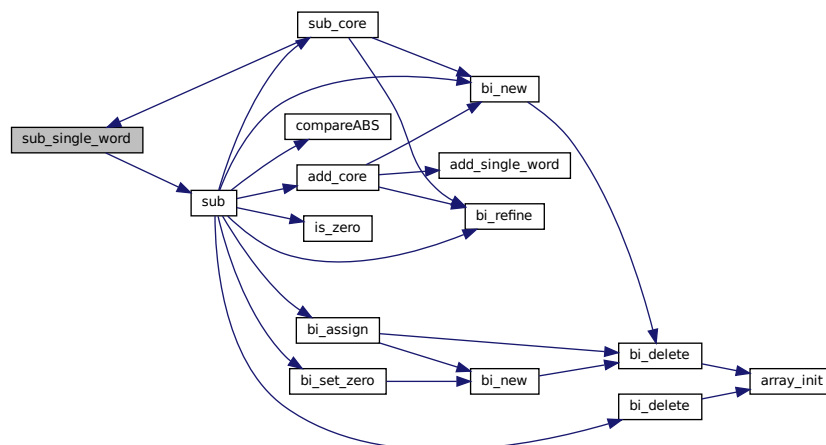
This function subtracts the second word (B) from the first word (A), taking into account any previous borrowing. It updates the result and the borrow status accordingly.

Parameters

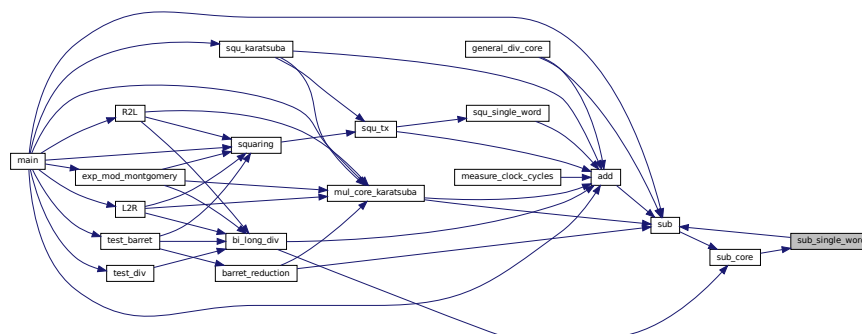
<i>A</i>	The first word, the value from which to subtract.
<i>B</i>	The second word, the value to be subtracted.
<i>borrow_in</i>	The previous borrow value.
<i>borrow_out</i>	Pointer to store the resulting borrow status.
<i>result</i>	Pointer to store the subtraction result.

Definition at line 123 of file arithmetic.c.

Here is the call graph for this function:



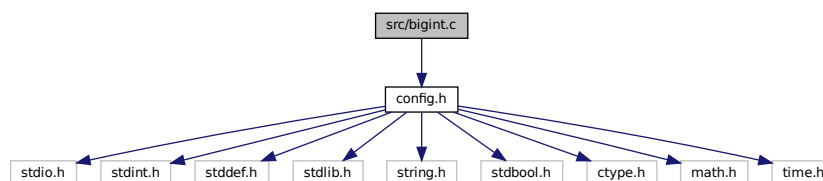
Here is the caller graph for this function:



4.7 src/bigint.c File Reference

```
#include "config.h"
```

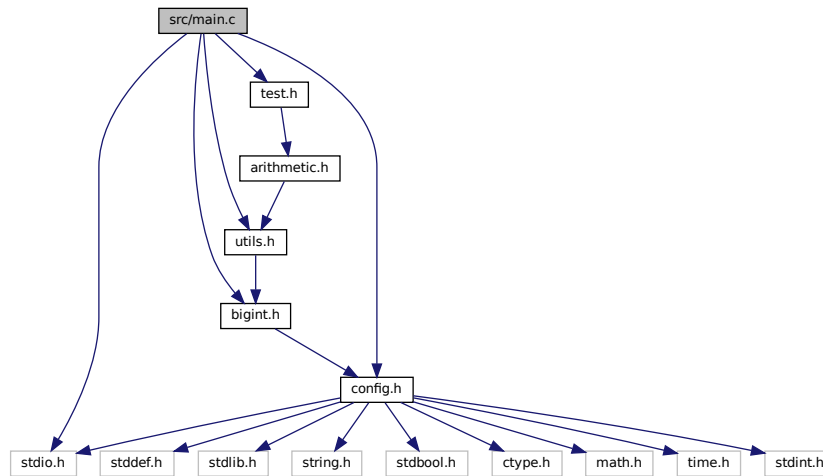
Include dependency graph for bigint.c:



4.8 src/main.c File Reference

```
#include <stdio.h>
#include "config.h"
#include "utils.h"
#include "test.h"
#include "bigint.h"
```

Include dependency graph for main.c:



Functions

- int `main` ()

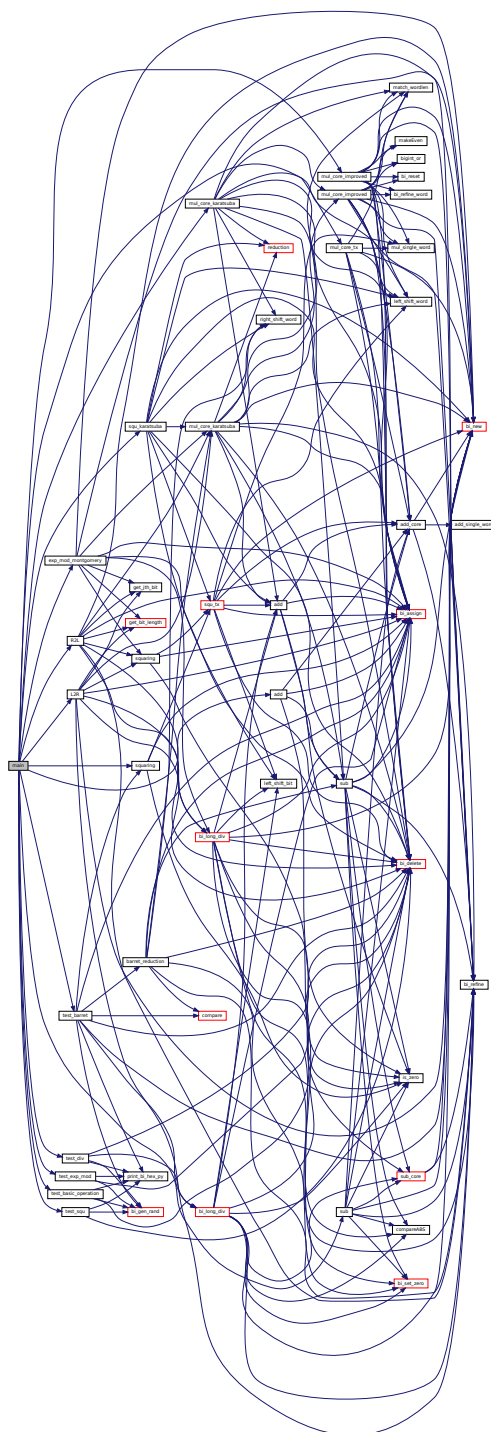
4.8.1 Function Documentation

4.8.1.1 `main()`

```
int main ( )
```

Definition at line 8 of file `main.c`.

Here is the call graph for this function:

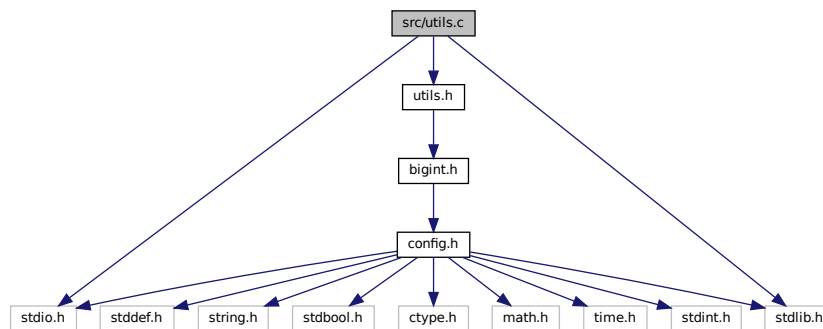


4.9 src/utils.c File Reference

```
#include "utils.h"
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for `utils.c`:



Functions

- void `bi_new` (IN bigint **x, IN int wordlen)
Allocates memory for a new bigint structure.
- void `array_init` (IN word *array, IN int length)
Initializes an array with a given length to a default value (0x00000000).
- void `bi_delete` (INOUT bigint **x)
Frees the memory occupied by a bigint structure.
- void `bi_show_hex` (IN bigint *x)
Displays the hexadecimal representation of a bigint.
- int `bi_set_by_array` (OUT bigint **x, IN int sign, IN word *a, IN int wordlen)
Initializes a bigint structure using an array of words and sets its sign.
- int `bi_set_by_string` (OUT bigint **x, IN int sign, IN char *str, IN int base)
Initializes a bigint structure from a character string representing a number.
- void `bi_refine` (INOUT bigint *x)
Refines a bigint by removing leading zeros and adjusting its length.
- void `bi_refine_word` (IN bigint *x, IN int num_words)
Refines the word length of a bigint by removing a specified number of words.
- void `bi_assign` (OUT bigint **dest, IN bigint *src)
Assigns the value of one bigint to another.
- void `bi_gen_rand` (OUT bigint **x, IN int sign, IN int wordlen)
Generates a random bigint with a specified sign and word length.
- void `array_rand` (OUT word *dst, IN int wordlen)
Fills an array with random bytes.
- void `bi_set_one` (OUT bigint **x)
Sets a bigint to the value of one.
- void `bi_set_zero` (OUT bigint **x)
Sets a bigint to the value of zero.
- int `compareABS` (IN bigint *x, IN bigint *y)
Compares the absolute values of two bigint numbers.
- int `compare` (IN bigint *x, IN bigint *y)
Compare two bigint numbers, taking into account their signs.
- int `get_bit_length` (IN bigint *x)
Calculate the bit length of a given bigint.

- bool `get_jth_bit` (IN `bigint *x`, IN int `j`)
Get the value of the j-th bit from a bigint.
- void `right_shift_bit` (INOUT `bigint *x`, IN int `shift`)
Perform a right shift operation on a bigint.
- void `left_shift_bit` (`bigint *pptrBint`, int `shift_amount`)
Perform a left shift operation on a bigint.
- void `reduction` (IN `bigint **x`, IN int `r`)
Perform a reduction operation on a bigint.
- void `left_shift_word` (INOUT `bigint **x`, IN int `shift_words`)
Left-shifts a bigint by a specified number of words.
- void `right_shift_word` (INOUT `bigint **x`, IN int `shift_words`)
Right-shifts a bigint by a specified number of words.
- int `is_zero` (IN `bigint *x`)
Checks if a bigint is zero.
- void `swap_bigint` (INOUT `bigint **x`, INOUT `bigint **y`)
Swaps the values of two bigint pointers.
- void `makeEven` (INOUT `bigint *x`)
Ensures that the word length of a bigint is even.
- void `match_wordlen` (INOUT `bigint *x`, INOUT `bigint *y`)
Matches the word lengths of two bigint objects by resizing them to the larger of their current lengths.
- void `bi_reset` (INOUT `bigint *x`)
Resets all words in a bigint to zero.
- `word_get_word` (IN `bigint *x`, IN int `m_th`)
Returns the value of the m-th word from a bigint.

4.9.1 Function Documentation

4.9.1.1 `array_init()`

```
void array_init (
    IN word * array,
    IN int length )
```

Initializes an array with a given length to a default value (0x00000000).

This function sets all elements of the input array to 0. It is typically used to clear or reset an array before performing other operations like big integer addition or multiplication.

Parameters

in	<code>array</code>	The pointer to the array to be initialized.
in	<code>length</code>	The number of elements (length) of the array to initialize.

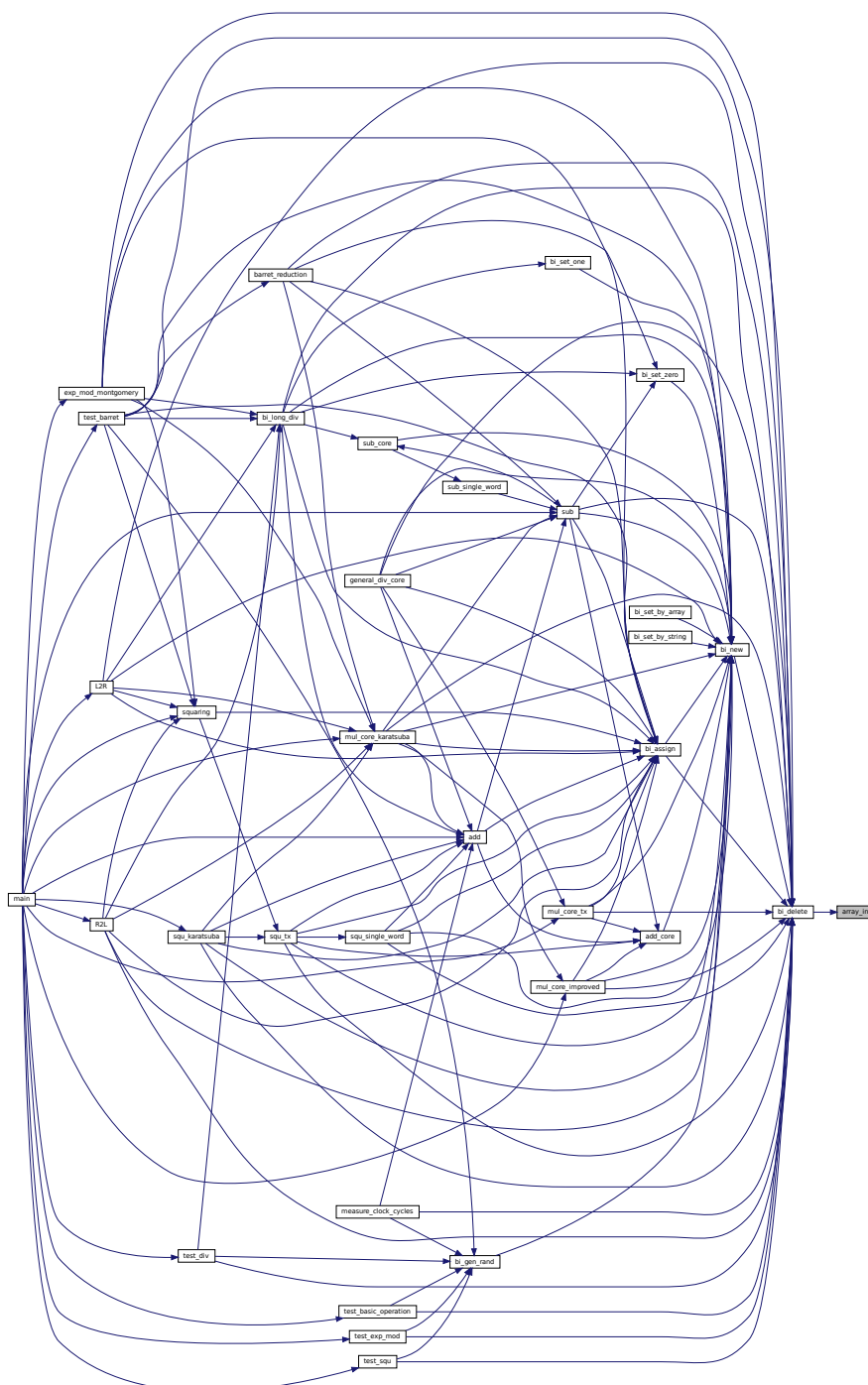
Returns

void This function does not return any value.

Warning

This function assumes the array is writable and properly allocated.

Here is the caller graph for this function:



4.9.1.2 array_rand()

```
void array_rand (
    OUT word * dst,
    IN int wordlen )
```

Fills an array with random bytes.

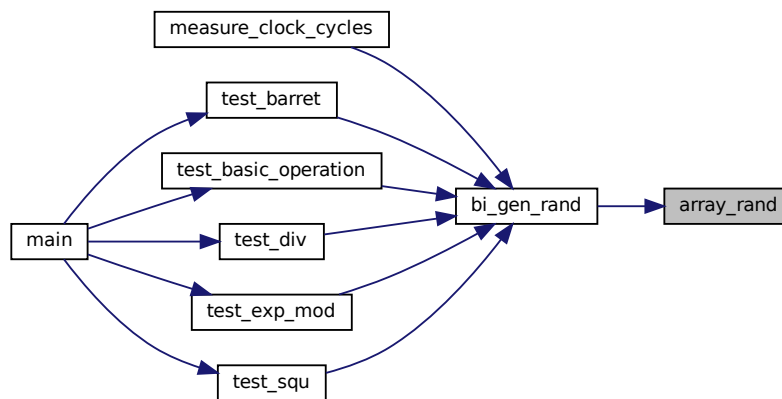
This function populates the specified array with random bytes using the rand function.

Parameters

out	<i>dst</i>	Pointer to the array to be filled with random values.
in	<i>wordlen</i>	The number of words to fill.

Definition at line 215 of file utlis.c.

Here is the caller graph for this function:



4.9.1.3 bi_assign()

```
void bi_assign (
    OUT bigint ** dest,
    IN bigint * src )
```

Assigns the value of one bigint to another.

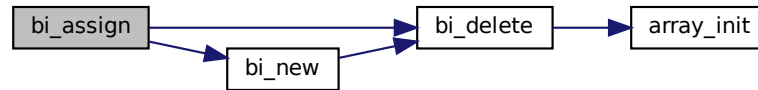
This function deletes the existing destination bigint, creates a new bigint of the same word length as the source, and copies the sign, word length, and value from the source bigint.

Parameters

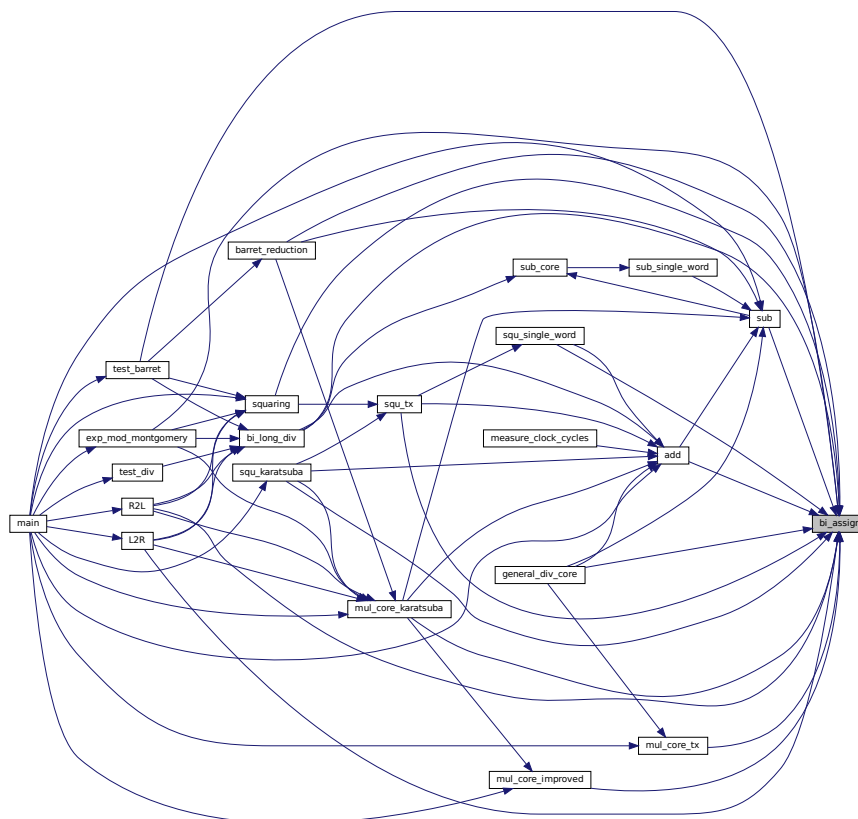
out	<i>dest</i>	Pointer to the destination bigint to be assigned to.
in	<i>src</i>	Pointer to the source bigint to copy from.

Definition at line 189 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.4 bi_delete()

```
void bi_delete (
    INOUT bigint ** x )
```

Frees the memory occupied by a bigint structure.

This function releases the memory allocated for the bigint structure and sets the pointer to NULL to avoid dangling references.

Parameters

in, out	x	Pointer to the bigint structure to be deleted.
---------	---	--

Returns

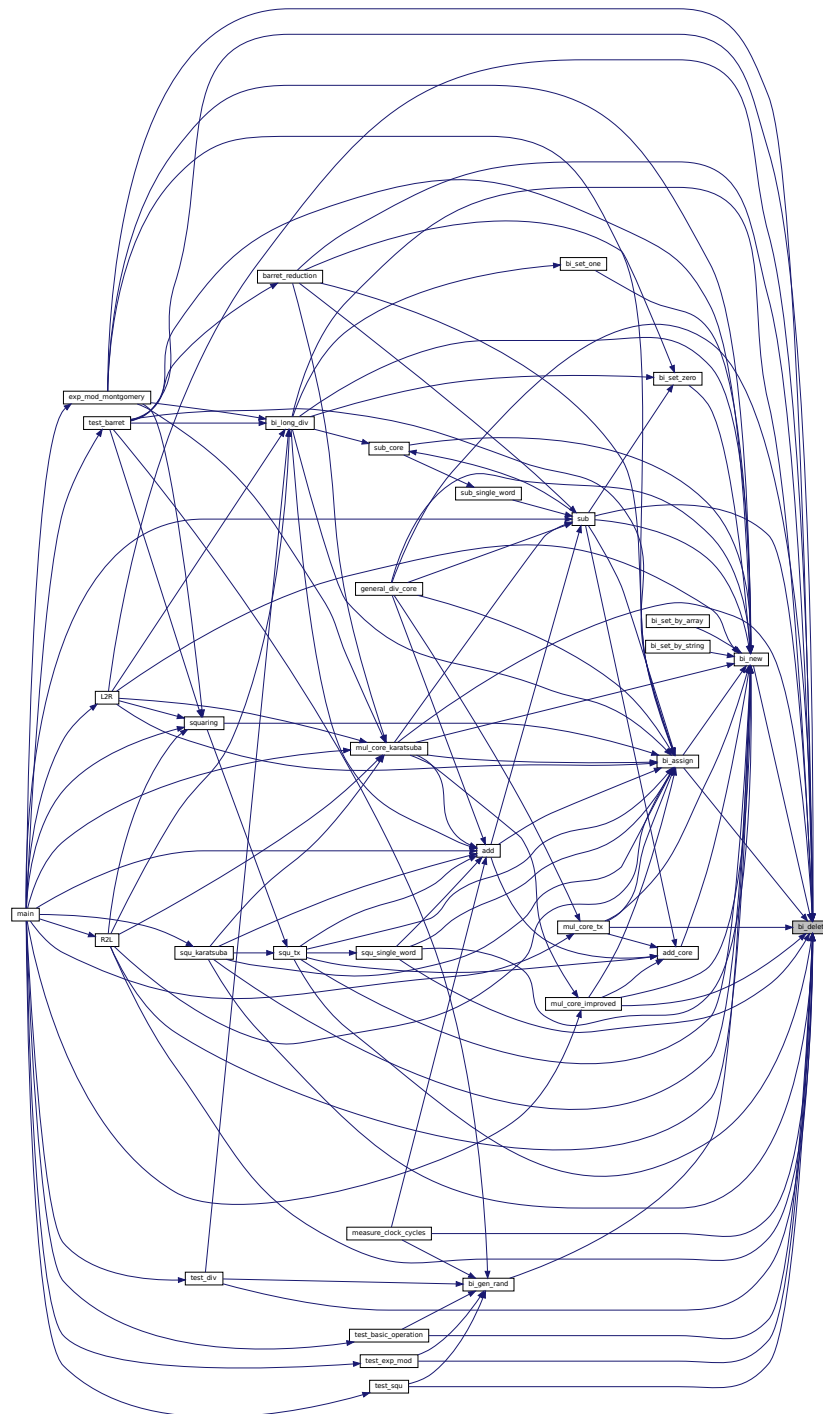
void

Definition at line 35 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.5 bi_gen_rand()

```
void bi_gen_rand (
    OUT bigint ** x,
```

```
IN int sign,
IN int wordlen )
```

Generates a random bigint with a specified sign and word length.

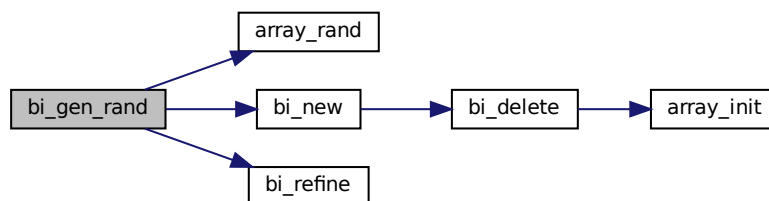
This function creates a new bigint, sets its sign, generates random values for its word array, and then refines it to remove any leading zeros.

Parameters

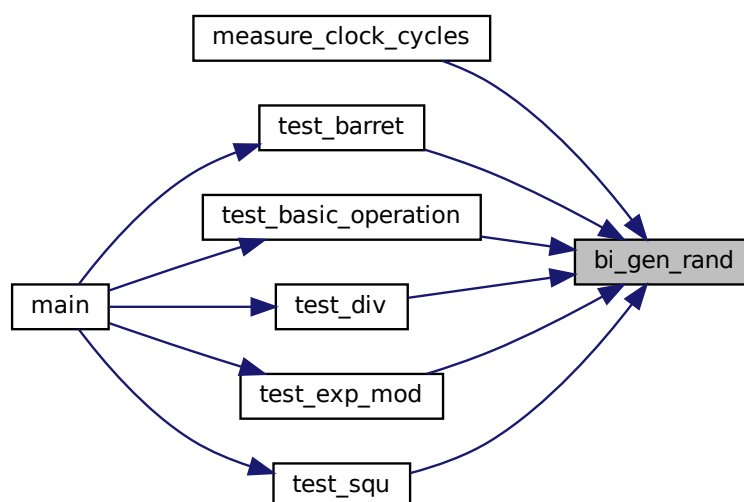
out	<i>x</i>	Pointer to the bigint to be generated.
in	<i>sign</i>	The sign of the bigint (NEGATIVE or NON_NEGATIVE).
in	<i>wordlen</i>	The desired length of the word array.

Definition at line 206 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.6 bi_new()

```
void bi_new (
    IN bigint ** x,
    IN int wordlen )
```

Allocates memory for a new bigint structure.

This function creates a new bigint structure and initializes it with the specified word length. The pointer to the newly created bigint is stored in the provided pointer to a pointer (x).

Parameters

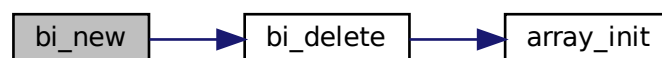
out	<i>x</i>	Pointer to the bigint structure to be initialized.
in	<i>wordlen</i>	The number of words to allocate for the bigint.

Returns

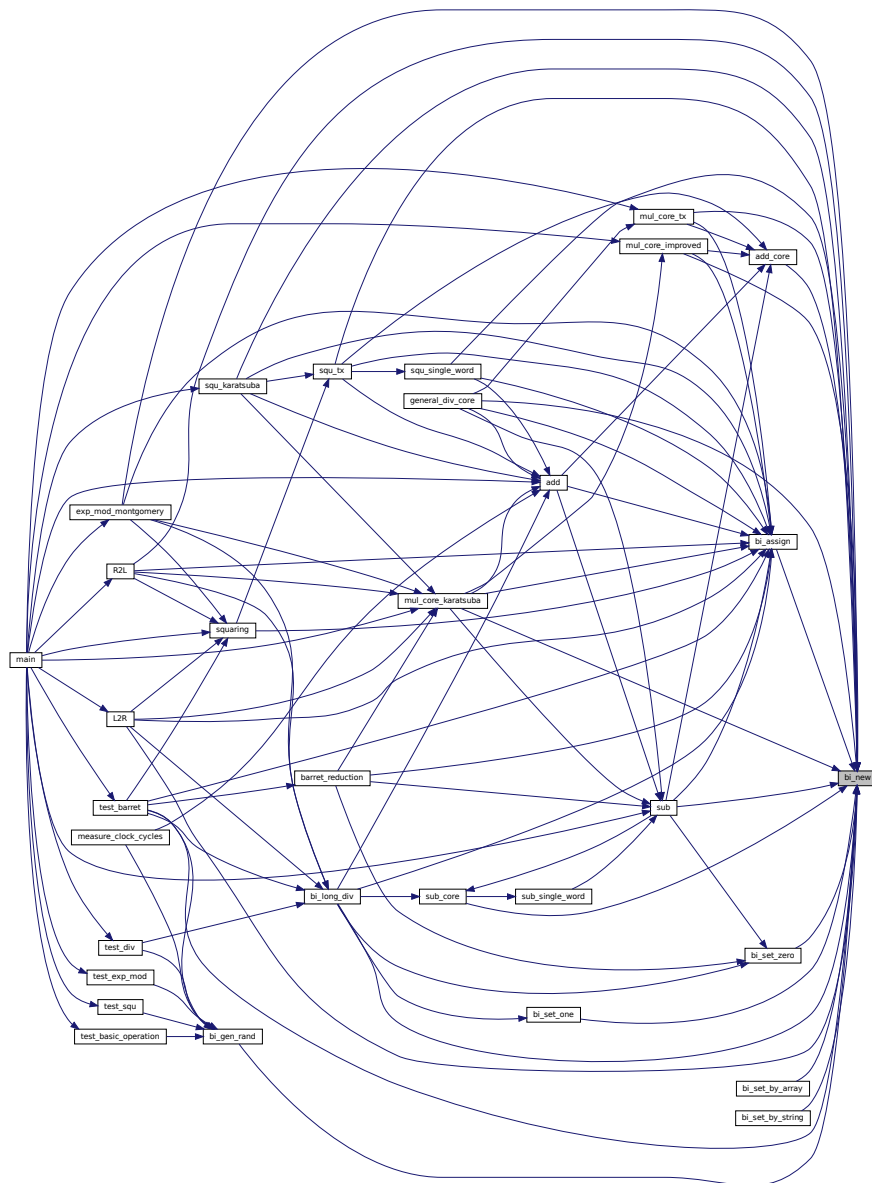
void

Definition at line 5 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.7 bi_refine()

```
void bi_refine (
    INOUT bigint * x )
```

Refines a bigint by removing leading zeros and adjusting its length.

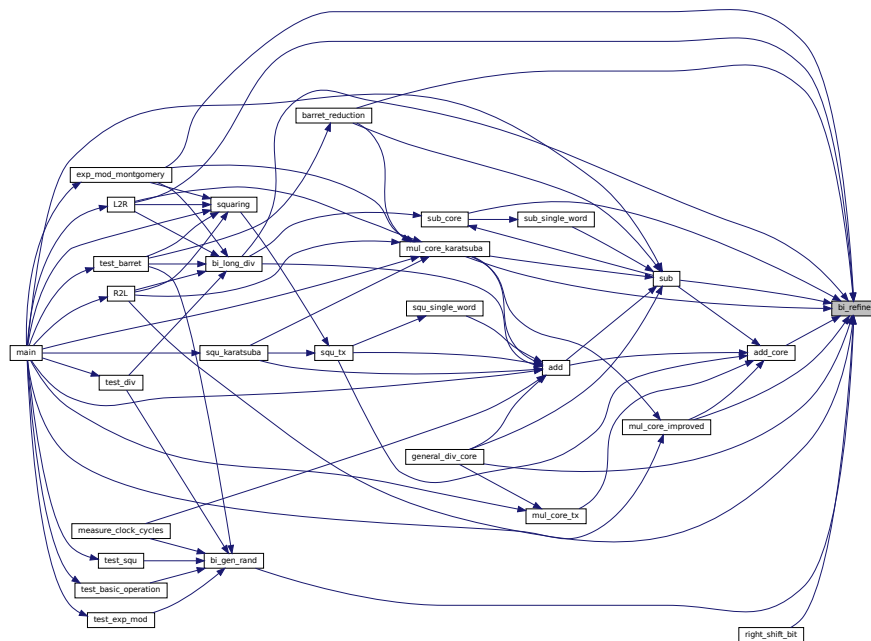
This function modifies the given bigint by removing any leading zeros, updating its word length accordingly, and ensuring that the sign is set correctly if the bigint becomes zero.

Parameters

in, out	<i>x</i>	Pointer to the bigint to be refined (input/output).
---------	----------	---

Definition at line 139 of file utils.c.

Here is the caller graph for this function:



4.9.1.8 bi_refine_word()

```
void bi_refine_word (
    IN bigint * x,
    IN int num_words )
```

Refines the word length of a `bigint` by removing a specified number of words.

This function adjusts the word length of a `bigint` object `x` by reducing it by `num_words`. It reallocates memory if necessary and ensures the integrity of the `bigint` structure. If the result represents zero (single word with value 0), the sign is reset to `NON_NEGATIVE`.

Parameters

in	<i>x</i>	A pointer to the <code>bigint</code> object to be refined.
in	<i>num_words</i>	The number of words to remove from the end of the <code>bigint</code> .

Precondition

`x` must be a valid pointer to an initialized `bigint` object. `num_words` should not exceed the current word length of `x`.

Postcondition

The `bigint`'s word length is reduced by `num_words`, and its memory is resized accordingly. If the resulting `bigint` represents zero, its sign is reset to `NON_NEGATIVE`.

Note

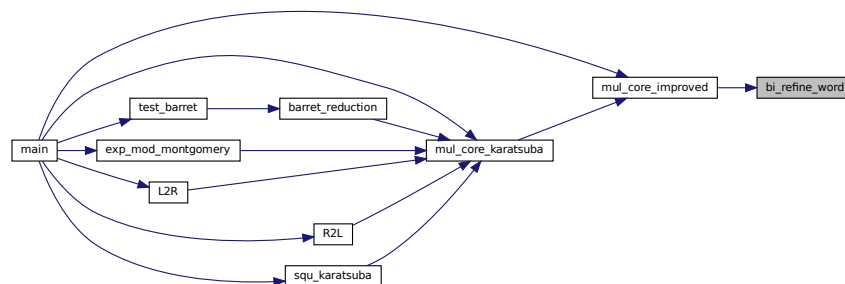
- If `x` is `NULL`, the function prints an error message (`INVALID_DATA`) and returns without performing any operation.
- The function assumes that the `bigint` structure has attributes `a` (an array of words), `wordlen` (length of the array), and `sign` (the sign of the integer).
- Memory reallocation is handled using `realloc`. Ensure proper error handling for memory operations if extended.

Warning

- If `num_words` is greater than or equal to `x->wordlen`, the behavior may be undefined.
- Always ensure that the `bigint` object `x` is valid before calling this function.

Definition at line 167 of file `utils.c`.

Here is the caller graph for this function:

**4.9.1.9 bi_reset()**

```
void bi_reset (
    INOUT bigint * x )
```

Resets all words in a `bigint` to zero.

This function iterates through all words in the given `bigint` object `x` and sets each word to zero. It does not alter the word length of the `bigint`, only the values of the words.

Parameters

in, out	<i>x</i>	A pointer to the <code>bigint</code> object to be reset.
---------	----------	--

Precondition

x must be a valid pointer to an initialized `bigint` object.

Postcondition

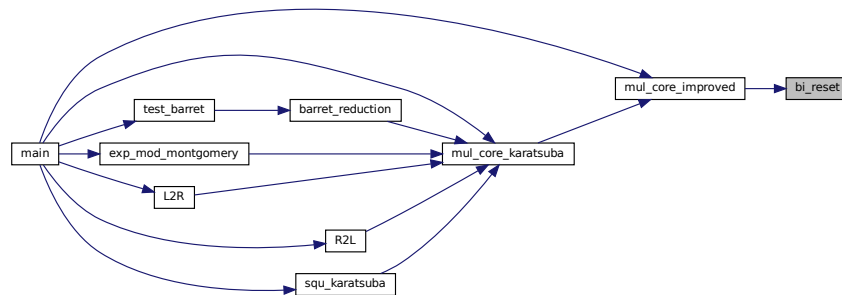
All words in the `bigint` pointed to by *x* are set to zero, but the word length remains unchanged.

Note

This function assumes that the `bigint` structure has an attribute `a` (an array of words) and `wordlen` (the number of words).

Definition at line 581 of file `utils.c`.

Here is the caller graph for this function:

**4.9.1.10 bi_set_by_array()**

```

int bi_set_by_array (
    OUT bigint ** x,
    IN int sign,
    IN word * a,
    IN int wordlen )

```

Initializes a `bigint` structure using an array of words and sets its sign.

Parameters

out	<i>x</i>	Pointer to the <code>bigint</code> structure to be initialized.
in	<i>sign</i>	The sign of the <code>bigint</code> (NEGATIVE or NON_NEGATIVE).
in	<i>a</i>	Array of words to set the value of the <code>bigint</code> .
in	<i>wordlen</i>	Length of the word array.

Returns

int Returns 0 on success, non-zero on failure.

Definition at line 80 of file utls.c.

Here is the call graph for this function:

**4.9.1.11 bi_set_by_string()**

```

int bi_set_by_string (
    OUT bigint ** x,
    IN int sign,
    IN char * str,
    IN int base )
  
```

Initializes a bigint structure from a character string representing a number.

Parameters

out	<i>x</i>	Pointer to the bigint structure to be initialized (output).
in	<i>sign</i>	The sign of the bigint (NEGATIVE or NON_NEGATIVE) (input).
in	<i>str</i>	The string representation of the number to be converted (input).
in	<i>base</i>	The base of the number system (e.g., 10 for decimal) (input).

Returns

int Returns 0 on success, non-zero on failure.

Definition at line 98 of file utls.c.

Here is the call graph for this function:



4.9.1.12 bi_set_one()

```
void bi_set_one (
    OUT bigint ** x )
```

Sets a bigint to the value of one.

This function initializes a new bigint structure and sets its value to one. The sign of the bigint is set to NON_NEGATIVE.

Parameters

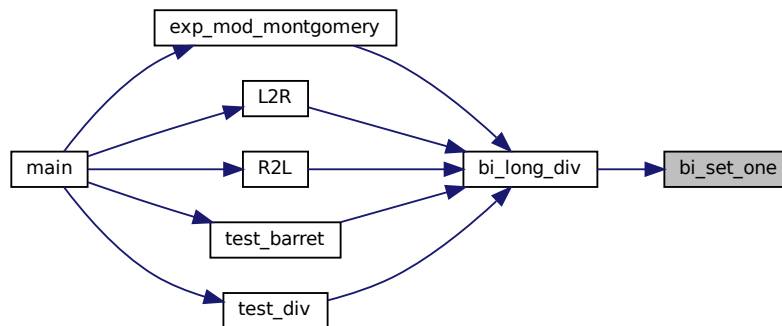
out	x	Pointer to the bigint structure to be initialized and set to one.
-----	---	---

Definition at line 227 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.13 bi_set_zero()

```
void bi_set_zero (
    OUT bigint ** x )
```

Sets a bigint to the value of zero.

This function initializes a new bigint structure and sets its value to zero. The sign of the bigint is set to NON_NEGATIVE.

Parameters

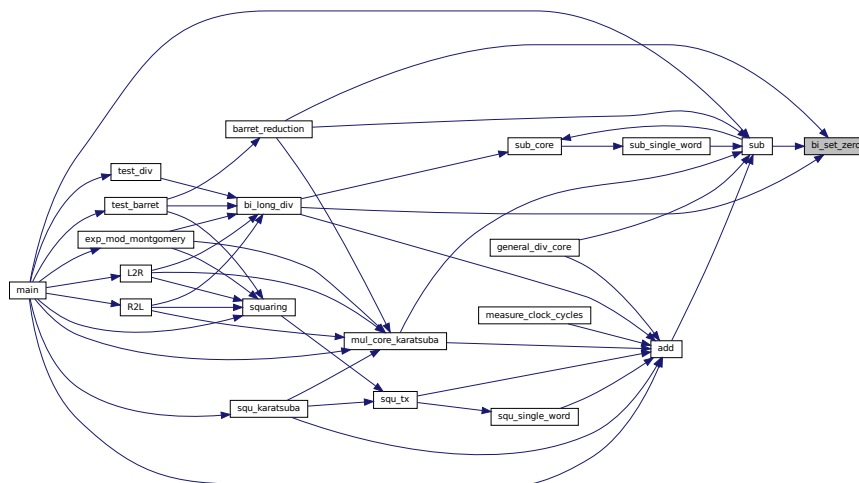
out	x	Pointer to the bigint structure to be initialized and set to zero.
-----	---	--

Definition at line 234 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.14 bi_show_hex()

```
void bi_show_hex (
    IN bigint * x )
```

Displays the hexadecimal representation of a bigint.

This function prints the value of the bigint in hexadecimal format. It also handles the sign of the bigint and checks for invalid inputs.

Parameters

in	x	Pointer to the bigint to be displayed. If NULL, an error message is printed.
----	---	--

Definition at line 49 of file utils.c.

4.9.1.15 compare()

```
int compare (
    IN bigint * x,
    IN bigint * y )
```

Compare two bigint numbers, taking into account their signs.

This function compares two bigint numbers, considering both their magnitudes and signs.

Parameters

in	<i>x</i>	Pointer to the first bigint number (input).
in	<i>y</i>	Pointer to the second bigint number (input).

Returns

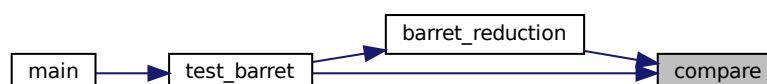
1 if $x > y$, -1 if $x < y$, or 0 if $x == y$.

Definition at line 268 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.16 compareABS()

```
int compareABS (
    IN bigint * x,
    IN bigint * y )
```

Compares the absolute values of two bigint numbers.

This function compares the absolute values of two `bigint` numbers `x` and `y`. The sign of `x` and `y` is ignored, and only the magnitudes are compared.

Parameters

in	<code>x</code>	Pointer to the first bigint number.
in	<code>y</code>	Pointer to the second bigint number.

Returns

int

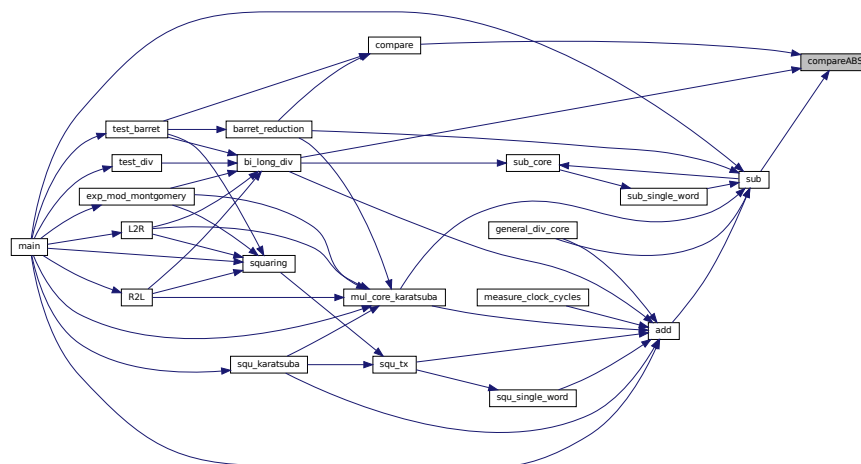
- Returns 1 if $|x| > |y|$
- Returns 0 if $|x| < |y|$
- Returns -1 if $|x| == |y|$

Note

Assumes that `x` and `y` are valid bigint pointers with non-null `a` arrays.

Definition at line 241 of file `utlis.c`.

Here is the caller graph for this function:



4.9.1.17 get_bit_length()

```
int get_bit_length (
    IN bigint * x )
```

Calculate the bit length of a given bigint.

This function calculates the bit length of the absolute value of a bigint. It accounts for the number of leading zeros in the last word and adds an extra bit for the sign if the bigint is negative.

Parameters

in	x	A pointer to the bigint structure (input).
----	---	--

Returns

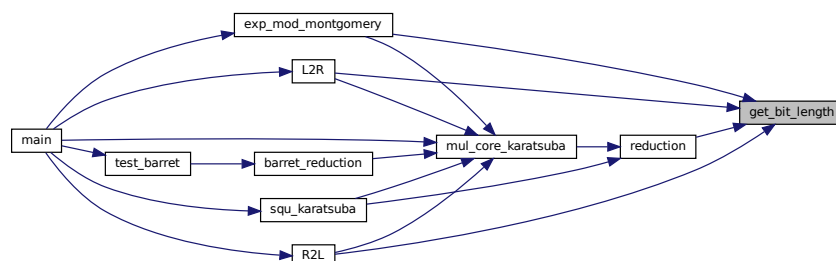
The bit length of the bigint. If there are no words, returns 0. If the bigint is negative, an extra bit for the sign is included.

Definition at line 289 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.18 get_jth_bit()

```
bool get_jth_bit (
    IN bigint * x,
    IN int j )
```

Get the value of the j -th bit from a `bigint`.

This function retrieves the value of the specified bit (j) from the given `bigint`. The indexing is 0-based, where the least significant bit is at index 0.

Parameters

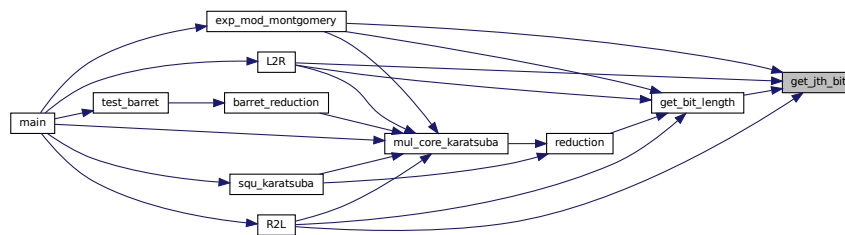
in	x	Pointer to the <code>bigint</code> structure from which the bit will be retrieved (input).
in	j	The index of the bit to retrieve (0-based) (input).

Returns

Returns 1 if the bit is set, 0 if the bit is not set, or -1 if the input is invalid (e.g., NULL pointer, out of bounds).

Definition at line 301 of file `utils.c`.

Here is the caller graph for this function:



4.9.1.19 get_word()

```
word get_word (
    IN bigint * x,
    IN int m_th )
```

Returns the value of the m -th word from a `bigint`.

This function retrieves the word at the specified index `m_th` from the `bigint`'s internal array of words. If the index is out of bounds, the function returns zero.

Parameters

in	x	A pointer to the <code>bigint</code> object from which the word will be retrieved.
in	m_{th}	The index of the word to retrieve.

Returns

The value of the m -th word, or 0 if the index is out of bounds.

Precondition

x must be a valid pointer to an initialized `bigint` object.

Postcondition

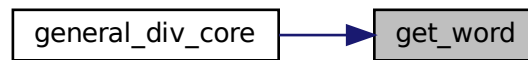
The function returns the value of the specified word in the `bigint`.

Note

This function assumes that the `bigint` structure has an attribute `a` (an array of words) and `wordlen` (the number of words).

Definition at line 587 of file `utils.c`.

Here is the caller graph for this function:

**4.9.1.20 is_zero()**

```
int is_zero (
    IN bigint * x )
```

Checks if a `bigint` is zero.

This function checks if all bits in the `bigint` x are zero. It iterates through each word and each bit in $x \rightarrow a$ to verify that no bit is set to 1.

Parameters

in	x	Pointer to the <code>bigint</code> structure to be checked.
----	-----	---

Returns

int Returns 1 if any bit in x is set to 1, indicating x is not zero. Returns 0 if all bits in x are zero.

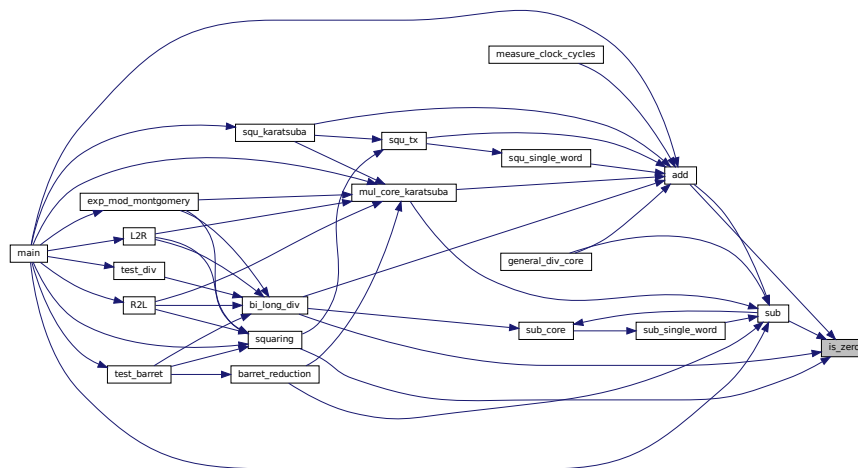
- The function uses bitwise operations to examine each bit of $x \rightarrow a$.
- If any bit in $x \rightarrow a$ is set to 1, the function returns immediately with a result of 1.
- If all bits are zero, the function returns 0, indicating that x represents zero.

Note

- The function assumes x is a valid pointer to a bigint structure.
- The use of an intermediate variable `temp` helps avoid modifying $x \rightarrow a[i]$ directly.

Definition at line 495 of file `utlis.c`.

Here is the caller graph for this function:



4.9.1.21 left_shift_bit()

```
void left_shift_bit (
    bigint * pptrBint,
    int shift_amount )
```

Perform a left shift operation on a bigint.

This function shifts the bits of the given bigint to the left by a specified number of positions. It reallocates memory for the underlying array to accommodate the shift and updates the word length accordingly.

Parameters

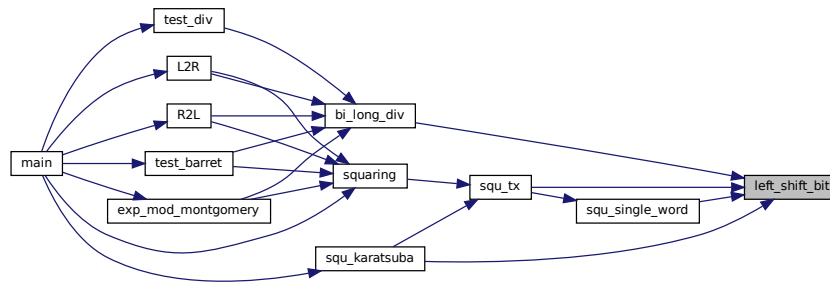
in, out	<i>x</i>	Pointer to the bigint to be shifted.
in	<i>shift</i>	The number of bits to shift to the left.

Note

The function handles memory allocation and will exit the program with an error message if memory allocation fails. It also removes any leading zero words after the shift operation.

Definition at line 340 of file `utils.c`.

Here is the caller graph for this function:

**4.9.1.22 left_shift_word()**

```

void left_shift_word (
    INOUT bigint ** x,
    IN int shift_words )

```

Left-shifts a bigint by a specified number of words.

This function shifts the bigint `x` to the left by the specified number of words (`shift_words`). It reallocates memory for the bigint array `(*x) -> a` to accommodate the shift and initializes any newly allocated lower-order words to zero.

Parameters

in, out	<code>x</code>	Double pointer to the bigint structure to be shifted.
in	<code>shift_words</code>	The number of words to shift <code>x</code> to the left.

Note

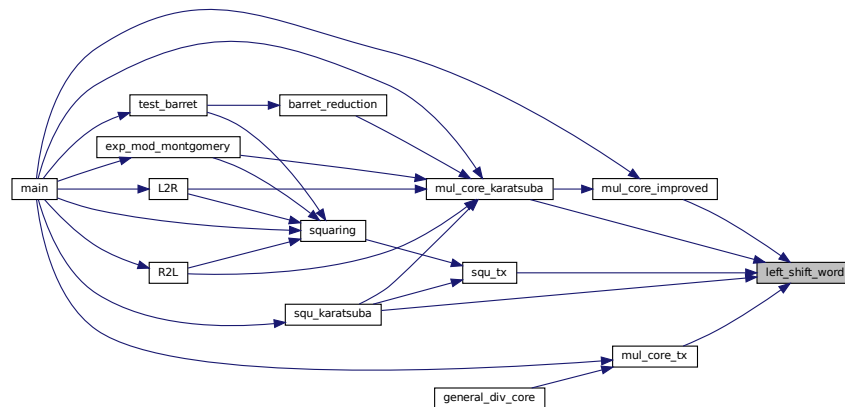
- If `shift_words` is negative, the function outputs an error message and calls `exit(1)`.
- The function reallocates memory for `(*x) -> a` to ensure there is enough space for the shift.
- The function increases the length of `(*x) -> a` by `shift_words`.
- Existing words in `(*x) -> a` are shifted to the left by `shift_words`.
- Newly added words are set to `ZERO`.
- The word length of `(*x)` is updated to the new length.

Warning

- On memory allocation failure, `exit(1)` is called.
- The function assumes `x` points to a valid `bigint` structure.

Definition at line 431 of file `utlis.c`.

Here is the caller graph for this function:

**4.9.1.23 makeEven()**

```
void makeEven (
    INOUT bigint * x )
```

Ensures that the word length of a `bigint` is even.

This function checks if the word length of the given `bigint` object `x` is odd. If it is, the function increases the word length by one, reallocates memory to accommodate the new size, and initializes the newly added word to zero.

Parameters

in, out	<code>x</code>	A pointer to the <code>bigint</code> object to be adjusted. The structure is modified to ensure an even word length.
---------	----------------	--

Precondition

`x` must be a valid pointer to an initialized `bigint` object.

Postcondition

If the original word length of `x` is odd:

- The word length is increased by one.

- Memory is reallocated to match the new size.
- The new higher-order word is initialized to zero.

Note

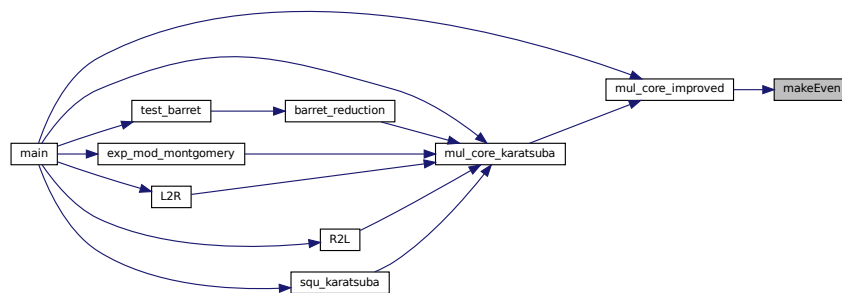
- If the word length of `x` is already even, no changes are made.
- The function assumes that the `bigint` structure has attributes `a` (an array of words) and `wordlen` (length of the array).
- Memory reallocation is handled using `realloc`. Ensure error handling for memory failures.

Warning

- If memory allocation fails during `realloc`, the program prints an error message (`MEM_ALLOCATION_FAIL`) and exits.
- This function modifies the memory directly, so ensure there are no conflicting references to the same `bigint`.

Definition at line 523 of file `utils.c`.

Here is the caller graph for this function:



4.9.1.24 match_wordlen()

```

void match_wordlen (
    INOUT bigint * x,
    INOUT bigint * y )

```

Matches the word lengths of two `bigint` objects by resizing them to the larger of their current lengths.

This function adjusts the word lengths of two `bigint` objects `x` and `y` so that both have the same word length. If either `x` or `y` has a smaller word length than the maximum of the two, it reallocates memory and initializes the newly added words to zero.

Parameters

in, out	<i>x</i>	A pointer to the first <code>bigint</code> object. Its word length and memory may be modified to match the other <code>bigint</code> .
in, out	<i>y</i>	A pointer to the second <code>bigint</code> object. Its word length and memory may be modified to match the other <code>bigint</code> .

Precondition

Both *x* and *y* must point to valid, initialized `bigint` objects.

Postcondition

- The word lengths of *x* and *y* are equal and match the larger of their original word lengths.
- Memory is reallocated for either *x* or *y* (or both) if necessary.
- Any newly added words are initialized to zero.

Note

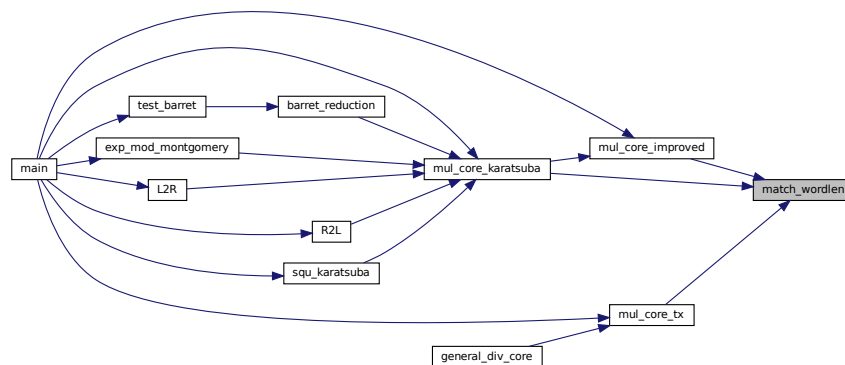
- The function assumes that the `bigint` structure has attributes `a` (an array of words) and `wordlen` (length of the array).
- Memory reallocation is performed using `realloc`. Proper error handling is in place to terminate the program if allocation fails.

Warning

- If memory allocation fails during `realloc`, the program prints an error message (`MEM_ALLOCATION_FAIL`) and exits.
- Ensure no conflicting references to the same `bigint` objects are active when calling this function.

Definition at line 541 of file `utils.c`.

Here is the caller graph for this function:



4.9.1.25 reduction()

```
void reduction (
    IN bigint ** x,
    IN int r )
```

Perform a reduction operation on a bigint.

This function reduces the given bigint A by shifting its bits to the right by a specified number of positions r. The result is stored in the provided bigint result. If r is greater than or equal to the total bit length of A, the result will be zero. The sign of the result will match the sign of A.

Parameters

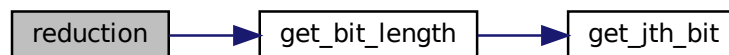
in	<i>x</i>	Pointer to the bigint to be reduced.
in	<i>r</i>	The number of bits to shift to the right.
out	<i>result</i>	Pointer to the bigint where the result will be stored.

Note

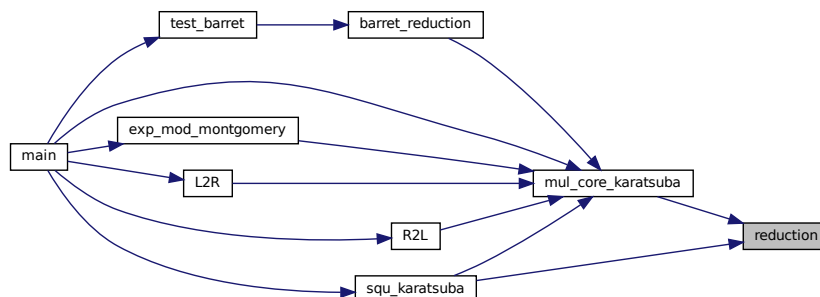
Memory for the result bigint is allocated inside this function. The caller is responsible for freeing the memory used by result.

Definition at line 380 of file utils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.1.26 right_shift_bit()

```
void right_shift_bit (
    INOUT bigint * x,
    IN int shift )
```

Perform a right shift operation on a bigint.

This function shifts the bits of the given bigint to the right by a specified number of positions. It reallocates memory for the underlying array to accommodate the shift and updates the word length accordingly. If the shift results in fewer than one word, the function ensures that at least one word remains.

Parameters

in, out	<i>x</i>	Pointer to the bigint to be shifted (input/output).
in	<i>shift</i>	The number of bits to shift to the right (input).

Note

The function handles memory allocation and will exit the program with an error message if memory allocation fails. It also removes any leading zero words after the shift operation.

Definition at line 309 of file utlis.c.

Here is the call graph for this function:



4.9.1.27 right_shift_word()

```
void right_shift_word (
    INOUT bigint ** x,
    IN int shift_words )
```

Right-shifts a bigint by a specified number of words.

This function shifts the bigint *x* to the right by the specified number of words (*shift_words*). It reallocates memory for the bigint array *x*→*a* to accommodate the shift and initializes any newly allocated words to zero.

Parameters

in, out	<i>x</i>	Pointer to the bigint structure to be shifted.
in	<i>shift_words</i>	The number of words to shift <i>x</i> to the right.

Note

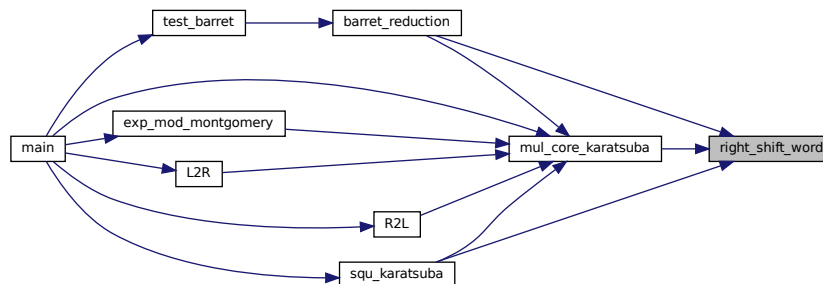
- If memory reallocation fails, the function outputs `MEM_ALLOCATION_FAIL` and calls `exit(1)`.
- The function assumes that `x` and `x->a` are valid pointers with allocated memory.
- The function increases the length of `x->a` by `shift_words`.
- After reallocation, newly added words in `x->a` are set to `ZERO`.
- The sign of `x` remains unchanged.

Warning

- `x->a` is reallocated, and its size is modified to include the shifted words.
- On memory allocation failure, `exit(1)` is called.

Definition at line 462 of file `utils.c`.

Here is the caller graph for this function:

**4.9.1.28 swap_bigint()**

```

void swap_bigint (
    INOUT bigint ** x,
    INOUT bigint ** y )

```

Swaps the values of two `bigint` pointers.

Exchanges the pointers of two `bigint` objects `x` and `y`. This effectively swaps the two `bigint` structures without copying their contents.

Parameters

in, out	<code>x</code>	A double pointer to the first <code>bigint</code> object.
in, out	<code>y</code>	A double pointer to the second <code>bigint</code> object.

Precondition

`x` and `y` must be valid, initialized double pointers to `bigint` objects.

Postcondition

The values of `*x` and `*y` are swapped. The `bigint` objects they point to are unchanged, but the pointers themselves are exchanged.

Note

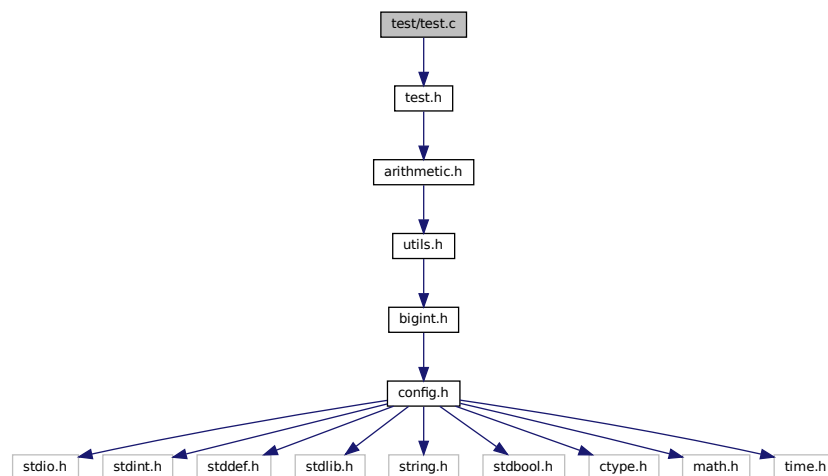
This function operates only on the pointers and does not modify the actual contents of the `bigint` objects.

Definition at line 515 of file `utils.c`.

4.10 test/test.c File Reference

```
#include "test.h"
```

Include dependency graph for `test.c`:

**Functions**

- void `print_bi_hex_py` (IN const `bigint` `*x`)
Prints a `bigint` in hexadecimal format (Python-style).
- void `test_squ` (void(*operation)(IN `bigint` **, OUT `bigint` **), const char *operate)
Tests the squaring operation on randomly generated `bigint` values.
- void `test_basic_operation` (void(*operation)(IN `bigint` **, IN `bigint` **, OUT `bigint` **), const char *operator, const char *operate)
Tests basic arithmetic operations on randomly generated `bigint` values.
- void `test_div` (const char *operate)
Tests division by verifying the quotient and remainder.

- void `test_exp_mod` (void(*operation)(IN bigint **, IN bigint **, OUT bigint **, IN bigint **), const char *operate)
Tests modular exponentiation with randomly generated values.
- void `test_barret` (const char *operate)
Tests Barrett reduction for modular arithmetic.
- void `measure_cycles` (void(*func)(IN bigint **, IN bigint **, OUT bigint **), IN bigint **x, IN bigint **y, OUT bigint **Z)
Measures the average number of CPU cycles for a specific operation.
- void `measure_clock_cycles` ()
Measures and prints the clock cycles for various operations.

4.10.1 Function Documentation

4.10.1.1 `measure_clock_cycles()`

```
void measure_clock_cycles ( )
```

Measures and prints the clock cycles for various operations.

This function executes different arithmetic operations multiple times and measures the average CPU clock cycles required for each operation.

Precondition

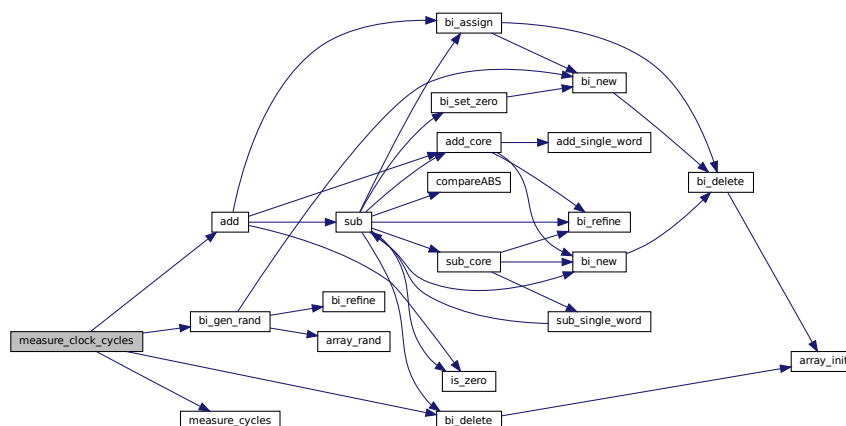
All operations to be measured must have properly defined function implementations.

Postcondition

The average clock cycles for each operation are printed to the standard output.

Definition at line 247 of file test.c.

Here is the call graph for this function:



4.10.1.2 `measure_cycles()`

```
void measure_cycles (
    void(*) (IN bigint **, IN bigint **, OUT bigint **) func,
    IN bigint ** x,
    IN bigint ** y,
    OUT bigint ** z )
```

Measures the average number of CPU cycles for a specific operation.

This function times the execution of a specified operation over multiple iterations to compute the average number of CPU cycles consumed.

Parameters

in	<i>func</i>	A function pointer to the operation being measured.
in	<i>x</i>	A pointer to the first <code>bigint</code> input.
in	<i>y</i>	A pointer to the second <code>bigint</code> input.
out	<i>z</i>	A pointer to the <code>bigint</code> where the result will be stored.

Precondition

func must be a valid function pointer, and all input/output `bigint` pointers must be initialized.

Postcondition

The average CPU cycles are printed to the standard output.

Definition at line 229 of file test.c.

Here is the caller graph for this function:



4.10.1.3 `print_bi_hex_py()`

```
void print_bi_hex_py (
    IN const bigint * x )
```

Prints a `bigint` in hexadecimal format (Python-style).

This function prints the hexadecimal representation of the given `bigint` object. The format is similar to Python's hexadecimal literals, prefixed with "0x". If the number is negative, a "-" sign is prepended.

Parameters

in	x	A pointer to the <code>bigint</code> to be printed.
----	---	---

Precondition

`x` must point to a valid `bigint` object with an initialized array `a` and word length.

Postcondition

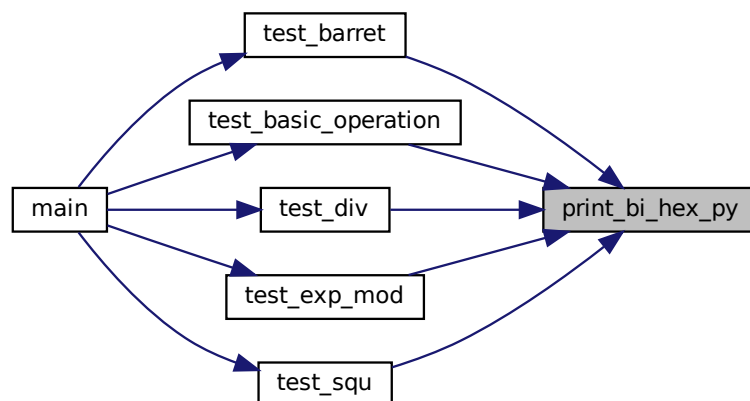
The hexadecimal representation of the `bigint` is printed to the standard output.

Note

This function handles different word lengths based on the `WORD_BITLEN` macro.

Definition at line 3 of file `test.c`.

Here is the caller graph for this function:

**4.10.1.4 test_barret()**

```
void test_barret (
    const char * operate )
```

Tests Barrett reduction for modular arithmetic.

This function verifies the Barrett reduction method for efficient modular reduction. It generates random `bigint` values and precomputes necessary constants for Barrett reduction.

Parameters

in	<i>operate</i>	A string representing the name of the operation (used in output).
----	----------------	---

Precondition

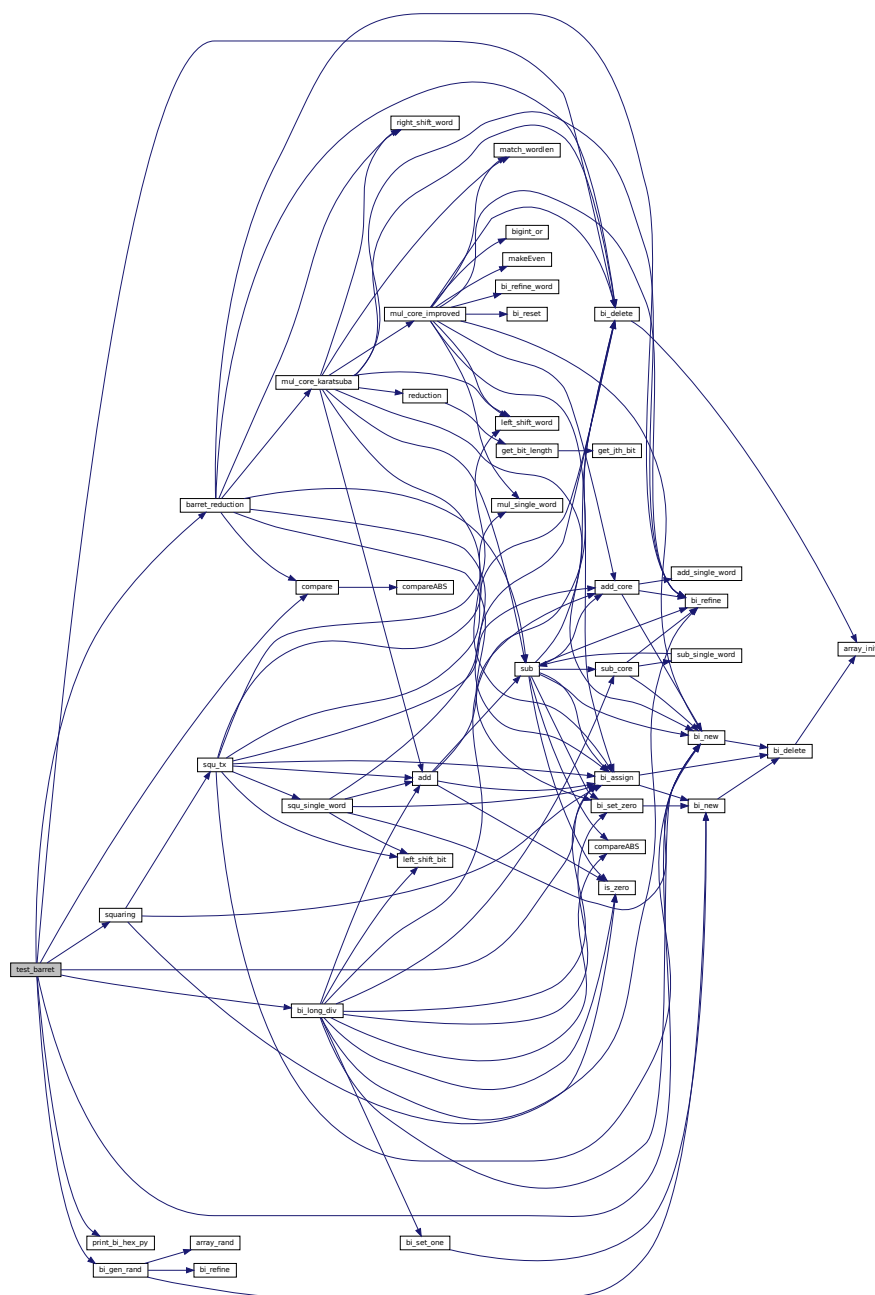
Input values must be valid `bigint` objects, and the modulus must be non-zero.

Postcondition

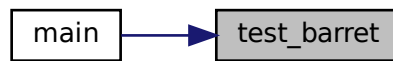
The reduction results are printed, verifying correctness.

Definition at line 165 of file test.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.5 test_basic_operation()

```

void test_basic_operation (
    void(*) (IN bigint **, IN bigint **, OUT bigint **) operation,
    const char * operator,
    const char * operate )
  
```

Tests basic arithmetic operations on randomly generated `bigint` values.

This function generates random `bigint` values, performs the specified operation, and prints the inputs and results in the format: `x [operator] y == z`.

Parameters

in	<i>operation</i>	A function pointer to the arithmetic operation being tested.
in	<i>operator</i>	A string representing the symbol of the operation (e.g., "+", "-", "*").
in	<i>operate</i>	A string representing the name of the operation (used in output).

Precondition

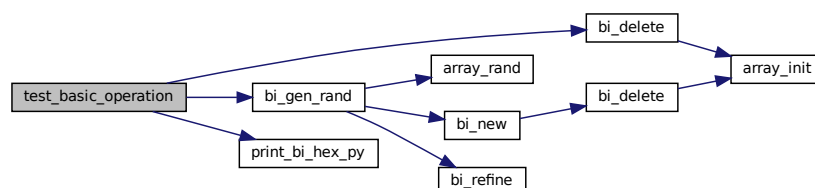
`operation` must point to a valid arithmetic function accepting input and output `bigint` pointers.

Postcondition

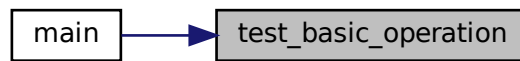
The inputs, operation, and results are printed to the standard output.

Definition at line 61 of file `test.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.6 test_div()

```
void test_div (
    const char * operate )
```

Tests division by verifying the quotient and remainder.

This function generates random `bigint` values, computes the quotient and remainder using long division, and verifies that the equation `x == q * y + r` holds.

Parameters

in	<i>operate</i>	A string representing the name of the operation (used in output).
----	----------------	---

Precondition

The `bigint` inputs must be non-negative, and the divisor `y` must not be zero.

Parameters

in	<i>operation</i>	A function pointer to the modular exponentiation function being tested.
in	<i>operate</i>	A string representing the name of the operation (used in output).

Precondition

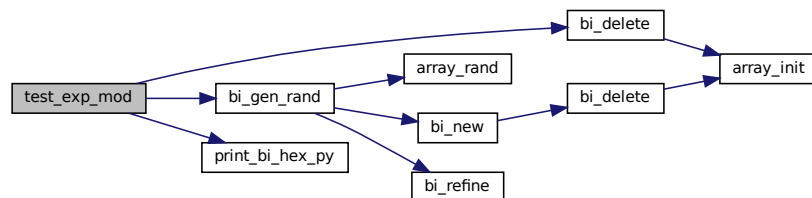
operation must point to a valid modular exponentiation function accepting input and output `bigint` pointers.

Postcondition

The inputs and the modular exponentiation result are printed to the standard output.

Definition at line 124 of file test.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.10.1.8 test_squ()

```

void test_squ (
    void(*) (IN bigint **, OUT bigint **) operation,
    const char * operate )

```

Tests the squaring operation on randomly generated `bigint` values.

This function generates random `bigint` values, computes their square using the given operation, and prints the inputs and results for verification.

Parameters

in	<i>operation</i>	A function pointer to the squaring operation being tested.
in	<i>operate</i>	A string representing the name of the operation (used in output).

Precondition

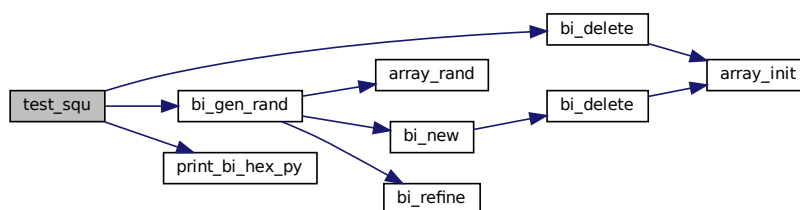
`operation` must point to a valid squaring function accepting input and output `bigint` pointers.

Postcondition

The inputs and squared results are printed to the standard output.

Definition at line 34 of file test.c.

Here is the call graph for this function:



Here is the caller graph for this function:

